THE EXPONENTIAL TIME HYPOTHESIS IS FALSE

OPENAI

Abstract. We give a deterministic algorithm for exactly counting the satisfying assignments of an arbitrary k-CNF formula. For every fixed $k \geqslant 2$, on a formula with n variables and m clauses, our algorithm uses $\mathrm{poly}_k(n, m)$ space and runs in time $(n+m)^{O_k(1)} 2^{n/(\log n)^{\frac{1}{2}+o(1)}}$. Our algorithm refutes the Exponential Time Hypothesis (ETH), which postulates that deciding 3-SAT on n variables requires $2^{\Omega(n)}$ time in the worst case.

Contents

1. Introduction 1
2. Proof of the main theorem 10
3. Proof of the contraction theorem 13
4. Encoding sparse polynomial systems 22
5. Constructing the parity tests 27
6. An interpolation algorithm: technique overview 30
7. A fast counting algorithm via shared linear summaries 40
8. Constructing low-degree replacement forms 46
9. Counting common zeros by polynomial interpolation over $\mathbb{Q}$ 55
10. Counting arbitrary formulas by interpolation 57
Appendix A. Partitioning the satisfying assignments 58
Appendix B. Exact arithmetic and space 62
Appendix C. Additional Consequences of Refuting ETH 66
References 69

1. Introduction

For every fixed $k \geqslant 3$, k-SAT is the canonical NP-complete problem [Coo71], and its exponential-time complexity underlies the Exponential Time Hypothesis (ETH) and the Strong Exponential Time Hypothesis (SETH) [IP01, IPZ01]. In this paper, we refute ETH by giving, for every fixed $k \geqslant 2$, a deterministic algorithm that counts the satisfying assignments of a k-CNF in subexponential time and polynomial space.

Write the input k-CNF on n declared variables with m input clauses as

$$F(x_1, \ldots, x_n) = \bigwedge_{e=1}^{m} \left[(x_{i_{e,1}} \oplus b_{e,1}) \vee \cdots \vee (x_{i_{e,k}} \oplus b_{e,k}) \right],$$

where $i_{e,j} \in \{1, \ldots, n\}$ and $b_{e,j} \in \{0, 1\}$ are the variable index and negation bit of the jth literal in clause e, and $\oplus$ denotes addition modulo two. Repeating a literal permits clauses with fewer than k literals. The indices $i_{e,j}$ and bits $b_{e,j}$ are fixed by F; only $x_1, \ldots, x_n$ vary over assignments. Our aim is to compute the quantity

$$\#F = \left| \{x \in \{0,1\}^n : F(x) = 1\} \right|. \tag{1.1}$$

Date: 17 October 2026, revised 25 October 2026.

Testing $\#F > 0$ also decides satisfiability. We keep k fixed as n grows. Throughout, log denotes the natural logarithm.

Theorem 1.1. *For each fixed $k \geqslant 2$, the count $\#F$ of a k-CNF F on $n \geqslant 20$ declared variables with m clauses can be computed by a deterministic algorithm using*

$$(n+m)^{O_k(1)} 2^{O_k(n \log\log n/\sqrt{\log n})}$$

bit operations and $(n+m)^{O_k(1)}$ bits of space. For each fixed $D > 0$, its running time on inputs with $m \leqslant Dn$ is

$$(n+m)^{O_{k,D}(1)} 2^{O_{k,D}(n/\sqrt{\log n})}.$$

Both bounds in Theorem 1.1 follow from a Boolean tensor contraction algorithm. A contraction sums, over one bit per wire, the product of the local tensor entries. Clause truth tables and equality constraints linking occurrences of the same variable reduce counting to contraction of a network with $O_k(n+m)$ wires, hence $O_{k,D}(n)$ wires at fixed clause density (Section 2.4). Since $0 \leqslant \#F \leqslant 2^n$, computation modulo 2^{n+1} determines the integer count. Theorem 2.1 contracts the networks used here with exponent $O(s/\sqrt{\log s})$, where s bounds their number of wires.

The contraction algorithm is recursive, and each call separates groups of vertices. Call the two sides of the bipartite network its row side and its column side, and give the row ends and the column ends of the wires independent Boolean vectors Y and X, indexed by the wires. The agreement factor is $\prod_x l_{Y_x}(X_x)$, where $l_0(z) = 1 - z$ and $l_1(z) = z$: it is one exactly when $X = Y$ and zero otherwise, so multiplying the tensor entries by it and summing over X and Y gives the contraction. A call replaces X in this product by $W(T)X$, where $W(T)$ is a matrix of polynomials in a parameter T with $W(0) = I$. To define it, Lemma 3.1 labels the vertices by strings in $[r]^a$, with coordinates numbered $0, \dots, a-1$, and places the wires into *arrays* of $L = 2^a$ *slots*, numbered $0, \dots, L-1$, with at most one wire per slot. Each wire has its row and column ends at the same slot; the parameters r and a are chosen as in Section 3.2. The matrix $W(T)$ is indexed only by the occupied row and column positions. Within an array, its entries are $W_{xy}(T) = \mathcal{C}(T)/(\mathcal{C}'(0)(T - (y - x)))$ on slots x, y, where $\mathcal{C}(T) = \prod_{t=-L}^{L-1}(T - t)$, and entries between different arrays are zero. At each nonzero integer t with $-L \leqslant t < L$, the matrix $W(t)$ is therefore supported only on pairs of slots differing by t. We select the points t whose two-adic valuation $v_2(t)$ is less than an integer parameter $1 \leqslant u \leqslant a$. The labels and slot numbers are chosen so that at a selected point t the surviving entries connect vertices in the same group, where the groups at t are the classes of coordinate $v_2(t)$ of the labels, each with few wires. Thus the deformed contraction has its original value at zero and separates at the selected points (Section 3.3).

The degree bound for the deformed contraction does not allow recovery from values at the selected points alone, so we also compute Taylor data. Moving away from a selected point reintroduces interactions between groups. Section 3.5 expands the crossing entries by a geometric series whose terms separate the row and column dependence. Successive terms have coefficients divisible by successively higher powers of two. Working modulo 2^p, we can therefore discard every product whose accumulated divisibility reaches 2^p. For each remaining monomial in the expansion, the sum over wire bits factors into contractions of smaller networks, as expressed by (3.13).

The saving depends on how much precision interpolation requires. Writing $T = t + \alpha z$ near a selected point t, with $\alpha = 2^{a+1}$, scales the local Taylor coordinate z and cancels the powers of two in the matrix-entry denominators (Lemma 3.2). We then use fewer Taylor coefficients at nodes divisible by higher powers of two. This gradual decrease bounds the precision lost to the Hermite interpolation denominators (Lemma 3.3). With M wires and u retained two-adic valuation levels, the resulting bound leaves only $2^{O(M/\sqrt{u})}$ monomials to enumerate. Each gives child networks with at most $M/8$ wires. Section 3.7 shows that the exponents decrease geometrically through the recursion, giving the exponent $O(s/\sqrt{\log s})$. The tensor entries are represented implicitly, and the recursion

is traversed depth first; the same section bounds their descriptions and the working precisions by polynomials, giving polynomial space.

For arbitrary clause density, counting-preserving sparsification [DHM+14, Appendix A] partitions the satisfying assignments among formulas on the original variables. With a parameter tending to zero, the number of leaves fits the desired time bound, but their clause counts can still be n times a polylogarithmic factor (Lemma 2.3). To obtain smaller networks, we compress each leaf into $O(n)$ sparse polynomial equations over $\mathbb{F}_2$. Each equation tests the parity of a short list of clause-violation bits. Such a test can easily miss an assignment violating very few clauses. We first apply expander-code checks [SS96, Sections IV and VI] to the violation vector, so that every nonzero vector still accepted has large support (Lemma 2.4). A short random parity test then rejects each remaining nonsatisfying assignment with constant probability. To choose tests deterministically, count the common zeros for each candidate and keep a candidate minimizing that count. Every satisfying assignment passes every candidate, so minimizing the total also minimizes the number of nonsatisfying assignments left. Each round removes a constant fraction of them, and $O(n)$ rounds remove all of them. Section 5 proves this construction, including its bounds on the number and sparsity of the equations.

The counter for these candidate systems must keep the number of wires small. Let L_i be the integer sum of the listed monomials of equation i, and put $\ell = n + 1$. The equation holds exactly when L_i is even. The complemented Beigel–Tarui modulus-amplifying polynomial [BT94, Section 2.3] represents this indicator modulo 2^ℓ, but multiplying one such polynomial per equation would give a quadratic degree bound. We write each indicator in powers of $h(Y) = Y(Y-1)$, with coefficients linear in Y. Since h is even at integer arguments, every product containing at least ℓ copies of h vanishes modulo 2^ℓ. Truncating by the *total* number of copies across all equations keeps the degree $O_k(n)$, even when the inputs of different equations are treated as distinct (Lemma 4.2). After multilinearization on Boolean inputs, the retained monomials select only $O_k(n)$ factor–variable incidences from a set of size $n(\log n)^{O_k(1)}$. Indexing these subsets in binary gives networks with $O_k(n \log\log n)$ wires (Section 4.3). Applying the contraction algorithm gives Lemma 2.2. Section 2.5 combines this counter with the parity tests and sums the disjoint leaf counts, proving the arbitrary-density bound of Theorem 1.1.

Sections 6–10 give a second counting algorithm, based on polynomial interpolation over $\mathbb{Q}$ and independent of the contraction algorithm. It proves Proposition 10.1, with the weaker exponent $O_k(\rho(n))$, where

$$\rho(n) = \frac{n(\log\log n)^2}{\sqrt{\log n}} = \frac{n}{(\log n)^{\frac{1}{2}+o(1)}}.$$

It uses the same parity tests and truncated indicator product, with a different way to sum over Boolean assignments. Replace each factor input by a linear form whose coefficients are polynomials in a parameter T. At zero these forms restore the original inputs. At selected nonzero points, fixing a few bounded integer linear measurements makes every factor depend on just one small group of variables, so the sum factors over groups (Lemma 7.3).

The difficulty is to obtain many such points while keeping the replacement degrees small. Section 8 works on auxiliary coordinates recording the variable and factor group choices. Swapping the two labels at every point gives a permutation that connects matching groups. We multiply this permutation by a low-degree polynomial in the individual swaps. Use the tensor-product basis built from the symmetric and antisymmetric eigenvectors of a single swap of two labels, with one factor per selected point. A symmetric factor has eigenvalue $+1$, and an antisymmetric factor has eigenvalue -1. Thus the complete swap acts by $(-1)^j$ on a basis vector with j antisymmetric factors. For a uniformly chosen vector from this basis, j has a binomial distribution, so interpolating these signs on a short central interval makes the resulting matrix fix all but a small tail of the basis directions. Measuring the exceptional directions determines the correction to the identity (Lemma 8.2). Low

degree allows only a few swaps to be undone, so each nonzero entry connects different groups at only a few points (Lemma 8.3). Keep the measured correction fixed and multiply each matrix entry by a polynomial equal to one at zero and vanishing at those points. This gives the desired low-degree replacement forms (Lemma 8.4).

Values at these points alone need not determine the counting polynomial. To compute its Taylor coefficients, give the changes in the replacement forms a separate formal input. Each occurrence of that input contributes at least one power of the Taylor variable, so the coefficient of order j has degree at most j in the correction variables. Boolean Möbius inversion expresses its sum as restricted sums with those variables fixed; fixing them preserves the separation into groups (Lemma 7.4 and Section 7.4). Hermite interpolation then recovers the value at zero. Section 9 applies this procedure to the sparse polynomial systems, and Section 10.2 sums the leaf counts to prove Proposition 10.1. The computations use polynomial space by processing the sums sequentially with common denominators of polynomial bit length (Appendix B).

When $\#F > 0$, the standard self-reduction finds a satisfying assignment by fixing one variable at a time and checking that the remaining count is positive. Choosing each bit according to the two conditional counts instead gives an exactly uniform random satisfying assignment, with $\mathrm{poly}_k(n, m)$ expected overhead [JVV86, Theorem 3.3(1)]. Indeed, if $C(w)$ counts satisfying extensions of a prefix w, choose its next bit b with probability $C(wb)/C(w)$. The probabilities along any satisfying assignment telescope to $1/C(\varnothing)$. This choice is exact: draw a uniform integer in $[0, C(w) - 1]$ by sampling uniform bit strings of the least sufficient length and rejecting out-of-range values, and choose zero precisely when it is smaller than $C(w0)$. Each trial is accepted with probability greater than $1/2$, and all counts have at most $n + 1$ bits.

The theorem also yields arbitrarily accurate subexponential-time approximation algorithms for Max-k-SAT. The Unique Games Theorem [Ope26, Section 8.1] implies NP-hardness bounds for Max-k-SAT that match, up to arbitrarily small error in the approximation ratio, the guarantees of the semidefinite-programming algorithm of [Rag08]. These bounds concern polynomial-time algorithms. Corollary 1.2 uses subexponential time, and the remark after it explains why hardness proved by polynomial-time reductions does not extend to such running times. Write $\mathrm{OPT}(F)$ for the maximum number of clauses of F simultaneously satisfied by an assignment.

Corollary 1.2. *For every fixed $k \geqslant 2$ and $\varepsilon > 0$, a randomized algorithm, given a k-CNF F on $n \geqslant 20$ variables with m clauses, finds an assignment satisfying at least $\mathrm{OPT}(F) - \varepsilon m$ clauses with probability $1 - 2^{-\Omega(n)}$, in*

$$(n+m)^{O_{k,\varepsilon}(1)} 2^{O_{k,\varepsilon}(\rho(n))}$$

time and $(n+m)^{O_{k,\varepsilon}(1)}$ space.

Appendix C.1 proves the corollary. The proof uses only the fixed-density bound of Theorem 1.1, for formulas with $O_{k,\varepsilon}(n)$ variables and clauses, so the interpolation algorithm of Sections 6–10 is not needed.

Running-time bounds of the form $2^{o(n)}$ do not transfer under general polynomial-time reductions. A reduction that produces formulas with n^c variables from instances of size n, for a constant $c > 1$, turns a $2^{o(n)}$ bound only into a $2^{o(n^c)}$ bound, and a $2^{n^{o(1)}}$-time algorithm would be needed to give every problem in NP a subexponential-time algorithm in this way. So our result does not imply that general NP problems admit subexponential-time algorithms. The bound does transfer through encodings with linearly many variables, and Corollary 1.3 uses such encodings to count independent sets, graph colorings and perfect matchings.

Corollary 1.3. *For simple graphs with v vertices and e edges, deterministic algorithms compute the following counts:*

- *independent sets and, for each fixed $q \geqslant 2$, proper q-colorings, in $2^{o(v)}$ time and $\mathrm{poly}_q(v, e)$ space;*

- *perfect matchings, in* $2^{o(v+e)}$ *time and* $\mathrm{poly}(v,e)$ *space.*

Appendix C.1 proves the corollary by counting-preserving CNF encodings, with v variables for independent sets, qv variables for q-colorings and $O(v+e)$ variables for perfect matchings.

The fixed-density case of Theorem 1.1 gives a circuit lower bound for E^{NP}, the class of languages decidable in deterministic time $2^{O(n)}$ with an NP oracle.

Corollary 1.4. *Let* $C_L(n)$ *be the minimum number of noninput gates in a circuit computing* L *on* $\{0,1\}^n$*, allowing arbitrary Boolean gates of fan-in at most two and unrestricted fan-out. There exist a language* $L \in \mathrm{E}^{\mathrm{NP}}$ *and a constant* $c > 0$ *such that*

$$C_L(n) > c\,n\sqrt{\log n} \qquad \text{for infinitely many } n. \tag{1.2}$$

Appendix C.2 proves the corollary. Appendix C.3 gives algorithms for inversion and exact preimage counting of Boolean maps with constant output locality.

1.1. **Related work.** We first discuss earlier SAT and counting algorithms and the complexity hypotheses contradicted by Theorem 1.1. We then compare both algorithms with earlier algebraic algorithms and identify the existing constructions they use.

1.1.1. *SAT algorithms and the exponential-time hypotheses.* Schöning's local-search algorithm and PPSZ are landmarks in the development of exponential-time algorithms for k-SAT [Sch99, PPSZ05]. Schöning's randomized algorithm starts from a random assignment and repeatedly flips a uniformly chosen variable in an unsatisfied clause. Its bounded-error decision time is

$$(n+m)^{O(1)}(2-2/k)^n$$

for each fixed $k \geqslant 3$ [Sch99, Section 2]. In particular, this gives $(n+m)^{O(1)}(4/3)^n$ time for 3-SAT. The PPSZ algorithm [PPSZ05] uses a different mechanism: bounded resolution derives additional clauses, after which variables are assigned in a random order, taking forced values when available and guessing otherwise. For $k = 3, 4$, its original analysis gave stronger bounds for uniquely satisfiable formulas than for general ones. The unique-SAT guarantees were later extended to general 3-SAT and 4-SAT; for 3-SAT this gave a randomized $O(1.308^n)$ bound [Her14]. Hansen, Kaplan, Zamir, and Zwick improved the PPSZ bound for every fixed $k \geqslant 3$ by biasing the algorithm's guesses [HKZZ19]. Scheder obtained an improvement for the same range of k by sharpening the analysis of the original PPSZ algorithm [Sch24, Theorem 1.5]. For general 3-SAT, a preprint by Jiang and Cai gives a randomized $O^*(1.307031578^n)$ bound [JC26, Corollary 1.2], where O^* suppresses factors polynomial in the input size. Liu's deterministic algorithm combines branching with derandomized local search and solves 3-SAT in $O(1.32793^n)$ time [Liu18, Theorem 21].

Randomized local search has also produced effective heuristics. The WalkSAT heuristic chooses an unsatisfied clause and flips one of its variables, combining random choices with greedy moves [SKC94]. Experiments on formulas from circuit synthesis and diagnosis demonstrated the practical value of this approach [SKC94, pp. 340–342, Tables 4–5].

Among uniformly random k-CNFs, instances at the satisfiability threshold were conjectured to be the hardest for deciding satisfiability [VW21, Section 1]; for fixed $k \geqslant 3$ and lower clause densities $m/n \leqslant 2^k/k^C$, where C is a universal constant, polynomial-time almost-uniform sampling of satisfying assignments is known with high probability over the formula [CLWYY25, Theorem 1.2]. For a uniformly random formula with polynomially many clauses, conditioned on satisfiability, Vyas and Williams [VW21, Theorem 3] give a PPZ-based algorithm that finds a solution in $2^{n(1-\Omega(\log k/k))}$ time with high probability over both the formula and the algorithm's random choices.

The Exponential Time Hypothesis (ETH) asserted that 3-SAT requires $2^{\Omega(n)}$ deterministic time in the worst case, and hence so does k-SAT for every fixed $k \geqslant 3$ [IP01, Theorem 1]. The Strong Exponential Time Hypothesis (SETH) made the sharper prediction that k-SAT requires $2^{n(1-o_k(1))}$ time as k grows. Precisely, for every $\varepsilon \in (0,1)$, there is a fixed k for which no deterministic algorithm

solves k-SAT in $2^{(1-\varepsilon)n}(n+m)^{O(1)}$ time. The running-time bounds for Schöning's algorithm and PPSZ helped motivate this prediction: their improvements over exhaustive search diminish as k grows [IP01, Open Problems, item 2].

The Gap Exponential Time Hypothesis (Gap-ETH) strengthens ETH by postulating constants $\delta, \gamma > 0$ for which no $O(2^{\gamma n})$-time algorithm can distinguish a satisfiable 3-CNF from one in which every assignment violates at least a δ fraction of the clauses, even when $m = O(n)$ [Din16, MR17]. The randomized version rules out an algorithm with the same running time that distinguishes the two cases with probability at least $2/3$ on every promised input [CCK+20, Conjecture 2.5]. It has been used to prove hardness of approximating Label Cover [Din16] and dense constraint satisfaction problems [MR17]. It also rules out nontrivial fixed-parameter approximations for Clique and Dominating Set. For Clique, the excluded ratios are $o(\mathrm{OPT})$, where OPT is the maximum clique size [CCK+20, Corollaries 5.3 and 5.6].

The Nondeterministic Strong Exponential Time Hypothesis (NSETH) asserts that, for every $\varepsilon > 0$, some fixed k admits no nondeterministic $2^{(1-\varepsilon)n}(n+m)^{O(1)}$-time algorithm for deciding whether a k-CNF is *unsatisfiable*, or equivalently whether a k-DNF is a tautology [CGI+16]. NSETH has been used to establish barriers to deterministic fine-grained reductions from SAT to problems such as 3SUM and All-Pairs Shortest Paths [CGI+16], and to delineate SETH-based lower bounds for approximating directed graph diameter [Li21].

Theorem 1.1 refutes ETH, SETH, Gap-ETH, and NSETH. Indeed, testing whether the exact count is positive or zero decides both satisfiability and unsatisfiability in deterministic $2^{o(n)}(n+m)^{O_k(1)}$ time for every fixed k; deciding satisfiability also solves the Gap-ETH promise problem. The deterministic algorithm also refutes the bounded-error randomized versions of ETH [DHM+14, Section 1] and SETH [CDL+16, Definition 2.1], and of Gap-ETH. It does not conflict with unconditional exponential lower bounds for restricted proof systems such as regular resolution [BI13]. In 2016, Ryan Williams refuted the Merlin–Arthur version of SETH (MA-SETH) by giving a protocol that certifies k-CNF unsatisfiability in $2^{n/2}\,\mathrm{poly}(n,m)$ time [Wil16, Theorem 3.3].

1.1.2. *Exact counting and counting complexity.* In Valiant's framework, #SAT and #3-SAT are #P-complete under parsimonious reductions [Val79, Section 3, Facts 1–2]. For fixed $k \geqslant 3$, previous exact-counting algorithms achieve $\mathrm{poly}_k(n,m)2^{n-\Omega(n/k)}$ time, with a Las Vegas expected-time bound in [IMP12, Lemma 3.5] and a deterministic bound in [CW21, Theorem 1.6]. For #2-SAT, Wahlström's algorithm counts satisfying assignments in $O^*(1.2377^n)$ time [Wah08]. For #3-SAT, Kutzkov uses complementary branching to count satisfying assignments in $O^*(1.637^n)$ time [Kut26]. Both algorithms are deterministic.

The counting Exponential Time Hypothesis (#ETH) asserted that exactly counting the satisfying assignments of a 3-CNF requires $2^{\Omega(n)}$ deterministic time [DHM+14, Section 1]. Under #ETH, Curticapean [Cur18] rules out deterministic $2^{o(v)}$-time algorithms for counting perfect matchings even in unweighted bipartite graphs with v vertices and $O(v)$ edges. The perfect-matching algorithm in Corollary 1.3 applies in particular to these graphs. Our algorithm (Theorem 1.1) solves #3-SAT deterministically in $2^{o(n)}\,\mathrm{poly}(n,m)$ time and therefore refutes #ETH.

1.1.3. *Implications for fine-grained complexity.* ETH and SETH became standard assumptions for parameterized and fine-grained complexity [IPZ01, VW19]. ETH-based reductions yielded lower bounds for the dependence on solution size in Clique [CHKX06], and on treewidth in constraint satisfaction and subgraph embeddings with prescribed vertex classes [Mar10]. SETH supported sharp exponential bounds for Hitting Set and Set Splitting [CDL+16], and for Independent Set and Dominating Set parameterized by treewidth [LMS18]. It also gave bounds on possible speedups of pseudopolynomial algorithms for Subset Sum and finding a path subject to two additive weight bounds [ABHS19].

For problems in P, SETH was used to rule out polynomial-factor speedups over quadratic running times for edit distance [BI18], longest common subsequence and dynamic time warping [BK15], Fréchet distance [Bri14], and regular-expression matching [BI16]. Other applications concerned approximating graph diameter [RV13] and maintaining reachability counts or the size of the largest strongly connected component under graph updates [AW14]. Together these results made SETH a common basis for explaining the apparent optimality of algorithms across quite different problems.

Our refutation of ETH and SETH removes one basis for the hardness claims above. Some of the same lower bounds follow from weaker hypotheses that may still hold. For example, the lower bounds for longest common subsequence and regular-expression matching follow from the Orthogonal Vectors Conjecture (OVC) [BK15, BI16]. OVC asserts that for every fixed $\varepsilon > 0$, no $O(N^{2-\varepsilon}\operatorname{poly}(d))$-time algorithm can decide whether two sets of N Boolean vectors of dimension d contain an orthogonal pair, one vector from each set. The reduction from SAT to Orthogonal Vectors shows that SETH implies OVC [Wil05, VW19]. The lower bounds for Fréchet distance, graph diameter approximation, and the treewidth dependence of Independent Set and Dominating Set also follow from the hypothesis that unrestricted CNF-SAT admits no $2^{(1-\delta)n}\operatorname{poly}(n,m)$-time algorithm for any constant $\delta > 0$ [Bri14, RV13, LMS18]. Theorem 1.1 gives a running-time guarantee for each fixed clause width k, while this hypothesis allows the width to grow with n.

To obtain a faster algorithm for a target problem from our SAT algorithm, we need a reduction in the other direction, from that problem to fixed-width SAT, that controls the number of variables. Set Splitting with sets of size at most k has such a reduction [CDL+16, Observation 3.6]: use one Boolean variable per element to specify its side of the split. For each set, one clause requires at least one of its variables to be true, and a second requires at least one to be false. The resulting k-CNF has one variable per universe element, so for fixed k Theorem 1.1 gives subexponential time in the universe size, up to a polynomial input-size factor.

Theorem 1.1 invites a renewed search for algorithms for problems whose known hardness results relied on ETH or SETH. The examples above suggest both immediate consequences and problems for which new algorithmic ideas are needed.

1.1.4. *Prior work on algebraic algorithms for combinatorial problems.* Both of our algorithms are algebraic. Each deforms the counting sum by a polynomial parameter so that, at selected points, it becomes a sum of products of sums over small groups, and each recovers the count at the origin by Hermite interpolation. The contraction algorithm of Sections 2–5 deforms the identification of the two sides of a tensor network (Section 3.3); the algorithm using polynomial interpolation over $\mathbb{Q}$ of Sections 6–10 replaces the inputs of the factors by linear forms with polynomial coefficients (Section 6.3). Below we discuss prior work on using algebraic methods to design algorithms for combinatorial problems, and name in each comparison the algorithm it concerns. For broader background, see the surveys of Koutis and Williams [KW16] on polynomial evaluations used to detect combinatorial structures, Williams [Wil14] on the polynomial method in algorithm design, and Husfeldt [Hus11] on algorithmic inclusion–exclusion.

Markov and Shi evaluate a tensor network by successive tensor contractions. Along a fixed order this takes $O(T\exp(O(d)))$ time for a network of size T, where d is the maximum rank of all tensors appearing in the process [MS08, Proposition 3.6], stated there for the networks of quantum circuits. In Section 4 they use single-edge contractions, retaining loops, and define the contraction complexity of a graph as the minimum, over edge orders, of the maximum degree of a newly merged vertex. They prove that this graph parameter equals the treewidth of the line graph [MS08, Definition 4.1 and Proposition 4.2]. The contraction algorithm of Section 3 uses no contraction order: under the degree and entry hypotheses of Theorem 2.1, its exponent $O(s/\sqrt{\log s})$ depends only on the number of wires.

Williams's split-and-list algorithm [Wil05] partitions the variables of a 2-CSP into three groups, enumerates partial assignments, and combines them using fast matrix multiplication. Koivisto [Koi06]

gives a direct sum–product formulation. The algorithm of Sections 7–10 likewise enumerates assignments within groups. Fixing the shared measurements $\vec{y}$ makes their contributions multiply independently. Scott and Sorkin [SS09, Section 6] extend split-and-list to polynomial-valued weights and arbitrary rings. The summation routine of Section 7 similarly uses truncated polynomial rings to retain several coefficients in one computation.

Björklund [Bjö16, Sections 2–3] uses inclusion–exclusion and polynomial interpolation to reduce permanent and directed Hamiltonian-cycle counting to a weighted sum over smaller instances. He adds the weights of identical instances in a table, then solves each distinct instance with nonzero total weight once. Björklund, Kaski, and Williams [BKW19] use low-degree curves and generalized Kakeya sets to recover polynomial values from evaluations on a small structured set, yielding faster algorithms for fermionants, including the permanent of integer matrices whose entries are bounded in absolute value by a fixed constant. The polynomial replacements of the algorithm using polynomial interpolation over $\mathbb{Q}$ serve a similar purpose: they connect the desired count to structured evaluations from which it can be reconstructed. Here the useful structure is separation into groups after fixing a few shared measurements.

Bhargava, Ghosh, Kumar, and Mohapatra [BGKM23, Sections 3.1–3.2] evaluate multivariate polynomials over finite fields along low-degree curves, using Hasse derivatives and Hermite interpolation when values alone do not suffice. For a univariate restriction g, the order-j Hasse derivative at t is the coefficient of z^j in $g(t+z)$. Both of our algorithms use the same recovery principle: compute several Taylor coefficients at each point admitting fast summation, then recover the original count by Hermite interpolation.

Woodruff and Yekhanin [WY07] use values and first derivatives along a line to recover a target value in private information retrieval. Kopparty, Saraf, and Yekhanin [KSY14] use higher-order Hasse derivatives for local correction of multiplicity codes, extending the line-based recovery of Reed–Muller codes. In the algorithm using polynomial interpolation over $\mathbb{Q}$, the matrix construction of Section 8 supplies low-degree replacement forms that let us compute the Taylor coefficients at the interpolation points by fast summation after fixing a few shared measurements.

1.1.5. *Antecedents of the matrix construction.* The algorithm using polynomial interpolation over $\mathbb{Q}$ of Sections 6–10 evaluates sums of products of polynomials over Boolean assignments. The polynomials share variables, so these sums do not factor into independent smaller sums. We introduce linear substitutions, depending polynomially on an interpolation parameter, that make such a factorization possible after fixing a few integer linear measurements. An auxiliary square rational matrix M_0 underlies these substitutions: a bound on $\operatorname{rank}(I - M_0)$ limits the number of measurements, while restrictions on the nonzero entries of M_0 keep the substitution degrees small. Section 6.7 explains this connection.

Related interpolation methods have been used to obtain low-rank approximations to structured matrices. In [AW17, Section 3], a low-degree polynomial matches $(-1)^j$ on an interval of integer inner products j, using the interpolation lemma of [AW15, Lemma 3.1]. This gives low-rank matrices agreeing with Walsh–Hadamard matrices in most entries, a result in the study of matrix rigidity [Val77]. Related interpolation gives rigidity bounds for Kronecker powers in [AGP23, Theorem 6.1]. Dvir and Liu [DL20, Claim 2.20 and Lemmas 3.3–3.4] use a spectral approach: sparse changes to a function invariant under permutations of its coordinates make many Fourier coefficients vanish, reducing the rank of its group-circulant matrix.

Our construction also interpolates $(-1)^j$ on a short interval, now to obtain a polynomial in commuting permutation matrices. The same polynomial gives both the rank bound for $I - M_0$ and the restrictions on the nonzero entries of M_0. We prove these properties and their quantitative bounds directly in Section 8.

1.1.6. *Sparsification and parity tests.* We use the counting-preserving version [DHM$^+$14, Appendix A] of the sparsification lemma [IPZ01], with quantitative bounds from [CIP06, Section 6] and [Cal09,

Lemma 5.14 and equation (5.4)]. This partitions the satisfying assignments among sparse formulas on the same Boolean variables. Appendix A records the disjoint branching argument and the bounds needed when its parameter varies with the input size.

The initial parity tests are the constraints of a Tanner code [Tan81, Section II], applied to the binary vector recording which clauses an assignment violates. We use the edge-based expander-code construction of [SS96, Sections IV and VI]: each graph edge carries one bit, and each vertex requires its incident bits to belong to a fixed binary linear code. The local code's minimum distance, together with graph expansion, forces every nonzero vector satisfying all these constraints to have large support. This construction and distance argument are standard. Lemma 2.4 supplies the parameter choices needed here, including the bounds on the number and support size of the tests, and constructs the graph and local code by deterministic exhaustive searches. The graph has two copies of a Cayley multigraph's vertices and the same adjacency matrix between the copies. The shifts defining that multigraph nearly balance every nonconstant parity on the base group: this is the small-bias condition [NN93, AR94], stated explicitly in (5.1). Fixing unused edge bits to zero gives a code on exactly m coordinates. Additional parity tests exclude the remaining nonzero violation vectors. We choose these tests by the averaging principle underlying the method of conditional expectations [Rag88], using exact counts to evaluate each candidate. Section 5 gives the constructions and quantitative bounds needed here.

1.2. **Organisation.** Sections 2–5, with Appendix A and Lemma B.1, contain the complete proof of Theorem 1.1. Section 2 gives the two deductions of Theorem 1.1. Section 3 proves the contraction theorem, Section 4 encodes sparse polynomial systems, and Section 5 constructs the parity tests. Appendix A proves disjoint sparsification, and Lemma B.1 bounds the coefficients of the linear coefficient polynomials used in Section 4.

The proof of Theorem 1.1 does not use Sections 6–10. These sections develop the alternative algorithm using polynomial interpolation over $\mathbb{Q}$, which sums over Boolean assignments by a different method and proves the weaker bound of Proposition 10.1. It reuses Sections 4.1–4.2, Lemma 2.3 and the parity tests of Section 5, but not the contraction theorem. Section 6 first works out the method for 3-CNFs in which every variable occurs at most three times. The rest of Appendix B gives the rational bit estimates for this algorithm, and Appendix C proves the additional consequences.

1.3. **Notation.** For an integer $s \geqslant 0$, write $[s] = \{1, \dots, s\}$, with $[0] = \varnothing$. For a condition E, the scalar $\mathbb{1}_{\{E\}}$ equals one if E holds and zero otherwise. We use natural logarithms, writing $\log_2$ when the base is two. For complexity bounds, $\mathrm{poly}(t_1, \dots, t_s)$ denotes a bound of the form $(1+t_1+\cdots+t_s)^{O(1)}$, with absolute implicit constants. Subscripts, as in $\mathrm{poly}_{k,c}(t_1, \dots, t_s)$, allow those constants to depend on the indicated fixed parameters. Integers are stored in signed binary notation, so an integer z uses $O(\log(|z|+2))$ bits. Rational numbers are stored as an integer numerator and a positive integer denominator; stated bit-length bounds apply to each of them. Space is measured in bits. Arithmetic-operation counts are distinguished from bit-operation counts.

For a finite set S, vectors in $\mathbb{Q}^S$ have coordinates indexed by S and are column vectors unless indicated otherwise. For $U \subseteq S$, the indicator vector $\mathbf{1}_U \in \{0,1\}^S$ has entries $(\mathbf{1}_U)_v = \mathbb{1}_{\{v \in U\}}$ for $v \in S$. We write $\vec{0}$ for the zero vector. For $v \in S$, the row $\delta_v \in \mathbb{Q}^{1 \times S}$ selects coordinate v, so $\delta_v x = x_v$. A vector indexed by $U \subseteq S$ is extended to S by putting zero in each coordinate of $S \setminus U$.

We write $\vec{X} = (X_1, \dots, X_n)$ for the vector of formal variables and use arrows for polynomial coefficient vectors such as $\vec{P}_{f,v}(T)$. For a coefficient row $\vec{p} = (p_i)_{i \in S}$ and a vector $z = (z_i)_{i \in S}$, we write their scalar pairing as $\langle \vec{p}, z \rangle = \sum_{i \in S} p_i z_i$. In particular, $\langle \vec{P}_{f,v}(T), \vec{X} \rangle$ is a scalar linear form in the variables X_i.

For a polynomial $P(\vec{X}, z)$ in the variables $\vec{X}$ and a variable z, $[z^j]P$ denotes the coefficient of z^j, viewed as a polynomial in $\vec{X}$, and $\deg_{\vec{X}} P$ denotes total degree in $\vec{X}$. The degree of a row of

polynomials is the largest degree of any entry. Unless otherwise specified, polynomial coefficients are rational and matrix kernels are taken over $\mathbb{Q}$.

2. Proof of the main theorem

We deduce both bounds of Theorem 1.1 from Boolean tensor contraction. A formula of fixed clause density has a network with $O_{k,D}(n)$ wires. For arbitrary density, disjoint sparsification and parity tests reduce the count to sparse polynomial systems, whose networks have $O_k(n \log\log n)$ wires. We state the needed outputs first and give the two deductions in Sections 2.4 and 2.5.

2.1. **Boolean tensor contraction.**

A Boolean tensor network is a bipartite multigraph G in which each edge, also called a wire, carries a single bit. Each vertex v has an ordered list $\delta(v)$ of incident edges and a tensor T_v assigning a ring element to each Boolean assignment to those edges. Parallel edges have distinct edge ends. The *contraction* of the network is

$$Z = \sum_{x \in \{0,1\}^{E(G)}} \prod_{v \in V(G)} T_v(x|_{\delta(v)}).$$

An isolated vertex contributes its scalar entry. We measure the size of the network by its number of edges.

Theorem 2.1 (Boolean tensor contraction)**.** *Fix positive constants C, A, b. Let $s \geqslant 2$ be an integer, and let $\mathcal{L}$ be the total length of the graph and tensor descriptions of a bipartite tensor network with at most Cs edges and maximum degree at most $\lceil s^{9/10} \rceil$. Let P_0, ℓ be positive integers. Suppose that the number of vertices is bounded by a fixed polynomial in s, and that the entries belong to*

$$\mathcal{B}_{P_0,\ell} = (\mathbb{Z}/2^{P_0}\mathbb{Z})[V]/(V^{\ell}), \qquad 1 \leqslant P_0, \ell \leqslant Cs.$$

Suppose, uniformly for every vertex and Boolean choice of its edge ends, that its entry can be computed from the description in time $(s+\mathcal{L})^b 2^{As^{9/10}}$ and space $(s+\mathcal{L})^b$. Then the contraction can be computed deterministically in

$$(s+\mathcal{L})^{O(1)} 2^{O(s/\sqrt{\log s})}$$

bit operations and $(s+\mathcal{L})^{O(1)}$ bits of space. Constants depend only on the fixed bounds in the hypotheses.

The proof is in Section 3. At recursion depth d, a call that is not a leaf has $M = O(s8^{-d})$ wires and $u = \Theta((\log s)/2^d)$ retained two-adic valuation levels. Interactions between groups are expanded in auxiliary variables representing linear forms in column bits. Only $2^{O(M/\sqrt{u})}$ monomials in these auxiliary variables survive the working precision, and each factors into contractions with at most $M/8$ wires. The exponents therefore sum geometrically to $O(s/\sqrt{\log s})$.

2.2. **Counting sparse polynomial systems.**

The following consequence of contraction provides the counter needed for every candidate system in the reduction, including candidates that still accept nonsatisfying assignments.

Lemma 2.2 (Counting sparse polynomial systems by contraction)**.** *Fix integers $k, c \geqslant 1$. Given polynomials $P_1, \dots, P_a$ over $\mathbb{F}_2$ on $n \geqslant 20$ declared Boolean variables, with $a \leqslant cn$ and each P_i listed as at most $\lceil \log_2 n \rceil^c$ monomials of degree at most k, the number of common zeros in $\{0,1\}^n$ is computable deterministically in $n^{O_{k,c}(1)} 2^{O_{k,c}(n \log\log n/\sqrt{\log n})}$ bit operations and $n^{O_{k,c}(1)}$ bits of space.*

The proof is in Section 4.

2.3. **Sparsification and parity tests.** Sparsification supplies formulas with few occurrences per variable, and the parity tests compress each leaf into $O(n)$ equations.

Lemma 2.3. *Fix $k \geqslant 2$. Let F be a k-CNF on n declared variables with m clauses, and let $0 < \sigma \leqslant 1$ be rational. Write L_σ for the total binary encoding length of its supplied numerator and denominator. A deterministic algorithm constructs a tree of k-CNFs on the same variables with the following properties. An assignment satisfies the input exactly when it satisfies one leaf formula, and it cannot satisfy two distinct leaves. The tree has*

$$\operatorname{poly}_k(n, \sigma^{-1})\, 2^{\sigma n}$$

nodes and depth $\operatorname{poly}_k(n, \sigma^{-1})$. In every leaf, each variable occurs at most B times, where B is an explicitly computed integer satisfying

$$B = O_k\Big(\big[\sigma^{-1}\log(2/\sigma)\big]^{k-1}\Big).$$

Input normalization uses $\operatorname{poly}_k(n, m)$ bit operations. Constructing the numerical parameters uses $\operatorname{poly}_k(L_\sigma)$ bit operations. Thereafter, processing each node uses $\operatorname{poly}_k(n, \log(2/\sigma))$ bit operations.

Thus the input count is the sum of the leaf counts. Appendix A proves the lemma, including the disjoint branching rule and the bounds for a varying rational parameter.

For a system $\mathcal{P}$ of equations $P_i(x) = 0$ on n Boolean variables, write

$$N(\mathcal{P}) = \#\{x \in \{0,1\}^n : P_i(x) = 0 \text{ for every } i\}. \tag{2.1}$$

Let F be a k-CNF on n declared variables with $m \geqslant 1$ indexed clauses, and let $v_e(x)$ be the violation bit of clause e. As a polynomial over $\mathbb{F}_2$, v_e is a product of the factors $1 + x_v$ for positive literals and x_v for negative literals. A tautological clause has $v_e = 0$ and an empty clause has $v_e = 1$. After removing repeated literals, $\deg v_e \leqslant k$ and v_e expands to at most 2^k monomials. Put $v(x) = (v_1(x), \dots, v_m(x))$. For a binary matrix A with m columns, row i specifies the equation

$$P_i(x) := \sum_{e : A_{ie} = 1} v_e(x) = 0 \qquad \text{over } \mathbb{F}_2. \tag{2.2}$$

Write $\mathcal{P}_A$ for this system and $N(A) = N(\mathcal{P}_A)$ for its number of common zeros. Every satisfying assignment passes every row. Hence $N(A) = \#F + D(A)$, where $D(A)$ counts the nonsatisfying assignments still passing. We do not know $\#F$, but it is the same for every candidate next row: the row minimizing N also minimizes D.

We represent a test row by a list of pairs (e, c) with $e \in [m]$ and $c \in \{0, 1\}$, summing coefficients modulo 2 if a clause index is repeated. The length of this list counts all pairs, including repeated indices and pairs with coefficient zero. A row of list length s thus reads at most ks variables before cancellation.

The initial tests exclude every nonzero violation vector with small support. They will leave only assignments that a short additional test catches with constant probability.

Lemma 2.4. *Let n, m, B be positive integers with $m \leqslant Bn$, and put*

$$\mu = \lfloor \log_2 m \rfloor + 1, \qquad P = 20000B^2\mu^2, \qquad D_0 = 320000B^2\mu^3.$$

There is a deterministic construction of a binary matrix $A^{(0)}$ with m columns and at most $3n$ rows such that every row has at most D_0 nonzero entries and

$$z \in \{0,1\}^m \setminus \{\vec{0}\}, \quad A^{(0)}z = \vec{0} \pmod 2 \quad \Longrightarrow \quad |\operatorname{supp} z| \geqslant m/P.$$

The construction uses $\exp(\operatorname{poly}(B, \log(m+2)))$ bit operations and $\operatorname{poly}(n, m, B)$ bits of working space.

Lemma 2.5. *Let F be a k-CNF on n declared variables with $1 \leqslant m \leqslant Bn$ indexed clauses, and take $P, D_0, A^{(0)}$ from Lemma 2.4. Using common-zero counts of the systems (2.2), a deterministic procedure extends $A^{(0)}$ by $3n$ rows of list length P to a matrix A satisfying $N(A) = \#F$. It uses at most $3n(2m)^P$ common-zero counts. Every candidate system has at most $6n$ equations, each involving at most $2^k \max(D_0, P)$ monomials of degree at most k.*

Section 5 proves these two lemmas.

2.4. **Fixed clause density.**

Proof of the fixed-density bound in Theorem 1.1. Give each clause a vertex carrying its Boolean satisfaction table, with one edge-end for each distinct variable in the clause. A tautology may be dropped, and an empty clause gives count zero. For a variable with $d \geqslant 1$ edge ends, require their bits to agree. For $d = 1$ attach the unary tensor $(1, 1)$, and for $d = 2$ attach a binary equality tensor. For $d \geqslant 3$ take a tree with d external edge ends and trivalent equality tensors at its internal vertices. Such a tree has $d - 2$ internal vertices, and each common value on its edge ends extends uniquely to all its internal edges.

There are at most km clause edge ends. The graph has $O_k(n + m)$ edges and vertices, and maximum degree at most $\max\{k, 3\}$. Subdivide every edge once by a binary equality tensor. The old vertices and the new vertices form the two sides of a bipartite network. The two bits on a subdivided edge must be equal, so this preserves the contraction. Every satisfying assignment on the occurring variables contributes one and every other assignment contributes zero.

With $s = n + m$, apply Theorem 2.1 over $\mathbb{Z}/2^{n+1}\mathbb{Z}$, taking $P_0 = n + 1$ and $\ell = 1$. The local tables have bounded size for fixed k. The degree and precision bounds hold for all sufficiently large s, and bounded instances can be enumerated directly. If u declared variables are unused, multiply the contraction by 2^u modulo 2^{n+1}. Its least nonnegative residue is $\#F$, since $0 \leqslant \#F \leqslant 2^n$. This includes the empty formula, for which the empty network has contraction one. The time is $s^{O_k(1)} 2^{O_k(s/\sqrt{\log s})}$ and the space is polynomial. When $m \leqslant Dn$, we have $n \leqslant s \leqslant (1 + D)n$, giving exponent $O_{k,D}(n/\sqrt{\log n})$. □

2.5. **Arbitrary clause density.**

Proof of the arbitrary-density bound in Theorem 1.1. Normalize the formula by deleting repeated literals, tautological clauses, duplicate clauses, and clauses containing another clause. Retain all n declared variables. Empty clauses and formulas with no clauses have immediate answers. For $n \geqslant 20$ apply Lemma 2.3 with $\sigma = 1/\lceil\sqrt{\lambda}\rceil$, where $\lambda = \lceil \log_2 n \rceil$. The tree has $n^{O_k(1)} 2^{O(n/\sqrt{\lambda})}$ nodes and polynomial depth. Every leaf has occurrence bound $B = O_k((\sigma^{-1} \log(2/\sigma))^{k-1}) = \lambda^{O_k(1)}$ and at most Bn nonempty clauses.

For a nontrivial leaf with m_G clauses, use the initial parity checks of Lemma 2.4 and the greedy procedure of Lemma 2.5. Each initial row has support size at most D_0, and each additional row has list length P. Both bounds are $\lambda^{O_k(1)}$. By (2.2), every row gives a degree-at-most-k equation with at most $2^k \max\{P, D_0\}$ monomials, and every queried system has at most $6n$ equations. A single constant $c = c(k)$ bounds both the ratio of equations to variables and the exponent of λ in the monomial bound for all sufficiently large n. At each greedy step, use Lemma 2.2 to compute $N(A; h_\gamma)$ for every candidate row h_γ, where h_γ is the row represented by the list γ and $A; h_\gamma$ is A with that row appended. Append a candidate attaining the minimum. The evaluator applies also to discarded candidates whose systems still accept nonsatisfying assignments. One final call gives $N(A)$ for the selected matrix, which equals the leaf count by Lemma 2.5.

Constructing the initial checks and enumerating candidates costs $2^{\lambda^{O_k(1)}}$ bit operations apart from the evaluations. There are at most $3n(2m_G)^P + 1 = 2^{\lambda^{O_k(1)}}$ evaluations per leaf. Since every fixed power of λ is $o(n \log\log n/\sqrt{\log n})$, these costs and the number of leaves are absorbed in the claimed

time bound

$$(n+m)^{O_k(1)}2^{O_k(n\log\log n/\sqrt{\log n})}.$$

Here m counts the input clauses. Traverse the tree depth first and process one query at a time. The leaf solution sets are disjoint, so their exact counts add with an $(n+1)$-bit accumulator. Together with polynomial space per query and per tree path, this gives the asserted polynomial-space bound. □

A single algorithm attains both bounds. With $\lambda = \lceil \log_2 n \rceil$ and $\nu = \lceil \log_2 \lambda \rceil$, use the clause network when $m \leqslant n\nu$ and the parity queries otherwise. In the first branch $n \leqslant s = n + m = O(n \log\log n)$, so its exponent is $O_k(n \log\log n/\sqrt{\log n})$ even at growing density. For each fixed D and all sufficiently large n, every input with $m \leqslant Dn$ takes that branch with $s = O_D(n)$.

3. Proof of the contraction theorem

Theorem 2.1 is proved by recursion on the number of wires. Lemma 3.1 balances the edge ends; Lemma 3.2 permits evaluation over the coefficient ring; and Lemma 3.3 recovers each call's answer from its local evaluations. The identity (3.13) expresses each evaluation as a sum of products of contractions with fewer than one eighth as many wires. Section 3.7 combines these facts and bounds the complete recursion.

A recursive call receives a bipartite network with M edges and maximum degree at most the bound Δ of Section 3.2, whose entries lie in the coefficient ring modulo 2^p for a precision p, together with its depth d in the recursion; it returns the contraction modulo 2^p. The root call has $d = 0$ and $p = P_0$. A call with $M \leqslant H_{\text{leaf}}$ edges, for the threshold H_{leaf} of Section 3.2, is a *leaf* and sums directly over all Boolean edge assignments. Every other call is *nonleaf* and performs the following steps, in which the array length L and the number u of retained two-adic levels are those of (3.1), and the grid S and the scaling α are those of Section 3.3.

(1) Lift its entries using least nonnegative coordinate representatives (Section 3.1).
(2) Label the vertices and place the edges into arrays (Lemma 3.1).
(3) Write the contraction as the value $\mathcal{D}(0)$ of the deformed contraction $\mathcal{D}(T)$ in (3.5), with the polynomial matrix $W(T)$. Call $t \in S \setminus \{0\}$ a *selected node* if $v_2(t) < u$; these are the nodes of positive multiplicity q_t in Lemma 3.3. At a selected node t, the matrix $W(t)$ joins only row and column edge ends of vertices whose labels from the previous step agree in coordinate $v_2(t)$; the r classes of this coordinate are the groups.
(4) For each selected node t and each of $M(2L-1)+1$ interpolation values ζ, compute $\mathcal{D}(t+\alpha\zeta)$ modulo 2^{p_t} by (3.13): the entries of W joining different groups are expanded in auxiliary variables (Section 3.5), and each surviving tuple of exponents of the auxiliary variables contributes the product of the contractions of r child networks with at most $M/8$ edges each (Section 3.6). The children are calls at depth $d+1$ with precision p_t.
(5) For each selected node t, recover $\mathcal{D}(t+\alpha z)$ modulo 2^{p_t} from these values by Lagrange interpolation; then recover $\mathcal{D}(0)$ modulo 2^p by Lemma 3.3.

The recovery in the last step divides by powers of two, so the children need the precision $p_t > p$. This is why each call first fixes lifts of its entries, which it receives only modulo 2^p.

3.1. **The coefficient ring and exact arithmetic.** The recursion recovers polynomials in an interpolation variable modulo a power of two from their values at fewer than s^4 points (Section 3.4). Interpolation modulo 2^p needs points whose pairwise differences are units, and modulo two the integers supply only two such points. We therefore work with polynomials in a root U of a polynomial f of degree e that is irreducible modulo two, with $2^e \geqslant s^4$. Write $\lambda = \lceil \log_2 s \rceil$ and $e = 4\lambda$. Choose a monic polynomial $f \in \mathbb{Z}[U]$ of degree e, with coefficients in $\{0,1\}$ below its leading coefficient, whose reduction modulo two is irreducible. This can be done deterministically in polynomial time in s by enumerating the 2^e candidates and testing divisibility over $\mathbb{F}_2$ by every monic polynomial of positive degree at most $e/2$. A reducible degree-e polynomial has a factor in this range. Both

enumerations have $2^{O(e)} = s^{O(1)}$ terms. Finite fields admit an irreducible polynomial of every positive degree [vzGG13]; specifically, the number of monic irreducibles of degree e over $\mathbb{F}_2$ is at least $2^e/(2e) > 0$ [Sho09, Theorem 19.12]. Put

$$\mathcal{O} = \mathbb{Z}_{(2)}[U]/(f), \qquad \mathcal{K} = \mathbb{Q}[U]/(f), \qquad \mathcal{A} = \mathcal{O}[V]/(V^\ell), \qquad \mathcal{A}_p = \mathcal{A}/(2^p).$$

Here $\mathbb{Z}_{(2)}$ consists of rational numbers with odd denominator. Irreducibility modulo two implies irreducibility over $\mathbb{Q}$, so $\mathcal{K}$ is a field. For a nonzero element of $\mathcal{K}$, let v_2 be the minimum of the ordinary two-adic valuations of its coordinates in $1, U, \ldots, U^{e-1}$. Reduction of $\mathcal{O}$ modulo two is the field $\mathbb{F}_{2^e}$: we have $\mathcal{O}/2\mathcal{O} \cong \mathbb{F}_2[U]/(\bar{f})$, where $\bar{f}$ is the reduction of f modulo two, and this quotient is a field with 2^e elements because $\bar{f}$ is irreducible of degree e. Now write $x = 2^\alpha x_0$ and $y = 2^\beta y_0$, where $x_0, y_0 \in \mathcal{O}$ have nonzero reductions modulo two. Their product has nonzero reduction in this field, so $v_2(x_0 y_0) = 0$. Thus

$$v_2(xy) = v_2(x) + v_2(y) \qquad (x, y \neq 0).$$

Every element of $\mathcal{O}$ that is nonzero modulo two is a unit because the determinant of its multiplication matrix is invertible in $\mathbb{Z}_{(2)}$, so the inverse matrix has entries in that ring.

Arithmetic in $\mathcal{A}_p$ takes polynomial time and space in s, p, ℓ. We use the bases $1, U, \ldots, U^{e-1}$ and $1, V, \ldots, V^{\ell-1}$, and represent each coordinate by its integer residue between zero and $2^p - 1$. Inverting a scalar unit can be done by inversion modulo two followed by Newton lifting, or by its multiplication matrix, using exact polynomial and integer operations [vzGG13]. For the lifting method, if $ab \equiv 1$ modulo 2^j, put $b' = b(2 - ab)$. The identity

$$1 - ab' = (1 - ab)^2$$

gives an inverse modulo 2^{2j} in the same coefficient ring. Starting with the inverse in $\mathbb{F}_{2^e}$ and doubling the precision therefore takes $O(\log(p+1))$ steps. This is the prime-power inversion iteration of [BZ10, Section 2.5, equation (2.3)]; the displayed identity also proves it for $\mathcal{O}/(2^p)$. Divisibility and precision for elements of $\mathcal{A}$ are always coordinatewise. We set $v_2(0) = +\infty$ and, for an element of $\mathcal{K}[V]/(V^\ell)$, use the minimum valuation of its coefficients in V.

The binary-coordinate lifts of the 2^e residue-field elements give interpolation points with pairwise unit differences. Lagrange interpolation at any required number of these points is therefore valid over $\mathcal{A}_p$. Explicitly, for such points $\zeta_1, \ldots, \zeta_N$ and $G \in \mathcal{A}_p[z]$ of degree less than N,

$$G(z) = \sum_{j=1}^{N} G(\zeta_j) \prod_{h \neq j} \frac{z - \zeta_h}{\zeta_j - \zeta_h}.$$

Every denominator is a scalar unit. The evaluation matrix has Vandermonde determinant $\prod_{j<h}(\zeta_h - \zeta_j)$, also a unit, so the formula gives existence and uniqueness even with the nilpotent variable V [LQ15, Chapter III, Exercise 1(3)–(5), p. 148]. We will use fewer than s^4 points. A contraction whose initial entries belong to $\mathcal{B}_{P_0,\ell}$ belongs to that subring after computation in $\mathcal{A}_{P_0}$, because it is a polynomial in those entries.

When the precision increases, we lift each tensor entry separately. For an entry originally specified modulo 2^p, its lift in $\mathcal{A}$ uses coordinate representatives in $\{0, \ldots, 2^p - 1\}$. This lifting need not preserve addition or multiplication. Each recursive call contracts its specified tensors modulo the precision of that call. To define its deformation, it first makes this entrywise choice of lifts. All subsequent evaluations in that call refer to these same lifts. More explicitly, each tensor description is tagged with its creation precision p_c (P_0 for an original tensor). At every later precision, its entry is evaluated modulo 2^{p_c} and those same integer representatives are used. Precisions strictly increase along a recursive path, so these representatives are also the least nonnegative coordinate representatives at every later precision. In particular, an array tensor retains its creation-time representatives throughout the recursion.

The relation needed between calls is reduction, which is a ring homomorphism. If $\rho_p : \mathcal{A} \to \mathcal{A}_p$ is reduction and Z denotes contraction, then for any chosen entrywise lifts $\widetilde{T}_v$,

$$\rho_p\Big(Z((\widetilde{T}_v)_v)\Big) = Z((\rho_p(\widetilde{T}_v))_v).$$

This follows term by term from the finite sum of products defining Z. In particular, the deformation at the origin of a child call reduces to the contraction specified by its parent, while its evaluations at higher precision all use the child's fixed lifts.

3.2. **Parameters and packing into arrays.** All estimates below hold for sufficiently large s, with the threshold depending only on the fixed constants. A fixed finite range of smaller sizes is handled by direct summation. The parameter inequalities can be tested directly. Set

$$H_{\mathrm{leaf}} = \left\lceil \frac{s}{\lfloor\sqrt{\lambda}\rfloor} \right\rceil, \qquad r = 64, \qquad \varepsilon = \frac{1}{128 \cdot 1024}, \qquad \Delta = \lceil s^{9/10} \rceil.$$

A call with $M \leqslant H_{\mathrm{leaf}}$ edges is a leaf and is evaluated directly. At a nonleaf call at depth d, with the root at depth zero, u_d is the number of retained valuation levels and a_d is the number of label coordinates, also used as slot bits. The cumulative choice of a_d will absorb inherited precision in Section 3.4. Set

$$u = u_d = \left\lfloor \frac{\varepsilon\lambda}{2^d} \right\rfloor, \qquad a = a_d = 2\sum_{j=0}^{d} u_j, \qquad a_{-1} = 0, \qquad L = 2^a, \qquad D_{\mathrm{arr}} = r^{2a}. \tag{3.1}$$

We will arrange that every child has at most $M/8$ edges and maximum degree at most Δ. These assertions imply, on every nonleaf path,

$$H_{\mathrm{leaf}} < M \leqslant C s\, 8^{-d}, \qquad d \leqslant \frac{\log_2 \lambda}{6} + O_C(1).$$

Consequently

$$u = \Theta_C(\lambda/2^d), \quad \lambda^{4/5} \leqslant u \leqslant M, \quad a \leqslant 4\varepsilon\lambda, \quad L, D_{\mathrm{arr}} \leqslant s^{1/1000} \tag{3.2}$$

for all sufficiently large s. In particular, at each nonleaf call

$$M \geqslant 2D_{\mathrm{arr}}^2\Delta, \qquad M \geqslant LD_{\mathrm{arr}}, \qquad 2L \leqslant \Delta. \tag{3.3}$$

For example, the right side of the first inequality is $O(s^{9/10+2/1000}) = o(H_{\mathrm{leaf}})$. The other inequalities follow in the same way.

We call the two sides of the bipartite network its row side and its column side, and speak of row and column vertices, edge ends and tensors accordingly. In the deformation of Section 3.3 the row edge ends index the rows of a matrix and the column edge ends its columns.

The lemma below uses the following terms. Each vertex receives a label in $[r]^a$, and the ith coordinate of its label is its *symbol* in coordinate i, for $0 \leqslant i < a$. An *array* consists of L *slots*, numbered $0, \dots, L-1$, and each slot has a row position and a column position. An edge placed in a slot is attached at its row end to the slot's row position and at its column end to the slot's column position. The occupied positions are the edge ends; unused slots carry no edge ends. Within an array, the original identification of the two ends of each wire is thus the identity between the row and column positions of the same slot. For a fixed coordinate, the vertices on both sides with a given symbol in that coordinate form a *group*. At a nonzero interpolation node t, Section 3.5 uses the r groups of coordinate $v_2(t)$. The slot orientation in the second property below will make the deformation local at that node; the edge-end bound in the third property will give the child-size bound (Section 3.6).

Lemma 3.1 (Packing into arrays)**.** *For a nonleaf call with M edges, maximum degree at most Δ, and the parameters in (3.1) satisfying (3.3), the vertices on each side admit labels in $[r]^a$ and the edges admit an embedding into arrays of L slots with the following properties.*

(1) *There are at most $3M/L$ arrays. Each edge occupies the same slot at its row and column ends. Unused slots have no edge ends.*
(2) *For every array and every coordinate i, there are symbols $b \leqslant c$ such that every edge in the array has the symbols b and c, in some order, at its row end and its column end in coordinate i. If $b < c$, the row end has symbol b exactly when bit i of the edge's slot number is zero. Consequently, a row edge end and a column edge end of the array belong to vertices with the same symbol in coordinate i exactly when $b = c$ or their slot numbers differ in bit i.*
(3) *For every coordinate and every symbol, at most $3M/r$ edge ends on each side belong to vertices with that symbol.*

A deterministic algorithm constructs these labels and this embedding in polynomial time and space in $s + \mathcal{L}$.

Proof. We give each vertex a single label in $[r]^a$, so all its edge ends carry the same symbol in each coordinate. We balance the loads of complete pairs of these labels, then pad them equally to arrange the orientations by slot bits while keeping these vertex labels fixed. For each pair of labels let X_γ be the number of edges with that pair. Independent uniform vertex labels give

$$\mathbb{E}\sum_\gamma (X_\gamma - M/D_{\mathrm{arr}})^2 \leqslant 2M\Delta.$$

Indeed, expansion expresses this expectation as a sum over ordered pairs of edges of their label-agreement probability minus $1/D_{\mathrm{arr}}$. Pairs with disjoint endpoints contribute zero. At most $2M\Delta$ ordered pairs share an endpoint, including equal edges, and each contributes at most one. Conditional expectation gives a labeling with this bound. To implement it, try each of the r^a labels of the next vertex and compute the conditional expectation by enumerating pairs of edges. Each probability concerns at most four vertices, so this is an exact polynomial-time calculation. This is the usual conditional-expectation method [Rag88].

Every load is at most $M/D_{\mathrm{arr}} + \sqrt{2M\Delta}$. Pad all pair-label classes to a common number H_{pad} of slots, the maximum load rounded up to a multiple of L. The assumptions imply

$$H_{\mathrm{pad}} \leqslant M/D_{\mathrm{arr}} + \sqrt{2M\Delta} + L \leqslant 3M/D_{\mathrm{arr}}.$$

Fix the unordered pair of symbols in each coordinate. If a' coordinates have unequal symbols, their orientations give $2^{a'}$ pair labels. Partition their padded slots into arrays using $2^{a-a'}$ slots from each of these labels per array. This is possible because H_{pad} is divisible by L. Number an array's positions from 0 to $L-1$. Its ith bit specifies the orientation when the ith pair is unequal and the copy when the pair is equal. For an unequal pair $\{b, c\}$ with $b < c$, use row symbol b and column symbol c at slots whose ith bit is zero, and the reverse symbols at slots whose ith bit is one. Thus a row slot and a column slot have equal symbols in this coordinate precisely when their ith bits differ.

There are $H_{\mathrm{pad}} D_{\mathrm{arr}}/L \leqslant 3M/L$ arrays. In a fixed coordinate, a given symbol occurs in exactly $1/r$ of all padded pair labels on either side. Its number of edge ends is therefore at most $H_{\mathrm{pad}} D_{\mathrm{arr}}/r \leqslant 3M/r$. □

3.3. **A polynomial deformation and integral local coordinates.** Fix a nonleaf call, its arrays, and its entrywise lifted tensors. Use

$$S = \{-L, -L+1, \dots, L-1\}, \qquad \mathcal{C}(T) = \prod_{t\in S}(T-t), \qquad \alpha = 2^{a+1}.$$

Within an array, for its row slot x and column slot y, put

$$W_{xy}(T) = \frac{\mathcal{C}(T)}{\mathcal{C}'(0)(T-(y-x))}. \tag{3.4}$$

The factor in the denominator cancels a numerator factor, since $y - x \in S$. Thus this is a polynomial over $\mathbb{Q}$ of degree $2L-1$. Set entries between different arrays to zero and restrict to the edge ends.

Under the original edge identification, $W(0) = I$, since the diagonal entries are one and each other entry has an uncancelled zero factor.

Let $A(Y)$ and $B(X)$ be the products of the lifted row and column tensors, respectively, on independent edge-end assignments $Y, X \in \{0,1\}^M$. Write $l_0(z) = 1 - z$ and $l_1(z) = z$. Define

$$\mathcal{D}(T) = \sum_{X,Y \in \{0,1\}^M} A(Y)B(X) \prod_{\text{row edge ends } x} l_{Y_x}((W(T)X)_x). \tag{3.5}$$

Its coefficients lie in $\mathcal{K}[V]/(V^\ell)$, its degree is at most $M(2L-1)$, and $\mathcal{D}(0)$ is the contraction of the lifted tensors. Indeed, for Boolean X, Y the product at $T = 0$ is $\prod_x l_{Y_x}(X_x) = \mathbb{1}_{\{X=Y\}}$; the sum over Y therefore restores the original edge identification. In particular, reducing $\mathcal{D}(0)$ modulo the call's precision gives the required answer.

Lemma 3.2. *For every $t \in S$, the entries of $W(t + \alpha z)$ belong to $\mathcal{O}[z]$, and $\mathcal{D}(t + \alpha z)$ belongs to $\mathcal{A}[z]$. For every integer $p \geqslant 1$ and $\zeta \in \mathcal{O}$ given by its e coordinates modulo 2^p, each entry $W_{xy}(t + \alpha\zeta) \bmod 2^p$ can be computed using polynomially many bit operations in s, p.*

Proof. The grid S is a complete residue system modulo 2^{a+1}. For $t \neq q$ in S, $v_2(t - q) \leqslant a$, and the counts of nonzero differences of valuation j are 2^{a-j} for $0 \leqslant j \leqslant a$: these are the residue classes divisible by 2^j but not by 2^{j+1}. The counts are independent of t. Hence $v_2(\mathcal{C}'(t)) = v_2(\mathcal{C}'(0))$. In

$$\frac{\mathcal{C}(t + \alpha z)}{\mathcal{C}'(0)} = \frac{\alpha z \prod_{q \in S \setminus \{t\}} (t - q + \alpha z)}{\mathcal{C}'(0)},$$

each factor indexed by $q \neq t$ is coefficientwise divisible by $2^{v_2(t-q)}$. After canceling these powers against the denominator, the displayed polynomial is divisible by 2^{a+1}. To obtain (3.4), omit just the factor indexed by $y - x$. If $y - x = t$, this removes αz and leaves an integral polynomial. Otherwise αz remains and the omitted factor has two-adic content $2^{v_2(t-y+x)}$ with $v_2(t - y + x) \leqslant a$, so the result is still divisible by two. Substitution in (3.5) proves the assertion for $\mathcal{D}$.

For the evaluation, first divide each factor $t - q + \alpha z$ with $q \neq t$ by the known integer $2^{v_2(t-q)}$. Both coefficients of the resulting linear polynomial are integers. Cancel the remaining explicit powers of two against the denominator as above, and invert its odd part modulo 2^p. We can now evaluate the product directly at the supplied residue of ζ, including $\zeta = 0$. The input has ep bits, and the product has $O(L)$ factors. All coordinate operations take polynomial time in s, p. □

3.4. **Hermite interpolation with decreasing multiplicities.** The next lemma isolates the precision estimate. All power-series truncations in its statement are ordinary truncations in z.

Lemma 3.3 (Hermite interpolation with decreasing multiplicities)**.** *Let integers u, M satisfy $1 \leqslant u \leqslant a$ and $M \geqslant u$, and put $h = \lceil 3M/u \rceil$. For $t \in S \setminus \{0\}$ set*

$$q_t = h \max(0, u - v_2(t)).$$

Suppose $\mathcal{D} \in (\mathcal{K}[V]/(V^\ell))[T]$ has degree at most $M(2L-1)$, $\mathcal{D}(0) \in \mathcal{A}$, and $\mathcal{D}(t + \alpha z) \in \mathcal{A}[z]$ for every $t \in S$. For every integer $p \geqslant 1$, the residue $\mathcal{D}(0) \bmod 2^p$ is determined by, and can be computed from, the data

$$\mathcal{D}(t + \alpha z) \pmod{(2^{p_t}, z^{q_t})}, \qquad p_t = p + 8hL/2^{v_2(t)} + 8M(a+1), \qquad q_t > 0.$$

Recovery takes polynomial time and space in M, L, ℓ, e and the precisions.

Proof. We use the Hermite formula for evaluation at zero and bound its denominators separately at each node. The decreasing multiplicities control the loss from the other nodes; scaling by α makes the local power-series division integral. For a nonzero node t, write $i = v_2(t)$, $D_t = L/2^i$, and $q(i) = h \max(0, u - i)$, so $q_t = q(i)$. There are L odd points in S, so $\sum_{t \neq 0} q_t \geqslant Lhu \geqslant 3ML > M(2L-1)$.

Also $h \leqslant 4M/u$, and hence $q_t \leqslant 4M$. For each t of positive multiplicity put, writing t' for the other nodes and $q_{t'}$ for their multiplicities,

$$E_t(T) = \prod_{t' \in S \setminus \{0,t\}} \left(\frac{T - t'}{t - t'} \right)^{q_{t'}}.$$

Hermite interpolation gives

$$\mathcal{D}(0) = \sum_{t : q_t > 0} E_t(0) \left(\left[\frac{\mathcal{D}(t + \alpha z)}{E_t(t + \alpha z)} \right]_{< q_t} \right) (-t/\alpha). \tag{3.6}$$

For completeness, replace zero in the right side by an indeterminate T, and $-t/\alpha$ by $(T - t)/\alpha$. Each summand has degree less than $\sum q_t$. At its own node it has the prescribed Taylor coefficients, and at every other node it vanishes to the required order. This verification may be made separately in each coefficient of V, over the field $\mathcal{K}$. The difference from $\mathcal{D}$ is therefore divisible by $\prod_{t \neq 0} (T - t)^{q_t}$ and has smaller degree, so is zero. This is the usual Chinese-remainder construction of Hermite interpolation [vzGG13, Section 5.6].

Each factor of $E_t(t + \alpha z)$ is a power of $1 + \alpha z/(t - t')$. It is integral and has constant term one. Its reciprocal truncated at q_t is therefore integral, so forming the quotient in (3.6) loses no precision.

Fix $i = v_2(t)$ and $D = D_t$. Directly from the product,

$$\begin{aligned} v_2(E_t(0)) &= \sum_{t' \neq 0, t} q_{t'} \big(v_2(t') - v_2(t - t') \big) \\ &= D \sum_{j=1}^{a-i} \frac{j}{2^j} \big(q(i + j) - q(i) \big) \ \geqslant -6Dh. \end{aligned} \tag{3.7}$$

To justify the second equality, points of valuation less than i contribute zero. For $1 \leqslant j \leqslant a - i$, there are $D/2^j$ points t' with $v_2(t') = i + j$, and the same number with $v_2(t - t') = i + j$. The latter have $v_2(t') = i$, and these levels exhaust the points other than t with that valuation. These counts follow from the complete residue grid, whose nonzero integer differences have valuation at most a. Finally, $q(i + j) - q(i) \geqslant -hj$ and $\sum_{j \geqslant 1} j^2/2^j = 6$.

Evaluation of a polynomial of degree below q_t at $-t/\alpha$ loses at most $4M(a + 1)$ further powers of two. Thus replacing each quotient coefficient by any lift of its residue modulo 2^{p_t} changes its contribution in (3.6) by an element of $2^p \mathcal{A}$. More explicitly, an error $\eta \in 2^{p_t} \mathcal{A}$ in coefficient z^j, for $0 \leqslant j < q_t \leqslant 4M$, satisfies coordinatewise

$$v_2\big(E_t(0)(-t/\alpha)^j \eta \big) \geqslant p_t - 6hD_t - j(a + 1 - i) \geqslant p.$$

Here $i = v_2(t)$, so the last inequality uses both precision increments in the statement. The inverse of $E_t(t + \alpha z)$ has integral coefficients and constant term one; multiplying by it preserves the modulus of every supplied Taylor coefficient. Summing these errors loses no further precision. The reconstructed sum is integral, since it differs from the integral value $\mathcal{D}(0)$ by an element of $2^p \mathcal{A}$.

One can perform this last combination by rational arithmetic in the specified bases. The multipliers $E_t(0)(-t/\alpha)^j$, $j < q_t$, are products of polynomially many rational factors of $O(a)$ bits, counting multiplicity. Their numerator and denominator lengths are therefore polynomial in M, L, a. There are polynomially many such coefficients, so even a product of their denominators is an adequate polynomial-bit common denominator. After addition, cancel its powers of two and reduce the resulting integral coordinates modulo 2^p. This proves the algorithmic assertion. □

At a call with input precision p, apply this lemma with the parameters in (3.1). For every selected t, obtain the entire polynomial $\mathcal{D}(t + \alpha z)$ modulo 2^{p_t} by evaluation at $M(2L - 1) + 1$ distinct binary-coordinate lifts of residue-field elements and Lagrange interpolation. Lemma 3.2 permits this

interpolation over $\mathcal{A}_{p_t}$. The number of points is less than s^4 by (3.2). The evaluation algorithm below passes precision p_t to its children.

For each selected node t, put $i = v_2(t)$ and $D_t = L/2^i$. The input and child precisions satisfy

$$p \leqslant s\lambda^{O(1)}2^{a_{d-1}}, \qquad p_t = O_C(MD_t/u). \tag{3.8}$$

The first assertion holds at the root since $P_0 \leqslant Cs$ and $a_{-1} = 0$. A nonleaf call at depth d' passes to its children the precision $p_t = p + 8hD_t + 8M(a_{d'} + 1)$. Here $h \leqslant 4M/u_{d'} \leqslant 4M$, $M \leqslant Cs$, $D_t \leqslant L = 2^{a_{d'}}$ and $a_{d'} \leqslant 4\varepsilon\lambda$, so the increment $p_t - p$ is at most $32Cs\,2^{a_{d'}} + 8Cs(4\varepsilon\lambda + 1)$. Summing the increments along the path from the root, a call at depth d has

$$p \leqslant Cs + \sum_{d'=0}^{d-1} \Big(32Cs\,2^{a_{d'}} + 8Cs(4\varepsilon\lambda + 1)\Big).$$

Its ancestors are nonleaf calls, so $a_{d'+1} - a_{d'} = 2u_{d'+1} \geqslant 2$ for $d' + 1 < d$, and the terms $2^{a_{d'}}$ sum to at most $\frac{4}{3}\,2^{a_{d-1}}$. There are $d = O_C(\log\lambda)$ terms, and $2^{a_{d-1}} \geqslant 1$. For sufficiently large s, the fixed constant C is at most λ, so

$$p \leqslant s\lambda^{O(1)}2^{a_{d-1}}.$$

This proves the first assertion for every call, including leaves. For the second, $q_t > 0$ implies $i < u$ and hence

$$D_t \geqslant 2^{a_d - u} = 2^{a_{d-1} + u}.$$

Since $M > H_{\text{leaf}}$ and $u \geqslant \lambda^{4/5}$, this extra factor 2^u dominates every fixed power of λ. Thus the first bound on p, as well as $8M(a + 1)$, is $O_C(MD_t/u)$. The remaining increment $8hD_t$ has that bound directly. All precisions remain polynomial in s because $a_d \leqslant 4\varepsilon\lambda$.

The decrease in multiplicities accounts for the factor $1/u$ in this estimate. For comparison, assign the same multiplicity $q_* = \Theta(M)$ to every level below u, and zero to the remaining levels. At the last retained level $i = u - 1$, the identity (3.7) would give the loss

$$-q_*D_t \sum_{j=1}^{a-i} \frac{j}{2^j} = -\Theta(MD_t).$$

Our multiplicities instead change by only $h = \Theta(M/u)$ per level, giving loss $O(MD_t/u)$. The cumulative choice of a_d provides the additional factor 2^u needed to absorb inherited precision and the evaluation loss. Thus the smaller bound holds at every retained level and every recursion depth.

3.5. **Auxiliary variables for interactions between groups.** Fix a selected t and an interpolation value $\zeta \in \mathcal{O}$ for z. In this subsection $T = t + \alpha\zeta$ and all calculations are modulo 2^{p_t}. Put $i = v_2(t) < u$ and $D = L/2^i$. Group vertices on both sides by their symbol in coordinate i. Entries of $W(T)$ whose two edge ends have the same group are called local and are kept unchanged.

This choice of coordinate makes $W(t)$ local. Indeed, $W_{xy}(t) = 0$ unless $y - x = t$. Adding an integer of valuation i flips bit i, so a surviving pair has different ith bits. When the array's two symbols in that coordinate differ, this means that the row and column symbols agree. Equal pairs of symbols already lie in one group. Moving from t to $t + \alpha\zeta$ introduces the crossing terms that we now expand.

In an array whose two symbols in coordinate i differ, a row position x and a column position y are in different groups exactly when their ith bits are equal. Partition columns into residue classes

$$y = y_0 + 2^{i+1}j, \qquad 0 \leqslant y_0 < 2^{i+1}.$$

The column group is constant on a residue class. For a row crossing to that residue class,

$$v_2(t - y_0 + x) \leqslant i.$$

Otherwise y_0 would agree with $x + 2^i$ modulo 2^{i+1}, contrary to equality of their ith bits. Put $d_x = T - y_0 + x$ and $v = v_2(t - y_0 + x) \leqslant i$. For every $\zeta \in \mathcal{O}$, adding $\alpha\zeta$ preserves this valuation, since

$v_2(\alpha\zeta) \geqslant a+1 > i$. Thus $d_x/2^v$ is a unit and $v_2(2^{i+1}j/d_x) \geqslant 1$. A geometric expansion consequently gives

$$W_{xy}(T) = \sum_{\substack{b\geqslant 0 \\ b+1<p_t}} \frac{\mathcal{C}(T)}{\mathcal{C}'(0)(T-y_0+x)} \left(\frac{2^{i+1}}{T-y_0+x}\right)^b j^b \pmod{2^{p_t}}. \tag{3.9}$$

Indeed $\mathcal{C}(T)/\mathcal{C}'(0)$ has valuation at least $a+1$ by the integrality proof, and the coefficient of j^b in the displayed sum has valuation at least

$$(a+1-v)+b(i+1-v) \geqslant b+1.$$

For $R_x = 2^{i+1}j/d_x$, use the finite identity

$$(1-R_x)^{-1} = \sum_{b=0}^{p_t-2} R_x^b + \frac{R_x^{p_t-1}}{1-R_x}.$$

The last denominator is a unit. Multiplying its remainder by $\mathcal{C}(T)/(\mathcal{C}'(0)d_x)$ gives an element of $2^{p_t}\mathcal{O}$. This proves the expansion also when $\zeta = 0$ or $j = 0$. These expressions are computable by factoring the known powers of two and using only unit inverses.

For each such array, residue class, and b introduce an auxiliary variable ω of weight $w_\omega = b+1$, to be replaced by

$$L_\omega(X) = \sum_{\text{occupied column positions } y \text{ in the residue class}} j^b X_y.$$

As usual $j^0 = 1$, including $j = 0$. Its source is the residue class's column group and its target is the other group in that array's coordinate pair. Formula (3.9) expresses each row form as its local part plus a linear combination of its auxiliary variables with that target. The coefficient of ω is divisible by 2^{w_ω}. There are at most

$$(3M/L)2^{i+1} = 6M/D \tag{3.10}$$

variables of any one weight.

In the product of the affine factors $l_{Y_x}((W(T)X)_x)$, every coefficient of a monomial in the auxiliary variables ω^β is divisible by $2^{\sum_\omega w_\omega\beta_\omega}$. Consequently only the exponent tuples

$$\sum_\omega w_\omega\beta_\omega < p_t \tag{3.11}$$

need be considered. The number of these tuples is

$$\exp\bigl(O(\sqrt{Mp_t/D})\bigr) = \exp\bigl(O_C(M/\sqrt{u})\bigr). \tag{3.12}$$

Here is the complete counting estimate. Fix $\theta > 0$. Every tuple β satisfying (3.11) has $\exp(\theta(p_t - \sum_\omega w_\omega\beta_\omega)) \geqslant 1$. Summing this quantity over all tuples of nonnegative integers gives the generating product, so the number of tuples is at most

$$\sum_\beta \exp\Bigl(\theta\bigl(p_t - \sum_\omega w_\omega\beta_\omega\bigr)\Bigr) = e^{\theta p_t}\prod_\omega \bigl(1-e^{-\theta w_\omega}\bigr)^{-1}.$$

Every weight w_ω is a positive integer, and by (3.10) at most $6M/D$ auxiliary variables have any one weight. The number of tuples is therefore at most

$$\exp\left(\theta p_t + \frac{6M}{D}\sum_{j\geqslant 1} -\log(1-\exp(-\theta j))\right).$$

Expanding the logarithm and summing the geometric series gives

$$\sum_{j\geqslant 1} -\log(1-\exp(-\theta j)) = \sum_{h\geqslant 1}\frac{1}{h(\exp(\theta h)-1)} \leqslant \frac{1}{\theta}\sum_{h\geqslant 1}\frac{1}{h^2}.$$

Take $\theta = \sqrt{M/(Dp_t)}$ and use (3.8). All auxiliary variables and their exponents have polynomial-size descriptions. Tuples satisfying (3.11) can be enumerated with polynomial delay and space by ordering the variables and successively choosing each exponent subject to the remaining budget. Every feasible partial choice has the all-zero continuation, so the number of visited partial choices is at most the number of complete tuples times the number of variables, up to an additive constant.

3.6. **The child networks and their descriptions.** For one tuple β in (3.11) construct a child network for each group $g \in [r]$. Place all current vertices in group g, from both sides of the current network, on one side of the child. Retain their edge ends and their chosen lifted entries, now reduced modulo 2^{p_t}. On the other side put one vertex for each array, with an edge to every retained edge end of that array. Its indices are precisely the row bits Y and column bits X of group g in the array.

To define this new array tensor, form the product of the row factors l_{Y_x} for group g in the array, with their arguments expressed in local column bits and auxiliary variables. Extract the coefficient with exponents β_ω for all auxiliary variables targeted at g in that array. Multiply it by

$$\prod_{\substack{\omega \text{ in this array} \\ \text{source}(\omega)=g}} L_\omega(X)^{\beta_\omega}.$$

Empty products equal one. If there are no target rows, the extracted coefficient is zero whenever a target exponent is positive. If the source residue class of ω has no occupied column position, then $L_\omega(X) = 0$ and its power contributes one for $\beta_\omega = 0$ and zero otherwise.

Every auxiliary variable has one target and one source. Coefficient extraction therefore factors over the array-group row products, and the substitution for each auxiliary variable depends only on bits in its source group. To see the factorization explicitly, denote the row product for array κ and target group g by $R_{\kappa,g}(Y_g, X_g; \omega)$, where Y_g and X_g are that group's row and column bits in the array. Its formal variables are precisely the auxiliary variables of that array targeted at g. These sets of formal variables are disjoint as (κ, g) varies. Consequently, before substituting the auxiliary variables,

$$[\omega^\beta] \prod_{\kappa,g} R_{\kappa,g} = \prod_{\kappa,g} [\omega^{\beta|_{\kappa,g}}] R_{\kappa,g},$$

where $\beta|_{\kappa,g}$ is the restriction to those auxiliary variables with that target. After substitution, assign each factor $L_\omega(X)^{\beta_\omega}$ to its source group. All factors in a group then depend only on that group's edge-end bits, including its original tensor entries. Distributing the product in (3.5), and then summing the independent bits within groups, proves the exact identity

$$\mathcal{D}(t + \alpha\zeta) = \sum_{\beta:\, \sum w_\omega \beta_\omega < p_t} \prod_{g=1}^{r} Z_{g,\beta} \pmod{2^{p_t}}, \tag{3.13}$$

where $Z_{g,\beta}$ is the contraction of the child just defined. The omitted monomials vanish by their coefficient divisibility.

Each edge of a child joins a retained edge end of a group-g vertex, from either side of the current network, to the vertex of that edge end's array. The edges of the child therefore correspond to the edge ends of group-g vertices on the two sides. For the coordinate i that defines the groups, Lemma 3.1(3) bounds these edge ends by $3M/r$ on each side, so a child has at most $6M/r = 3M/32 < M/8$ edges. Old vertex degrees are unchanged, and a new array vertex has at most $2L \leqslant \Delta$ edge ends. Thus the invariants used in (3.2) hold inductively.

All new tensors are given implicitly. To evaluate an array tensor entry, expand its product of at most L affine row factors. Each factor has polynomially many terms in s, including its local term. Enumerating these choices, checking the desired exponents, and summing the resulting terms takes

$$\exp(O(L \log s))$$

time and polynomial space. The factors $L_\omega(X)^{\beta_\omega}$ are computed by ordinary repeated squaring in $\mathcal{A}_{p_t}$. Because $L \leqslant s^{1/1000}$, this entry cost is $\exp(O(s^{9/10}))$.

A description of a new tensor consists of its array data, the tuple β, the interpolation value, and the creation precision. These data and the matrix entries within the array suffice to compute its entry directly. A retained tensor is evaluated from its original description, or from its own creation-time description if it was created earlier in the recursion. Its residue at that precision is then lifted using least nonnegative coordinate representatives to the current precision. The old descriptions are immutable and shared by reference. Thus no composition of entry routines builds up with the depth, and all entry routines continue to have the required time and space bounds. In particular, a new array entry uses the assigned edge-end bits and the matrix entries, while the tensors incident to those edge ends remain separate vertices; evaluating the array never calls their evaluators. Only polynomially many vertices and polynomially much new description data are added along a path.

3.7. **Time and space bounds.**

Proof of Theorem 2.1. Embed the original entries into $\mathcal{A}_{P_0}$ and perform the recursive algorithm above. At a nonleaf call, for every node of positive multiplicity and every required residue-distinct interpolation value, compute (3.13) by its child contractions. Interpolate $\mathcal{D}(t+\alpha z)$ and apply Lemma 3.3. At a leaf, sum directly over all Boolean edge assignments. Lemma 3.2, the recursion identity, and the recovery lemma prove correctness by induction from the leaves.

The number of interpolation nodes, values, and groups at a call is polynomial in s, independently of $\mathcal{L}$. By (3.12), the number of children of a nonleaf depth-d call is at most

$$s^{O(1)} \exp\left(O_C\left(\frac{s}{\sqrt{\lambda}}\left(\frac{\sqrt{2}}{8}\right)^d\right)\right).$$

Indeed $M \leqslant Cs\,8^{-d}$ and $u_d = \Theta_C(\lambda/2^d)$. Multiplying these bounds over the $O_C(\log\lambda)$ depths, or bounding the number of nodes on each level in this way, gives at most

$$\exp\Big(O_C(s/\sqrt{\lambda}) + O_C(\lambda\log\lambda)\Big) = \exp(O_C(s/\sqrt{\lambda}))$$

calls in total. At a leaf, direct summation uses at most $2^{H_{\mathrm{leaf}}}$ assignments and polynomially many entry evaluations per assignment. Its cost is bounded by a polynomial in $s+\mathcal{L}$ times

$$\exp(O_C(H_{\mathrm{leaf}}) + O_{C,A}(s^{9/10})) = \exp(O_{C,A}(s/\sqrt{\lambda})).$$

Local arithmetic, packing, descriptions, and recovery use polynomial time per generated term, apart from the already bounded entry computations. The description-dependent factor bounds the work per call. Since the branching bound is independent of $\mathcal{L}$, the total work retains a fixed polynomial factor in $s+\mathcal{L}$. This proves the time bound.

For space, traverse the recursion depth first, accumulating one sum over β and one product over groups at a time. Store the polynomially many Taylor data needed for a call's recovery, its current interpolation and auxiliary-variable indices, and its graph and new tensor descriptions. Their sizes are polynomial in $s+\mathcal{L}$, including the precisions by (3.8). There are only $O_C(\log\lambda)$ stack levels. The common-denominator calculation in Lemma 3.3 also uses only polynomial-bit integers. Each call accumulates its contraction terms modulo its own fixed precision. Original entry routines use polynomial space by hypothesis, and the new routines have the same property. This proves the space bound. Finally extract the base coordinates in $\mathcal{A}_{P_0}$, which give the answer in $\mathcal{B}_{P_0,\ell}$. □

4. Encoding sparse polynomial systems

We prove Lemma 2.2. The first step represents each equation by an integer polynomial modulo a power of two. Truncating the product across all equations keeps its total input degree linear in

the number of variables. The proof in Section 4.3 then encodes its monomials as tensor indices and applies Theorem 2.1.

Recall the common-zero count $N(\mathcal{P})$ from (2.1). In the lift below, m_{test} denotes the number of equations, and each P_i has degree at most k. The list of its monomials is supplied over $\mathbb{F}_2$.

4.1. **An integer polynomial for one equation.** On Boolean inputs we may replace each positive power x_v^j by x_v. Combine duplicate monomials modulo two. If $\mathcal{M}_i$ is the resulting list of variable sets for P_i, define its integer lift by

$$L_i(\vec{X}) = \sum_{M \in \mathcal{M}_i} \prod_{v \in M} X_v.$$

The empty monomial is 1 and the empty sum is 0. The value $L_i(x)$ counts the monomials equal to one, so $P_i(x) = 0$ precisely when $L_i(x)$ is even. Write $I_i = \bigcup_{M \in \mathcal{M}_i} M$. Its scope size is at most k times its monomial-list length, while its degree is at most k.

Set $\ell = n + 1$. Since $0 \leqslant N(\mathcal{P}) \leqslant 2^n < 2^\ell$, it suffices to obtain its residue modulo 2^ℓ. We first represent the evenness indicator by an integer polynomial modulo this power of two. Multiplying m_{test} such indicators directly would give only a quadratic degree bound; the form of the next lemma will let us control the degree across all equations together.

Put $h(Y) = Y(Y-1)$. Its value is even at every integer Y. Writing the indicator in powers of h exposes one factor of two for each copy of h in the product.

Lemma 4.1. *For every integer $\ell \geqslant 1$, one can compute polynomials $e_0, \dots, e_{\ell-1} \in \mathbb{Z}[Y]$, each of degree at most one, such that*

$$p_\ell(Y) = \sum_{j=0}^{\ell-1} e_j(Y) h(Y)^j, \qquad p_\ell(y) \equiv \mathbb{1}_{\{2 \mid y\}} \pmod{2^\ell} \quad (y \in \mathbb{Z}). \tag{4.1}$$

The coefficients of the e_j have $\operatorname{poly}(\ell)$ *bits, and the construction uses* $\operatorname{poly}(\ell)$ *bit operations and* $\operatorname{poly}(\ell)$ *bits of space.*

Proof. Let F_ℓ be the modulus-amplifying polynomial of Beigel and Tarui [BT94, Section 2.3]. Chan and Williams apply F_ℓ to sums of clause-violation indicators to count satisfying assignments deterministically [CW21, Theorem 2.5 and proof of Theorem 1.6]. We need the indicator of even parity, so we take its complement:

$$p_\ell(Y) = 1 - F_\ell(Y) = (1-Y)^\ell \sum_{j=0}^{\ell-1} \binom{\ell + j - 1}{j} Y^j. \tag{4.2}$$

The finite sum is the inverse of $(1-Y)^\ell$ modulo Y^ℓ. Indeed, the coefficient of Y^j in $(1+Y+Y^2+\cdots)^\ell$ counts ℓ-tuples of nonnegative integers summing to j and equals $\binom{\ell+j-1}{j}$. Therefore

$$p_\ell(Y) \equiv 1 \pmod{Y^\ell}, \qquad p_\ell(Y) \equiv 0 \pmod{(1-Y)^\ell}.$$

For even y, the first congruence gives $p_\ell(y) \equiv 1 \pmod{2^\ell}$. For odd y, the second gives $p_\ell(y) \equiv 0 \pmod{2^\ell}$.

The polynomial p_ℓ has integer coefficients and degree less than 2ℓ. Successive division by the monic quadratic $h = Y^2 - Y$ writes it uniquely as $\sum_{j<\ell} e_j h^j$, with integer remainders e_j of degree less than two. Lemma B.1 in Appendix B bounds their coefficients and proves the $\operatorname{poly}(\ell)$ bit-operation bound for this construction. □

4.2. **Keeping the product degree linear.** The full product $\prod_{i=1}^{m_{\text{test}}} p_\ell(L_i(\vec{X}))$ has degree at most $km_{\text{test}}(2\ell-1)$. Each factor $h(y)$ contributes a factor of two at integer arguments, so a term containing at least ℓ such factors can be discarded modulo 2^ℓ. This lets us retain only terms whose total power of h is less than ℓ, across all equations together. Introduce a formal variable V to record that power, using the same V for every equation. Its exponent records a guaranteed power of two dividing the corresponding term at integer arguments. It is not an additional Boolean variable to be summed over. For the rational computation we use the coefficient algebra

$$\mathcal{E}_\ell = \mathbb{Q}[V]/(V^\ell).$$

In this algebra every multiplication discards powers V^s with $s \geqslant \ell$. Once the product is formed, we add its retained coefficients using the linear map

$$\mathcal{S}_\ell\left(\sum_{s=0}^{\ell-1} b_s V^s\right) = \sum_{s=0}^{\ell-1} b_s.$$

This map need not preserve products, so it is applied only after the factors have been multiplied. We use the same notation for polynomial coefficients b_s.

Lemma 4.2. *Let $m_{\text{test}} \geqslant 0$ and $\ell \geqslant 1$ be integers. For integers $y_1, \dots, y_{m_{\text{test}}}$, put*

$$B_i(V) = \sum_{j=0}^{\ell-1} e_j(y_i) h(y_i)^j V^j.$$

Then

$$\mathcal{S}_\ell\Big(\prod_{i=1}^{m_{\text{test}}} B_i(V) \bmod V^\ell\Big) \equiv \prod_{i=1}^{m_{\text{test}}} \mathbb{1}_{\{2|y_i\}} \pmod{2^\ell}.$$

Independently, for any nonnegative integer d_*, if the y_i are polynomials over $\mathbb{Q}$ in any common collection of inputs, each of total degree at most d_*, then each retained coefficient of V^s, and hence their sum, has total input degree at most $d_*(m_{\text{test}} + 2(\ell-1))$.

Proof. Before truncation, a term contributing to the coefficient of V^s has the form

$$\prod_{i=1}^{m_{\text{test}}} e_{j_i}(y_i) h(y_i)^{j_i}, \qquad \sum_{i=1}^{m_{\text{test}}} j_i = s.$$

At integer arguments it is divisible by 2^s. Dropping every coefficient with $s \geqslant \ell$ therefore leaves the sum of coefficients unchanged modulo 2^ℓ. Without truncation, that sum is $\prod_i p_\ell(y_i)$, which has the required congruence by Lemma 4.1. Truncating after each multiplication gives the same result as truncating the full product: every later factor has nonnegative V-degree, so a discarded term can never return to degree below ℓ. This also explains why the coefficient sum is taken at the end; it is the retained total orders that have the required congruence.

For polynomial arguments, every retained term has $\sum_i j_i \leqslant \ell - 1$. Since $\deg e_{j_i} \leqslant 1$ and $\deg h = 2$, its degree is at most

$$\sum_{i=1}^{m_{\text{test}}} d_*(1 + 2j_i) \leqslant d_*(m_{\text{test}} + 2(\ell-1)).$$

The bound holds coefficient by coefficient, as well as for their sum. □

To see why the truncation uses the total order across all equations, take $\ell = 2$. Then

$$p_2(Y) = 1 - 3Y^2 + 2Y^3 = (1-Y) + (2Y-1)h(Y).$$

For two equations, multiplication modulo V^2, followed by $\mathcal{S}_2$, retains

$$(1-y_1)(1-y_2) + (2y_1-1)h(y_1)(1-y_2) + (1-y_1)(2y_2-1)h(y_2).$$

Its degree is at most four. The omitted term $(2y_1 - 1)(2y_2 - 1)h(y_1)h(y_2)$ has degree six, but is divisible by four at integer inputs.

4.3. **From the retained monomials to a network.**

Proof of Lemma 2.2. The network will sum coefficients of incidence monomials, with test vertices supplying those coefficients and variable vertices supplying their sums over Boolean values. Put $\lambda = \lceil \log_2 n \rceil$ and $\nu = \lceil \log_2 \lambda \rceil$. We will apply Theorem 2.1 with $s = n\nu$, so we need $O_{k,c}(n\nu)$ wires and maximum degree at most $\lceil (n\nu)^{9/10} \rceil$. Each vertex will stand for a group of tests or of free variables, and its degree will be at most the number of incidences in its group. Each test already reads only polylogarithmically many variables, but a single variable may belong to all a scopes. We first fix the high-incidence variables in all possible ways. Afterwards every free variable lies in only polylogarithmically many scopes. After grouping, this will bound the degrees on both sides by $n^{3/4}\lambda^{O_{k,c}(1)}$, below the required maximum for all sufficiently large n. Within each monomial replace positive powers of a variable by its first power, and combine duplicate monomials modulo two. The scope of test i consists of the variables in its remaining monomials. It has size at most $k\lambda^c$. Let N_0 be the sum of the scope sizes, counting a pair (i, v) once if variable v belongs to test i. Thus $N_0 \leqslant ak\lambda^c \leqslant ckn\lambda^c$, since $a \leqslant cn$. Fix, in all possible ways, the variables belonging to more than

$$D_* = \max\{1, \lceil N_0/n \rceil\}\lceil \sqrt{\lambda} \rceil$$

scopes. There are at most $n/\sqrt{\lambda}$ such variables. Their assignments partition the original count into at most $2^{n/\sqrt{\lambda}}$ branch counts. On each branch substitute the fixed bits and combine duplicate monomials again. The free scope I_i of test i still has size at most $k\lambda^c$, and each free variable appears in at most $D_* = \lambda^{O_{k,c}(1)}$ scopes. Free variables absent from every scope remain declared. A constant-one equation makes the branch count zero. We may discard constant-zero equations, and if none remain the answer for the branch is two to the power of its number of free variables. Below a denotes the number of remaining equations and n the original variable count.

The polynomial on the incidences. Give each pair (i, v) with $v \in I_i$ its own indeterminate z_{iv}. Write $z_i = (z_{iv})_{v \in I_i}$. Form $\widetilde{L}_i(z_i)$ by summing the remaining monomials of the normalized branch polynomial with integer coefficients, using z_{iv} for x_v. We use the ring

$$\mathcal{B} = (\mathbb{Z}/2^p\mathbb{Z})[V]/(V^\ell), \qquad p = \ell = n + 1.$$

Take $h(Y) = Y(Y - 1)$ and the integer polynomials e_j from Lemma 4.1. Reduce their coefficients modulo 2^p, and form the factor

$$B_i(z_i, V) = \sum_{j=0}^{\ell-1} e_j(\widetilde{L}_i(z_i))\, h(\widetilde{L}_i(z_i))^j V^j.$$

Let $\overline{B}_i$ be its multilinear representative in the z_i, obtained in each coefficient of V by replacing each positive power of an incidence indeterminate by its first power. It agrees with B_i on every Boolean assignment to z_i. Define $F(z, V) = \prod_i \overline{B}_i(z_i, V)$ in $\mathcal{B}[z]$. The incidence indeterminates for different tests are disjoint, so F is multilinear even before substituting the same Boolean x_v for all copies of variable v.

The coefficient of V^j in $\overline{B}_i$ has incidence degree at most $k(1 + 2j)$, since $\deg e_j \leqslant 1$, $\deg h = 2$, and multilinearization only lowers monomial degrees. Multiplication in $\mathcal{B}$ retains only terms with $\sum_i j_i \leqslant \ell - 1$. Consequently every incidence monomial in F has degree at most

$$K = k\big(a + 2(\ell - 1)\big) = O_{k,c}(n). \tag{4.3}$$

Over $\mathcal{B}$, use the same coefficient-sum formula $\mathcal{S}_\ell(\sum_{j<\ell} b_j V^j) = \sum_{j<\ell} b_j$, now with $b_j \in \mathbb{Z}/2^p\mathbb{Z}$. At $z_{iv} = x_v$, the multilinearized factor $\overline{B}_i$ agrees with B_i for the normalized branch polynomial. The

integer $\widetilde{L}_i((x_v)_{v\in I_i})$ has the same parity as the original P_i evaluated at x and the fixed bits. Applying Lemma 4.2 to these integers shows that the branch count is congruent modulo 2^p to

$$\mathcal{S}_\ell\left(\sum_x F\big((z_{iv}=x_v)_{iv},V\big)\right), \tag{4.4}$$

where x runs over all free Boolean assignments. We apply the coefficient-sum map after multiplying the test factors in the truncated ring.

Indexing the monomials. Giving each incidence its own wire would use up to $N_0 \leqslant ckn\lambda^c$ wires, while the claimed exponent allows only $O_{k,c}(n\nu)$. A monomial of F selects at most K incidences. We group the tests and the free variables, fix how many selected incidences join each pair of groups, and let the wires between two groups index the subsets of that size. The sizes are enumerated separately as an array (d_{gh}), with one entry per pair of groups; by (4.5) below the binary subset indices give the $O_{k,c}(n\nu)$ wire bound. We choose $J=\lceil n^{1/4}\rceil$ groups on each side so that the J^2 array entries cost only $2^{O_{k,c}(\sqrt{n}\log n)}$ choices, while each group has few enough incidences to meet the maximum-degree bound. Partition the tests and the free variables separately into these J groups, with sizes differing by at most one on each side. Write $\mathcal{T}_g$ for a test group and $\mathcal{X}_h$ for a variable group. A group contains $O_c(n^{3/4})$ objects, so the number of its incidences is at most $n^{3/4}\lambda^{O_{k,c}(1)}$ on either side. Let N_{gh} be the number of incidences from $\mathcal{T}_g$ to $\mathcal{X}_h$ and let $N=\sum_{g,h}N_{gh}\leqslant N_0$.

For an incidence monomial let d_{gh} be the number of its selected incidences joining groups g,h. Enumerate arrays of integers (d_{gh}) with $0\leqslant d_{gh}\leqslant N_{gh}$ and $\sum_{g,h}d_{gh}\leqslant K$. There are at most

$$(K+1)^{J^2}=2^{O_{k,c}(\sqrt{n}\log n)}$$

arrays, which can be enumerated using J^2 counters. For a fixed array there are

$$C_{gh}=\binom{N_{gh}}{d_{gh}}$$

subsets of the specified size for this pair. Draw $\lceil\log_2 C_{gh}\rceil$ Boolean wires between the two group vertices to index them in lexicographic order. For $C_{gh}=1$ both endpoints use the unique subset (possibly the entire incidence set) with no wires. Any unused bit string gives tensor entry zero. To decode a valid index, take the incidences in order and count the possible completions with or without each incidence using binomial coefficients. These have polynomial bit length, so decoding uses polynomial time and space.

The union of the subsets over all pairs specifies a squarefree monomial of degree at most K in N indeterminates. Including all monomials of this degree bound gives the upper bound $\binom{N+K}{K}$ by stars and bars. Thus the number of wires M in the network for this array satisfies

$$\begin{aligned} M &\leqslant J^2+\log_2\prod_{g,h}C_{gh}\leqslant J^2+\log_2\binom{N+K}{K}\\ &\leqslant J^2+K\log_2\left(\frac{\mathrm{e}(N+K)}{K}\right)=O_{k,c}(n\nu). \end{aligned} \tag{4.5}$$

Here $K=\Theta_{k,c}(n)$ and $N\leqslant n\lambda^{O_{k,c}(1)}$. Also $\lceil\log_2 C_{gh}\rceil\leqslant N_{gh}$, so the degree of each vertex is at most its number of incidences, $n^{3/4}\lambda^{O_{k,c}(1)}$.

Tensor entries and contraction. A valid assignment of wire bits determines a subset $S_i\subseteq I_i$ at each test. Write $z_{i,S_i}=\prod_{v\in S_i}z_{iv}$. The entry at a test group g is the coefficient of its indicated monomial in $\prod_{i\in\mathcal{T}_g}\overline{B}_i(z_i,V)$. Since the z_i are disjoint, this is

$$\prod_{i\in\mathcal{T}_g}[z_{i,S_i}]\overline{B}_i(z_i,V)\qquad\text{in }\mathcal{B}.$$

For a variable group h, put $b_v = |\{i : v \in S_i\}|$ for $v \in \mathcal{X}_h$. Its entry is

$$\sum_{x \in \{0,1\}^{\mathcal{X}_h}} \prod_{v \in \mathcal{X}_h} x_v^{b_v} = 2^{|\{v \in \mathcal{X}_h : b_v = 0\}|} \pmod{2^p}.$$

Indeed a positive exponent requires $x_v = 1$ and a zero exponent leaves x_v free. This includes variables outside all test scopes.

Each monomial of F occurs in exactly one array and has exactly one valid index at each group pair. Its coefficient in F is the product of the test-group coefficients. After the incidences with the same v are set equal, its sum over Boolean assignments is the product of the variable-group entries. By (4.3), summing over these arrays exhausts all monomials. Consequently the element of $\mathcal{B}$ inside $\mathcal{S}_\ell$ in (4.4) is the sum of the contractions of these bipartite networks.

To evaluate one test-side coefficient, let $\mathbf{1}_T$ be the Boolean vector on I_i with support $T \subseteq S_i$. At $\mathbf{1}_T$, the multilinear polynomial evaluates to the sum of its coefficients indexed by subsets of T, and it agrees there with B_i. Inverting this subset sum gives

$$[z_{i,S_i}]\overline{B}_i(z_i, V) = \sum_{T \subseteq S_i} (-1)^{|S_i|-|T|} B_i(\mathbf{1}_T, V) \qquad \text{in } \mathcal{B}.$$

Each term costs polynomially many bit operations. Evaluate $\widetilde{L}_i(\mathbf{1}_T)$ from the listed monomials and use the polynomials e_j of Lemma 4.1, which are constructible in polynomial time. Store the ℓ coefficients of $B_i(\mathbf{1}_T, V)$ modulo 2^p. There are at most $2^{|S_i|} \leqslant 2^{k\lambda^c}$ subsets to process. Sum one test at a time and multiply the resulting coefficients in $\mathcal{B}$ over the group. This evaluates a test-group entry in $2^{\lambda^{O_{k,c}(1)}}$ bit operations and polynomial space. The variable-side entries and checks for invalid wire indices take polynomial time. The graph, degree array, and entry procedures have polynomial-size descriptions.

Apply Theorem 2.1 with size parameter $s = n\nu$, root precision $p = n + 1$, and truncation order $\ell = n + 1$. The graph has $2J$ vertices and $O_{k,c}(s)$ wires by (4.5). For all sufficiently large n its maximum degree is at most $\lceil s^{9/10} \rceil$, its precision and truncation order are $O(s)$, and its entry-evaluation time is at most $2^{O_{k,c}(s^{9/10})}$. Each contraction then takes $n^{O_{k,c}(1)} 2^{O_{k,c}(s/\sqrt{\log s})}$ bit operations and polynomial space. Here $s/\sqrt{\log s} = O(n\nu/\sqrt{\lambda})$. The high-incidence branches and degree arrays multiply the running time by at most

$$2^{n/\sqrt{\lambda} + O_{k,c}(\sqrt{n} \log n)}.$$

Both terms in this exponent are $O_{k,c}(n\nu/\sqrt{\lambda})$. Process the branches and arrays sequentially. On each branch, sum the contractions in $\mathcal{B}$ and apply $\mathcal{S}_\ell$. Adding the branch residues modulo 2^{n+1} gives the original count, since it lies between 0 and 2^n. This proves the claimed bounds, using direct enumeration for bounded exceptional sizes. □

5. Constructing the parity tests

We prove Lemmas 2.4 and 2.5, whose statements and notation are in Section 2.3. The common-zero counter may be either Lemma 2.2, used in the main theorem, or Theorem 6.2, used in the interpolation algorithm.

5.1. **Initial tests that exclude small violation sets.**

A short random test can miss an assignment violating very few clauses. The initial matrix $A^{(0)}$ will reject every nonzero violation vector of weight less than m/P, so that subsequent short tests can detect each remaining nonsatisfying assignment with constant probability.

We use the standard edge-based Tanner code: bits are assigned to the edges of an expander, and the bits incident to each vertex must form a word of a prescribed local linear code (see Tanner [Tan81, Section II] for the local constraints and Sipser and Spielman [SS96, Sections IV and VI] for the expander construction). The usual local-distance and expansion argument gives the needed lower

bound on the weight of a nonzero codeword. The proof below constructs the graph and local code deterministically, with parameters that meet our bounds on the number and length of the tests.

Proof of Lemma 2.4. We construct a bipartite graph with between m and $2m$ edges, using its edges as temporary binary coordinates. Checks at each vertex will force every vertex touched by a nonzero edge to have many nonzero incident edges; a mixing bound will then force large total weight. At the end, we keep m edge coordinates and fix the rest to zero. Let

$$d_G = \min(m, D_0), \qquad q_G = \min\{2^a : a \in \mathbb{Z}_{\geqslant 0},\ 2^a d_G \geqslant m\}.$$

Then $m \leqslant q_G d_G < 2m$. Write $q_G = 2^{\ell_G}$ and take two copies of $\{0,1\}^{\ell_G}$ as the vertex sets. For each left vertex u and each edge end $j \in [d_G]$, put an edge from u to $u \oplus s_j$, where $\oplus$ denotes bitwise addition modulo 2 and the shifts $s_1, \dots, s_{d_G} \in \{0,1\}^{\ell_G}$ will be chosen below. Edges are indexed by the pairs (u, j); parallel edges remain distinct. Every vertex on either side has one incident edge at each edge end j, so the graph is d_G-regular.

To obtain the mixing estimate (5.2), we choose the shifts so that every parity of a nonempty set of coordinates is nearly balanced. For a nonzero vector $b \in \{0,1\}^{\ell_G}$, the bit $b \cdot s_j \bmod 2$ is the parity of those coordinates of s_j where b equals 1. With $\varepsilon = 1/(200B\mu)$, we require

$$\Big|\sum_{j=1}^{d_G} (-1)^{b \cdot s_j}\Big| \leqslant \varepsilon d_G \qquad \big(b \in \{0,1\}^{\ell_G} \setminus \{\vec{0}\}\big). \tag{5.1}$$

The sum in (5.1) is the number of indices j with even parity minus the number with odd parity. Thus, for every nonzero b, each parity occurs at a fraction of indices between $(1-\varepsilon)/2$ and $(1+\varepsilon)/2$. A list of shifts with this property is called ε-biased [NN93, Definitions 2.1–2.2].

If $q_G = 1$, there is no condition to check. Otherwise $d_G = D_0$. To show that suitable shifts exist, we choose them independently and uniformly at random. For each fixed nonzero b, the signs $(-1)^{b \cdot s_j}$ are independent and uniform in $\{-1, 1\}$. The number of indices with even parity and the number with odd parity are each a sum of d_G independent uniform bits. Hoeffding's inequality [AW15, Lemma 3.3], with $p = 1/2$ and $k = (1-\varepsilon)d_G/2$, shows that each of the two numbers is at most $(1-\varepsilon)d_G/2$ with probability at most $\exp(-\varepsilon^2 d_G/2)$. Hence the probability of violating (5.1) for this b is at most $2\exp(-\varepsilon^2 d_G/2)$. A union bound over b gives total failure probability at most

$$2q_G \exp(-\varepsilon^2 d_G/2) = 2q_G \exp(-4\mu) < 1,$$

where $q_G < 2m < 2^{\mu+1}$. Exhaustively enumerating the shift tuples and checking (5.1) therefore finds a valid tuple in $\exp(O(d_G \log(m+2)))$ time.

We next derive the mixing estimate from (5.1). Let A_G be the $q_G \times q_G$ matrix with $\mathsf{A}_G[u, v] = |\{j \in [d_G] : u \oplus s_j = v\}|$; its entries count edges from left vertices u to right vertices v. Both sides have the same index set $\{0,1\}^{\ell_G}$, on which the functions $\chi_b(u) = (-1)^{b \cdot u}$ form an orthogonal basis. For every $b, u \in \{0,1\}^{\ell_G}$,

$$(\mathsf{A}_G \chi_b)(u) = \sum_{j=1}^{d_G} \chi_b(u \oplus s_j) = \chi_b(u) \sum_{j=1}^{d_G} (-1)^{b \cdot s_j}.$$

Thus each χ_b is an eigenvector: the constant function $\chi_{\vec{0}}$ has eigenvalue d_G, while (5.1) bounds all other eigenvalues in absolute value by εd_G. For sets U of left vertices and V of right vertices, let $e_G(U, V)$ count the edges between them, including multiplicity. Decomposing the indicator vectors of U, V into their constant and orthogonal parts and applying Cauchy–Schwarz gives the estimate below. Explicitly, write $\mathbf{1}_U = (|U|/q_G)\mathbf{1} + u_0$ and $\mathbf{1}_V = (|V|/q_G)\mathbf{1} + v_0$, where u_0, v_0 are orthogonal to $\mathbf{1}$. Regularity makes the two cross terms zero, and the character eigenvalue bound gives

$$|u_0^{\mathsf{T}} \mathsf{A}_G v_0| \leqslant \varepsilon d_G \|u_0\|_2 \|v_0\|_2 \leqslant \varepsilon d_G \sqrt{|U||V|}.$$

This calculation counts all parallel edges through the entries of A_G, and yields

$$e_G(U,V) \leqslant \frac{d_G|U||V|}{q_G} + \varepsilon d_G\sqrt{|U||V|}. \tag{5.2}$$

To specify the local code, we put

$$c_G = \left\lceil \frac{d_G}{8B} \right\rceil, \qquad \delta = \frac{1}{100B\mu}.$$

We need a binary matrix A_{loc} with c_G rows and d_G columns such that every nonzero word in its kernel modulo 2 has weight at least δd_G. To prove that such a matrix exists, we consider a matrix whose entries are independent uniform bits. For each fixed nonzero word, one of its nonzero coordinates makes the parity in each row uniform; independence of the rows therefore gives probability 2^{-c_G} that the word lies in the kernel. Set $h = \lceil \delta d_G \rceil - 1$. If $h = 0$, there are no nonzero words of weight less than δd_G. Otherwise their number is at most

$$\sum_{j=1}^{h} \binom{d_G}{j} \leqslant (h+1)(2d_G)^h \leqslant (2d_G)^{2h} < 2^{d_G/(25B)}.$$

For the last inequality, use $h < \delta d_G$, $d_G \leqslant m < 2^\mu$, and $\mu + 1 \leqslant 2\mu$. Since $c_G \geqslant d_G/(8B)$, a union bound shows that a suitable A_{loc} exists. Enumerating the $2^{c_G d_G}$ binary matrices and testing all words of weight less than δd_G finds one in $\exp(\mathrm{poly}(B, \log(m+2)))$ bit operations.

At every vertex on both sides, impose the c_G rows of A_{loc} on its incident edge coordinates, ordered by their edge ends. These constraints define a Tanner code with local code $C_{\mathrm{loc}} = \ker_{\mathbb{F}_2} A_{\mathrm{loc}}$. Its incidence graph has one bit node for every edge of our graph and one local-code node for every vertex; each bit participates in the constraints at its two endpoints. Consider a nonzero edge word accepted by all these checks. Let U, V be the vertices touched by its nonzero edges on the two sides, and let h' be its weight. Each vertex in $U \cup V$ is incident to at least δd_G nonzero edges, so $h' \geqslant \delta d_G \max(|U|, |V|)$. Therefore

$$\delta d_G\sqrt{|U||V|} \leqslant h' \leqslant e_G(U,V) \leqslant \frac{d_G|U||V|}{q_G} + \varepsilon d_G\sqrt{|U||V|}.$$

Both U and V are nonempty. Dividing by $d_G\sqrt{|U||V|}$ yields $\sqrt{|U||V|} \geqslant (\delta - \varepsilon)q_G$, and hence

$$h' \geqslant \delta(\delta - \varepsilon)q_G d_G = \frac{q_G d_G}{20000B^2\mu^2} \geqslant \frac{m}{P}.$$

Shorten this Tanner code by keeping any m of the $q_G d_G$ edge coordinates, identifying them with $[m]$, and setting the remaining coordinates to zero. Restricting all the vertex checks to the retained coordinates gives $A^{(0)}$. Any word accepted by $A^{(0)}$ extends by zeros to an accepted edge word, so it has the required weight. Every row has at most $d_G \leqslant D_0$ nonzero entries. There are $2q_G c_G$ rows. If $q_G = 1$, then $d_G = m$ and

$$2c_G \leqslant \frac{m}{4B} + 2 \leqslant \frac{n}{4} + 2 \leqslant 3n.$$

If $q_G > 1$, then $d_G = D_0 \geqslant 8B$, and

$$2q_G c_G \leqslant \frac{q_G d_G}{4B} + 2q_G \leqslant \frac{q_G d_G}{2B} < \frac{m}{B} \leqslant n.$$

The searches over shifts, local matrices, and test words are sequential. Storing the current choices and the final rows uses $\mathrm{poly}(n, m, B)$ bits of working space. □

5.2. **Choosing additional equations by counting common zeros.** After the initial checks, every nonsatisfying assignment still passing violates at least m/P clauses. This lets a short random test catch each such assignment with constant probability. To make the choice deterministic we use the averaging principle underlying conditional expectations [Rag88].

Proof of Lemma 2.5. At a stage with current matrix A, enumerate the ordered lists

$$\gamma = ((e_1, c_1), \dots, (e_P, c_P)) \in \mathcal{C} := ([m] \times \{0,1\})^P.$$

The list γ represents the binary row h_γ with

$$(h_\gamma)_e = \Big(\sum_{j : e_j = e} c_j \Big) \bmod 2 \qquad (e \in [m]).$$

There are $(2m)^P$ lists; distinct lists are still counted separately when they represent the same row. Write $A; h_\gamma$ for the matrix obtained by appending h_γ to A.

Fix a nonsatisfying assignment x accepted by A. The initial rows remain in A, so $|\operatorname{supp} v(x)| \geqslant m/P$. For a uniform list γ, the probability that none of $e_1, \dots, e_P$ belongs to this support is at most

$$(1 - 1/P)^P \leqslant e^{-1} < 1/2.$$

Conditional on the indices hitting the support, choose one hit position j. After fixing every coefficient except c_j, the uniform bit c_j makes $h_\gamma \cdot v(x)$ uniform modulo 2. This remains true when some clause indices are repeated. Thus, for each such x, at least a quarter of the candidate lists give a row that rejects x.

Summing this bound over the nonsatisfying assignments still passing, we find

$$\mathbb{E}_\gamma D(A; h_\gamma) \leqslant \tfrac{3}{4} D(A).$$

For every candidate, compute $N(A; h_\gamma)$ by counting the common zeros of its polynomial system. Append a candidate attaining the minimum, breaking ties by enumeration order. Since $N(A; h_\gamma) = \#F + D(A; h_\gamma)$ for every γ, the chosen row leaves at most $\frac{3}{4} D(A)$ nonsatisfying assignments. After $3n$ rounds at most $(3/4)^{3n} 2^n = (27/32)^n < 1$ remain; their number is an integer, so it is zero. The procedure runs for a fixed number of rounds and never needs to know $D(A)$ or $\#F$.

The initial matrix has at most $3n$ rows, and every candidate is evaluated with at most $3n$ additional rows. A row listing s clauses expands, by (2.2), to at most $2^k s$ monomials on the original variables; duplicates cancel over $\mathbb{F}_2$. These observations give the claimed size and query bounds. □

6. An interpolation algorithm: technique overview

Sections 6–10 give an algorithm using polynomial interpolation over $\mathbb{Q}$ with exponent $O_k(\rho(n))$. It uses the same parity tests as the main theorem and a different method for counting their common zeros. This method separates the summation routine from the construction of the replacement forms: Section 7 gives the routine for arbitrary polynomial factors over a commutative $\mathbb{Q}$-algebra, with the replacement forms and measurements supplied. Its development is independent of the contraction recursion in Section 3. We first illustrate the routine by counting satisfying assignments of a 3-CNF in which each variable occurs in at most three clauses. The product of its clause polynomials has degree at most $3n$, so we can use interpolation to recover the exact count from sums over small groups of variables. Section 6.8 then uses disjoint sparsification and variable copying to count the satisfying assignments of an arbitrary k-CNF in $(n+m)^{O_k(1)} 2^{o(n)}$ time for every fixed $k \geqslant 2$. Taking $k = 3$ already refutes ETH. Section 6.9 explains how counting on the original variables gives the exponent $O_k(\rho(n))$ of Proposition 10.1.

6.1. **The bounded-occurrence setting.** Let F have n declared Boolean variables. Each clause contains at most three literals, and each variable appears in at most three clauses, counting positive and negative occurrences together. Our task is to count the satisfying assignments of F exactly.

We remove tautological clauses and repeated literals within a clause. An empty clause gives count zero immediately. These operations preserve the occurrence bound. For each remaining clause e, let $I_e \subseteq [n]$ be its variable set. For each variable that occurs in no clause, add a factor equal to one with scope $\{v\}$. Such a factor leaves the count unchanged and ensures that every variable has an assigned position in the construction. Let $\mathcal{F}$ index the clause polynomials and these constant-one factors. Then

$$|I_e| \leqslant 3, \qquad \#\{e \in \mathcal{F} : v \in I_e\} \leqslant 3, \qquad n \leqslant M := \sum_{e \in \mathcal{F}} |I_e| \leqslant 3n. \tag{6.1}$$

All sums below still range over the original n Boolean variables.

6.2. **A direct polynomial for the count.** Write $L_{e,v}(X_v) = X_v$ for a positive literal and $L_{e,v}(X_v) = 1 - X_v$ for a negative literal. The satisfaction polynomial of clause e is

$$\Phi_e((X_v)_{v \in I_e}) = 1 - \prod_{v \in I_e} (1 - L_{e,v}(X_v)).$$

For example, $x_a \vee \neg x_b \vee x_c$ gives

$$\Phi_e(X_a, X_b, X_c) = 1 - (1 - X_a)X_b(1 - X_c).$$

On Boolean inputs this polynomial is one when the clause is satisfied and zero otherwise. For a constant-one factor e, put $\Phi_e = 1$. Therefore

$$\#F = \sum_{x \in \{0,1\}^n} \prod_{e \in \mathcal{F}} \Phi_e((x_v)_{v \in I_e}). \tag{6.2}$$

The product has total degree at most $M \leqslant 3n$. This linear degree bound is the useful consequence of the three-occurrence assumption.

Directly evaluating (6.2) would be exhaustive search. If the variables admitted a partition into groups $S_1, \ldots, S_r$, each of size at most $2n/r$, with every clause contained in one group, we could instead count within groups. Writing N_g for the number of assignments to the variables in S_g that satisfy that group's clauses, independence gives

$$\#F = \prod_{g=1}^{r} N_g.$$

The local enumerations examine at most $\sum_{g=1}^{r} 2^{|S_g|} \leqslant r\, 2^{2n/r}$ assignments, with polynomial work per assignment. An arbitrary formula need not admit such a partition. We will replace the clause inputs so that groups interact only through a small collection of shared linear summaries. Fixing those summaries will again let us sum over each group separately.

6.3. **A polynomial family with easier evaluation points.** We introduce a parameter T. At $T = 0$, the replacements must restore the original factor inputs. At selected nonzero points $T = 1, \ldots, Q$, they should let us separate the groups by fixing shared linear summaries. Here Q is the number of selected points, and R will bound the degree of the replacement coefficients. We choose both parameters later.

Write $\vec{X} = (X_1, \ldots, X_n)$ for the vector of formal variables. For each occurrence $v \in I_e$, use the polynomial coefficient vector $\vec{P}_{e,v}(T) = (p_{e,v,1}(T), \ldots, p_{e,v,n}(T))$ to replace X_v in factor e by the scalar inner product

$$\langle \vec{P}_{e,v}(T), \vec{X} \rangle = \sum_{i=1}^{n} p_{e,v,i}(T) X_i.$$

We call this expression the *replacement linear form* for the occurrence of variable v in factor e. It replaces the input X_v in the factor Φ_e. We require

$$\langle \vec{P}_{e,v}(0), \vec{X} \rangle = X_v, \qquad \deg \vec{P}_{e,v} \leqslant R.$$

Different factors may use different replacement linear forms for the same variable. Define

$$H(T) = \sum_{x \in \{0,1\}^n} \prod_{e \in \mathcal{F}} \Phi_e\big((\langle \vec{P}_{e,v}(T), x \rangle)_{v \in I_e}\big) \in \mathbb{Q}[T]. \tag{6.3}$$

At zero every replacement restores the original variable, while the degree of each factor grows by at most a factor of R. Hence

$$H(0) = \#F, \qquad \deg H \leqslant MR \leqslant 3nR. \tag{6.4}$$

We will recover $H(0)$ from Taylor coefficients at $t = 1, \dots, Q$. The degree bound means that $O(1 + nR/Q)$ coefficients per point suffice, as Section 6.5 explains. Our goal is therefore to make R small relative to Q: many points with low-degree replacements require fewer coefficients at each point.

At a nonzero t, the value $\langle \vec{P}_{e,v}(t), x \rangle$ of a replacement linear form need not lie in $\{0, 1\}$, even when x is Boolean. The transformed factors therefore need not act as satisfaction indicators.

6.4. **Separating the sum by fixing measurements.** Fix $t \in [Q]$. We are given the coefficient vectors $\vec{P}_{e,v}(t)$, whose entries are now constants. We explain what structure lets us compute $H(t)$ quickly, and how to use it. Throughout this subsection, write $S_g = S_{t,g}$ and $C_g = C_{t,g}$.

Definition 6.1 (Partitions and measurements)**.** For the given coefficient vectors $\vec{P}_{e,v}(t)$ and parameters (r, τ, D), the groups S_g, factor assignments $\sigma_e(t)$, and measurement matrices C_g satisfy the following four conditions:

(1) *Small groups.* Partition the variables into disjoint groups $[n] = S_1 \sqcup \cdots \sqcup S_r$, with $|S_g| \leqslant 2n/r$.
(2) *One assigned group per factor.* Assign each factor e to one group $\sigma_e(t) \in [r]$. Condition 4 must hold with this same group for every replacement linear form of factor e.
(3) *Few bounded measurements.* Each group g has an integer matrix C_g with columns indexed by S_g and c_g rows. Each row specifies one linear measurement of the variables in S_g. There are at most $2\tau n$ rows in total, and the sum of absolute values of the coefficients in each row is at most D. For an assignment x, write

$$x_g = x|_{S_g}, \qquad y_g = C_g x_g, \qquad \vec{y} = (y_1, \dots, y_r).$$

(4) *Dependence on other groups only through measurements.* Suppose factor e is assigned to group $g = \sigma_e(t)$. Each replacement linear form of this factor may depend freely on the variables in S_g. Its dependence on any other group S_h must be a linear combination of that group's measurements y_h. Precisely, for every $v \in I_e$, there must be coefficient vectors $\vec{\lambda}_{e,v,h} \in \mathbb{Q}^{c_h}$, for $h \neq g$, such that, for every $x \in \mathbb{Q}^n$,

$$\langle \vec{P}_{e,v}(t), x \rangle = \langle \vec{P}_{e,v}(t)|_{S_g}, x_g \rangle + \sum_{h \neq g} \langle \vec{\lambda}_{e,v,h}, y_h \rangle. \tag{6.5}$$

Equivalently, $\vec{P}_{e,v}(t)|_{S_h}$ lies in the row space of C_h for every $h \neq g$.

The matrices C_h are shared by all factors. A factor's assigned group need not contain its original variables. Once $\vec{y}$ is fixed, however, the entire transformed factor depends only on the variables in its assigned group.

Find the coefficient vectors $\vec{\lambda}_{e,v,h}$ by solving the linear systems guaranteed by condition 4. For a measurement tuple $\vec{y} = (y_1, \dots, y_r)$, put

$$\mathcal{X}_{t,g}(y_g) = \{u \in \{0,1\}^{S_g} : C_g u = y_g\}, \qquad \mathcal{Y} = \prod_{g=1}^{r} \{-D, \dots, D\}^{c_g}.$$

For $u \in \mathcal{X}_{t,g}(y_g)$, use (6.5) to evaluate the transformed factors assigned to g from u and $\vec{y}$, and let $A_{t,g}(u; \vec{y})$ be their product. It is one if no factor is assigned to g. Each measurement of a Boolean assignment is an integer in $[-D, D]$. Hence each full Boolean assignment has one measurement tuple in $\mathcal{Y}$ and one restriction to each group, and its product of factors is the product of the values $A_{t,g}$ at these restrictions. Conversely, because $S_1, \dots, S_r$ partition $[n]$, any choice of $u_g \in \mathcal{X}_{t,g}(y_g)$ for every g combines into exactly one assignment x with $x_g = u_g$ for all g, and the measurement tuple of this x is $\vec{y}$. Thus, for each $\vec{y} \in \mathcal{Y}$, restriction to the groups is a bijection from the assignments with measurement tuple $\vec{y}$ onto $\prod_{g=1}^{r} \mathcal{X}_{t,g}(y_g)$. Distributing the product over these restrictions therefore gives

$$H(t) = \sum_{\vec{y} \in \mathcal{Y}} \prod_{g=1}^{r} \left(\sum_{u \in \mathcal{X}_{t,g}(y_g)} A_{t,g}(u; \vec{y}) \right). \tag{6.6}$$

An empty measurement fiber contributes a zero group sum.

Evaluate this identity one measurement tuple at a time. For each group, scan its Boolean assignments, retaining those with $C_g u = y_g$, and accumulate the inner sum. Multiply the group sums and add the product to the running total. There are at most $(2D+1)^{2\tau n}$ measurement tuples and at most $\sum_g 2^{|S_g|} \leqslant r 2^{2n/r}$ assignments to scan per tuple. The time is consequently

$$(2D+1)^{2\tau n} 2^{2n/r}$$

up to polynomial factors. Processing the groups sequentially retains only the current measurement tuple, assignment, and running sums and products; Lemma 7.3 gives the general routine.

For the parameter choices in Section 6.6, we can construct replacement forms of degree at most R that restore the original variables at zero and admit partitions, factor assignments, and measurement matrices with parameters (r, τ, D) at every $t \in [Q]$. Section 6.7 gives the construction. We first explain how these properties let us recover the count efficiently.

6.5. **Recovering the count from Taylor coefficients.** Our goal is to recover $H(0) = \#F$ from information at the points $t = 1, \dots, Q$. Since $\deg H \leqslant 3nR$, the Q values $H(t)$ alone may not determine H. We instead compute several Taylor coefficients at each point. Choose

$$q = \left\lfloor \frac{3nR}{Q} \right\rfloor, \qquad J = q + 1.$$

Suppose we know the coefficients of $z^0, \dots, z^q$ in $H(t+z)$ for every $t \in [Q]$. These data determine H uniquely: if two polynomials of degree at most $3nR$ have the same data, their difference is divisible by $(T-t)^J$ for every $t \in [Q]$, hence by $\prod_{t=1}^{Q} (T-t)^J$. This product has degree $QJ > 3nR$, so the difference must be zero. Each Taylor coefficient gives a linear equation in the coefficients of H. Solving this rational linear system recovers H, and its constant coefficient is the desired count $H(0)$.

It remains to compute these Taylor coefficients. Fix $t \in [Q]$; we want $H(t+z)$ through degree q in z. The row-space condition controls the replacement forms at t, but need not control their changes from t to $t+z$. We first give those changes a separate input, so that fixing this input preserves the separation of groups.

Keeping the shifted inputs separate. Write

$$\Delta \vec{P}_{e,v}(z) = \vec{P}_{e,v}(t+z) - \vec{P}_{e,v}(t).$$

Keep x as the Boolean assignment being summed over, and introduce a separate formal vector $\vec{X}$ for the changes. Define

$$\mathcal{G}_x(\vec{X}, z) = \prod_{e \in \mathcal{F}} \Phi_e\big((\langle \vec{P}_{e,v}(t), x\rangle + \langle \Delta\vec{P}_{e,v}(z), \vec{X}\rangle)_{v \in I_e}\big). \tag{6.7}$$

Here x indexes the polynomial, while $\vec{X}$ and z are its formal variables. Setting $\vec{X} = x$ restores each shifted input $\langle \vec{P}_{e,v}(t+z), x\rangle$, so

$$H(t+z) = \sum_{x \in \{0,1\}^n} \mathcal{G}_x(x, z).$$

On the other hand, fixing $\vec{X} = \mathbf{1}_U$ for some $U \subseteq [n]$ makes every correction $\langle \Delta\vec{P}_{e,v}(z), \mathbf{1}_U\rangle$ independent of x. All dependence on x then remains through $\langle \vec{P}_{e,v}(t), x\rangle$, where the row-space condition applies. Our task is therefore to recover the sum with $\vec{X} = x$ from sums with $\vec{X}$ fixed.

The degree bound makes this possible with few fixed substitutions. Every entry of $\Delta\vec{P}_{e,v}(z)$ is divisible by z, so each occurrence of an X_i in an expanded term contributes at least one factor of z. Thus

$$\deg_{\vec{X}}([z^j]\mathcal{G}_x(\vec{X}, z)) \leqslant j.$$

Thus, after discarding powers of z above q, every monomial involves at most q distinct variables from $\vec{X}$.

Recovering the shifted sum by inclusion–exclusion. Work modulo z^J, so that only the coefficients through z^q are retained. Replace every positive power of X_i in $\mathcal{G}_x$ by X_i. This preserves its values on Boolean inputs. The degree bound above gives, for Boolean $\vec{X}$,

$$\mathcal{G}_x(\vec{X}, z) \equiv \sum_{\substack{E \subseteq [n] \\ |E| \leqslant q}} a_E(x, z) \prod_{i \in E} X_i \pmod{z^J},$$

where $a_E(x, z)$ is the coefficient of the indicated monomial after this replacement. Evaluating at $\mathbf{1}_U$, the vector with ones precisely in U, sums the coefficients indexed by subsets of U. Inclusion–exclusion extracts each one:

$$a_E(x, z) \equiv \sum_{U \subseteq E} (-1)^{|E|-|U|} \mathcal{G}_x(\mathbf{1}_U, z) \pmod{z^J}.$$

When we set $\vec{X} = x$, the monomial $\prod_{i \in E} x_i$ equals one exactly when all the bits in E are one. Its contribution to the sum over x is therefore $\sum_{x:\, x_i = 1 \ (i \in E)} a_E(x, z)$, with x Boolean. Substituting the coefficient formula above reduces this to sums with the formal input fixed.

Computing the restricted sums. For $U \subseteq E \subseteq [n]$ with $|E| \leqslant q$, define

$$\mathcal{Z}_{t,E,U}(z) = \sum_{\substack{x \in \{0,1\}^n \\ x_i = 1 \ (i \in E)}} \mathcal{G}_x(\mathbf{1}_U, z) \quad \text{in } \mathbb{Q}[z]/(z^J). \tag{6.8}$$

Here U fixes the formal input $\vec{X} = \mathbf{1}_U$, while E fixes bits of the summation variable x to one. Summing the monomial contributions gives

$$H(t+z) \equiv \sum_{\substack{E \subseteq [n] \\ |E| \leqslant q}} \sum_{U \subseteq E} (-1)^{|E|-|U|} \mathcal{Z}_{t,E,U}(z) \pmod{z^J}. \tag{6.9}$$

This recovers all $J = q + 1$ Taylor coefficients together. Lemma 7.4 states the general identity used here.

Each sum $\mathcal{Z}_{t,E,U}$ can be evaluated by the algorithm of Section 6.4. With $\vec{X} = \mathbf{1}_U$ fixed, each correction $\langle \Delta\vec{P}_{e,v}(z), \mathbf{1}_U\rangle$ is independent of x. All dependence on x comes through the forms

$\langle \vec{P}_{e,v}(t), x\rangle$, so fixing $\vec{y}$ still separates the factors. The restrictions $x_i = 1$ are local to each group. The separation identity uses only addition, multiplication, and distributivity, so it also applies to these truncated polynomial values. Each factor is evaluated after substituting $\vec{X} = \mathbf{1}_U$, using only univariate polynomials in z of degree less than J and discarding higher powers after each multiplication. The multivariate expansion and its coefficients $a_E(x, z)$ are used only to justify the identity; the algorithm never constructs them.

Repeating this calculation at $t = 1, \dots, Q$ provides all the Taylor coefficients needed for the interpolation above, and therefore recovers $H(0) = \#F$.

6.6. **The three costs and the running time.** Jumping ahead, Section 6.7 will construct replacement forms that restore the original variables at zero, have degree at most R, and admit partitions, factor assignments, and measurement matrices with parameters (r, τ, D) at every $t \in [Q]$. For sufficiently large n, the construction takes polynomial time and achieves the following parameter bounds:

$$\begin{aligned} r &= \Theta\left(\frac{\sqrt{\log n}}{(\log\log n)^2}\right), \quad Q = \Theta\left(\frac{\log n}{\log\log n}\right), \\ R &= \Theta(\sqrt{\log n}), \qquad \tau = \Theta\left(\frac{1}{r\log n}\right), \qquad D = r^{2Q} \leqslant (2n)^{1/128}. \end{aligned} \tag{6.10}$$

We first show how these bounds give a fast counting algorithm; the next subsection will justify that they can all be achieved simultaneously.

Recall that $q = J - 1 = \lfloor 3nR/Q \rfloor$. For each evaluation point t, Section 6.5 computes one restricted sum for each pair $U \subseteq E$ with $|E| \leqslant q$. There are $\binom{n}{j}$ choices of an E of size j, and then 2^j choices of U. Each restricted sum enumerates measurement tuples and assignments within individual groups. Thus the three costs multiply to give, up to polynomial factors,

$$\underbrace{\sum_{j=0}^{\min(q,n)} 2^j \binom{n}{j}}_{\text{Taylor data}} \cdot \underbrace{(2D+1)^{2\tau n}}_{\text{measurements}} \cdot \underbrace{2^{2n/r}}_{\text{one group}}. \tag{6.11}$$

To bound the first cost, note that $1 \leqslant q \leqslant n/2$ for sufficiently large n, since $R/Q \to 0$. Using $\binom{n}{j} \leqslant (en/j)^j$ and the fact that $(2en/j)^j$ increases for $1 \leqslant j \leqslant n$, we obtain

$$\sum_{j=0}^{q} 2^j \binom{n}{j} \leqslant (q+1)\left(\frac{2en}{q}\right)^q.$$

Since $q = \Theta(nR/Q)$, the logarithm of this cost is $O(n(R/Q)\log(2Q/R))$. Taking logarithms of all three costs in (6.11) therefore gives

$$O\left(n\left[\frac{R}{Q}\log\frac{2Q}{R} + \tau\log(2D+1) + \frac{1}{r}\right]\right).$$

Each term is small under (6.10):

$$\begin{aligned} \frac{R}{Q}\log\frac{2Q}{R} &= O\left(\frac{\log\log n}{\sqrt{\log n}} \cdot \log\log n\right), \\ \tau\log(2D+1) &= O\left(\frac{1}{r\log n} \cdot \log n\right) = O(1/r), \\ \frac{1}{r} &= O\left(\frac{(\log\log n)^2}{\sqrt{\log n}}\right). \end{aligned}$$

Thus the bounded-occurrence instance can be counted in

$$n^{O(1)} 2^{O(n(\log\log n)^2/\sqrt{\log n})} \tag{6.12}$$

bit operations and polynomial space. The finitely many smaller inputs are handled by exhaustive search.

The number of points Q, the number of groups r, and the truncation order $J = O(1 + nR/Q)$ are polynomial in n. Hence the loops over points and groups, arithmetic on polynomials of degree less than J, and the final interpolation add only polynomially many operations per iteration of the three enumerations above.

For the bit-complexity and space bounds, see Appendix B.4.

6.7. **Constructing the low-degree replacement linear forms.** Consider one occurrence of X_v in a factor. At $T = 0$ its replacement must equal X_v. At each $t = 1, \dots, Q$, the factor must be able to evaluate that same replacement using only its assigned group and a few shared measurements. Choosing a different linear form at each point and interpolating would generally cost degree Q. To keep Taylor recovery cheap, we need degree R much smaller than Q.

Suppose a term uses a variable outside the factor's assigned group only at $t = 2$. Multiplying it by $1 - T/2$ preserves its value at zero and removes it at $t = 2$. More generally, one such factor per incompatible point suffices. We therefore seek a decomposition of each input into a part determined by measurements and terms that are incompatible at at most $R - 1$ points. We describe the mechanism here; Section 8 proves the construction for general factor scopes.

The auxiliary decomposition. An occurrence has a variable-group choice and a factor-group choice at every point. Index all possible choices by

$$\mathcal{U} = ([r] \times [r])^Q, \qquad D = r^{2Q}, \qquad u = ((a_u(t), d_u(t)))_{t\in[Q]}.$$

For placeholders $\vec{\zeta} = (\zeta_w)_{w\in\mathcal{U}}$, the replacement for ζ_u may freely use ζ_w at point t when $a_w(t) = d_u(t)$. Put

$$E_{u,w} = \{t \in [Q] : a_w(t) \neq d_u(t)\}.$$

We need a rational matrix M_0 and an integer measurement matrix $C \in \{0, 1, -1\}^{c\times D}$ satisfying

$$\begin{aligned} c &\leqslant D\tau/(3r), \qquad M_0\vec{\xi} = \vec{\xi} \quad (\vec{\xi} \in \ker C), \\ M_0[u, w] \neq 0 \quad &\Longrightarrow \quad |E_{u,w}| \leqslant R - 1. \end{aligned} \tag{6.13}$$

The kernel condition makes $(I_D - M_0)\vec{\zeta}$ a linear function of $C\vec{\zeta}$. Thus the identity

$$\zeta_u = ((I_D - M_0)\vec{\zeta})_u + \sum_w M_0[u, w]\zeta_w$$

has the desired form. Define its replacement by

$$\langle \vec{P}_u(T), \vec{\zeta}\rangle = ((I_D - M_0)\vec{\zeta})_u + \sum_{w:M_0[u,w]\neq 0} M_0[u, w] \prod_{t\in E_{u,w}} (1 - T/t)\, \zeta_w. \tag{6.14}$$

It equals ζ_u at zero and has degree at most $R - 1$. At point t, every surviving term outside the measured correction belongs to group $d_u(t)$. This is the replacement property proved in Lemma 8.4.

Why a matrix can satisfy both conditions. Let Π_s be the permutation matrix that swaps the two entries of pair s in every index, leaving all other pairs unchanged. The commuting swaps give

$$V_0 = \prod_{s=1}^{Q} \Pi_s, \qquad W_0 = \tfrac{1}{2}\sum_{s=1}^{Q}(I_D - \Pi_s), \qquad M_0 = V_0 f(W_0).$$

The complete swap V_0 connects matching groups at every point. A polynomial f of degree at most $R - 1$ can undo at most $R - 1$ swaps in any term of its expansion. Hence its nonzero entries have at most $R - 1$ incompatible points (Lemma 8.3).

To make M_0 fix most directions, use the tensor products of the symmetric and antisymmetric eigenvectors of a single swap. A basis vector with j antisymmetric factors has eigenvalues j for W_0 and $(-1)^j$ for V_0. For a uniformly chosen basis vector, j has distribution $W \sim \mathrm{Bin}(Q, (r - 1)/(2r))$.

Interpolate $f(j) = (-1)^j$ on an interval $\mathcal{I}$ of R consecutive integers around the mean. Then M_0 fixes the basis vectors with $j \in \mathcal{I}$. The transposes of the other, orthogonal basis vectors form C; their entries are $0, 1, -1$. Lemma 8.2 gives

$$c = D \Pr[W \notin \mathcal{I}].$$

The standard deviation of W is $\sqrt{Q \cdot \frac{r-1}{2r} \cdot \frac{r+1}{2r}} = \Theta(\sqrt{Q})$ for $r \geqslant 2$. Taking R much larger than $\sqrt{Q}$ therefore puts most basis vectors inside $\mathcal{I}$, so only a small fraction of them need to be measured. We can still take R much smaller than Q, so that Taylor recovery needs only $J = 1 + \lfloor 3nR/Q \rfloor = o(n)$ coefficients per point.

Returning to the original variables. Choose variable schedules $a_v \in [r]^Q$ and factor schedules $d_e \in [r]^Q$, so that occurrence (e, v) receives label $u(e, v) = ((a_v(t), d_e(t)))_{t \in [Q]}$. With M as in (6.1), Lemma 8.6 gives groups of size at most $2n/r$ and at most $h \leqslant 2M/D$ occurrences per label, provided

$$12D^2 \leqslant M, \qquad 2Qr^2 \leqslant n.$$

Place occurrences sharing a label into different copies of $\mathcal{U}$. In each used coordinate put its original variable X_v, and put zero in unused coordinates. These copies introduce no new Boolean variables. Apply (6.14) in the copy containing each occurrence. At zero it restores X_v; at t, its unmeasured terms use only variables in the factor's assigned group $\{v' : a_{v'}(t) = d_e(t)\}$.

Split each of the hc auxiliary measurements into its contributions from the r actual variable groups. Fixing these separate values allows independent summation within groups; fixing only their total would still couple the groups. Since $hD \leqslant 2M \leqslant 6n$, the resulting matrices have at most

$$rhc \leqslant rh\frac{D\tau}{3r} \leqslant 2\tau n$$

rows in total at each point. Each row has integral coefficients with absolute sum at most D: substitution and restriction cannot increase the absolute sum of the D entries of an auxiliary row. This supplies the partitions, factor assignments, and measurement matrices with parameters (r, τ, D) from Section 6.4. The full transfer and its kernel condition are proved in Section 8.5.

Parameters and construction cost. Put $\lambda = \lceil \log_2 n \rceil$ and $\nu = \lceil \log_2 \lambda \rceil$, and choose

$$\begin{aligned} r &= \left\lfloor \frac{\sqrt{\lambda}}{\nu^2} \right\rfloor, \quad Q = \left\lfloor \frac{\lambda}{128\nu} \right\rfloor, \quad R = 4\lceil \sqrt{\lambda} \rceil, \\ \tau &= \frac{1}{r\lambda}, \qquad D = r^{2Q}. \end{aligned} \tag{6.15}$$

These are precisely the parameters of Lemma 8.8 with $\beta = 1$. Apply that lemma with $b = w = 3$: each variable occurs in at most three clauses and each clause has at most three literals, so $b, w \leqslant \lambda$, and its hypotheses hold with $n \leqslant M \leqslant 3n$ for sufficiently large n. The lemma gives an interval

$$\mathcal{I} = \{A_0, \dots, A_0 + R - 1\}, \qquad A_0 = \left\lfloor \frac{Q(r-1)}{2r} - \frac{R-1}{2} \right\rfloor,$$

and states that these parameters satisfy the numerical hypotheses of Lemma 8.1. With $b = w = 3$, these hypotheses are the two balancing inequalities $2(b + w)D^2 = 12D^2 \leqslant M$ and $2Qr^2 \leqslant n$, and $\Pr[W \notin \mathcal{I}] \leqslant \tau/(rb) = \tau/(3r)$. It also gives $D \leqslant 2^{\lambda/128} \leqslant (2n)^{1/128}$ and the orders in (6.10). Lemma 8.1, including the coefficient estimates of Lemma 8.7, therefore constructs these forms, partitions, factor assignments, and measurement matrices in polynomial time. The parameter checks are exact; only finitely many sizes fail them and are handled by enumeration. This completes the construction needed for (6.12).

6.8. **A weaker algorithm for general #k-SAT.** The algorithm above already gives a subexponential algorithm for general #k-SAT: after disjoint sparsification, we reduce each instance directly to a 3-CNF in which every variable occurs at most three times. Both steps preserve the count, as follows.

Fix $k \geqslant 2$. We use the counting-preserving version [DHM+14, Appendix A] of sparsification [IPZ01], with quantitative bounds from [CIP06, Section 6] and [Cal09, Lemma 5.14 and equation (5.4)]. As recorded in Lemma 2.3, this expresses the input count as the sum of the counts of at most $\mathrm{poly}_k(n, \sigma^{-1})2^{\sigma n}$ formulas G. Each G has the same n declared variables, with at most

$$B = O_k\Big((\sigma^{-1}\log(2/\sigma))^{k-1}\Big)$$

occurrences per variable, and hence at most Bn nonempty clauses. An empty clause gives count zero immediately.

First turn G into a 3-CNF without changing its count. Keep clauses of size at most three. For each longer clause, compute its disjunction with binary OR gates and require the final output to be one. For a gate $z = u \vee v$, impose

$$(\neg u \vee z) \ \wedge \ (\neg v \vee z) \ \wedge \ (u \vee v \vee \neg z).$$

Here u, v are input literals or earlier gate outputs, and z is fresh. These clauses enforce equality with the OR, so the gate values are uniquely determined by the original assignment. Consequently, a satisfying assignment of G has exactly one extension to the new formula, and an assignment not satisfying G has none. This uses $O(k)$ gates per clause.

Next reduce the occurrences. If a variable x occurs $s \geqslant 2$ times in this 3-CNF, replace its occurrences by distinct copies $x_1, \dots, x_s$, preserving their signs, and add Tovey's implication cycle [Tov84, p. 86]:

$$(\neg x_1 \vee x_2) \ \wedge \cdots \wedge \ (\neg x_{s-1} \vee x_s) \ \wedge \ (\neg x_s \vee x_1).$$

The cycle forces all copies to agree. Each copy appears in one clause inherited from the 3-CNF and two cycle clauses, hence in exactly three clauses. Variables with at most one occurrence are kept unchanged, including unused declared variables. This standard variable-copy reduction [Gol08, Proposition 4.7] therefore also gives a bijection between satisfying assignments. The final formula $\widehat{G}$ has $\#\widehat{G} = \#G$ and

$$n \leqslant N := \#\{\text{variables of } \widehat{G}\} = O_k(Bn).$$

All its clauses have size at most three, including the two-literal cycle clauses, and every variable occurs at most three times. Thus it is exactly an instance of the setting of this overview.

Apply (6.12) to each $\widehat{G}$ and add the counts. When σ^{-1} is bounded by a fixed power of $\log n$, so is B, and the resulting time is

$$(n+m)^{O_k(1)}2^{O_k\left(\sigma n + Bn(\log\log n)^2/\sqrt{\log n}\right)}.$$

For example, choosing σ within a constant factor of $(\log n)^{-1/(2k)}$ yields

$$(n+m)^{O_k(1)}2^{O_k\left(n(\log\log n)^{k+1}/(\log n)^{1/(2k)}\right)} = (n+m)^{O_k(1)}2^{o(n)}$$

with polynomial space, processing the sparsification tree and the reduced instances one at a time. The parameter σ may be chosen rational.

6.9. **A faster algorithm for #k-SAT via counting solutions to sparse low-degree polynomial equations over $\mathbb{F}_2$.** The running-time exponent in Section 6.8 is $O_k(\sigma n + B\rho(n))$, where $\rho(n) = n(\log\log n)^2/\sqrt{\log n}$. If sparsification gave $B = O_k(1)$ at $\sigma = \Theta(\rho(n)/n)$, we would already have a $2^{O_k(\rho(n))}$-time algorithm for general #k-SAT up to polynomial factors: copying would use only $O_k(n)$ variables per leaf, and the number of leaves would fit within the same bound. At this choice of σ, however, the guarantee is only $B = (\log n)^{O_k(1)}$.

We obtain the $2^{O_k(\rho(n))}$ bound in two steps. First, the counting algorithm of this overview extends to systems of $O(n)$ polynomial equations over $\mathbb{F}_2$, each of degree at most k with polylogarithmically many monomials (Theorem 6.2). Second, even if a leaf G has Bn clauses, we can construct such a system on the original n variables whose common zeros are exactly the satisfying assignments of G. We illustrate the reduction first with a self-contained randomized construction, then explain the counting algorithm.

A randomized reduction to sparse polynomial equations. Write $v_e(x) = 1$ if x violates clause e, and 0 otherwise. A *parity test* on a set S of clauses is the polynomial equation $\sum_{e\in S} v_e(x) = 0$ over $\mathbb{F}_2$: it accepts if x violates an even number of clauses in S. Every satisfying assignment passes, so we need only catch every nonsatisfying one. If G has $m_G \leqslant n$ clauses, one test per clause suffices. Suppose $m_G > n$.

A short test can easily miss a single violated clause, but all assignments that violate only that clause give identical answers. Thus there are only m_G singleton patterns to catch. Use $8n$ independent tests. For each, draw $L = 2B\lceil \log_2(2m_G)\rceil$ clauses uniformly with replacement. For each draw, independently add its violation bit to the parity sum with probability $1/2$; two included copies cancel. Since $m_G \leqslant Bn$, each clause is drawn at least logarithmically often in expectation overall. For a fixed set of $w \geqslant 1$ violated clauses, let $u = Lw/m_G$ be the expected number of hits per test. If the sample hits the set, the coin on the first hit makes the parity odd with probability $1/2$, even with all other coins fixed. Thus the rejection probability is

$$p_w = \tfrac{1}{2}\big(1 - (1 - w/m_G)^L\big) \geqslant \tfrac{1}{2}(1 - e^{-u}).$$

We count *sets* for few violations and *assignments* for many:

- If $u \leqslant 1$, then $p_w \geqslant u/4$. There are at most m_G^w sets of size w, so the chance that at least one escapes all tests is at most $m_G^w e^{-8np_w} \leqslant m_G^w e^{-2nLw/m_G} \leqslant 16^{-w}$. Summing over $w \geqslant 1$ gives failure probability at most $1/15$.
- If $u > 1$, then $p_w > 1/4$. A union bound over the 2^n assignments bounds the chance that any assignment in this case escapes by $2^n(3/4)^{8n} < 1/4$.

Thus with probability greater than $2/3$, the common zeros are exactly the satisfying assignments of G. Each v_e is a product of at most k factors of the form x_j or $1 + x_j$, so each test has degree at most k and at most $2^k L$ monomials, polylogarithmic in n. Section 5 constructs short tests with the same solution set deterministically.

Counting the common zeros. Recall from Section 6.2 that $\Phi_e(x)$ is 1 if x satisfies clause e and 0 otherwise. What should replace this factor when we test a whole set S_i of clauses at once? First count how many clauses in *this test* are violated, using ordinary integer addition:

$$\Lambda_i(x) = \sum_{e\in S_i} v_e(x) = \sum_{e\in S_i} \big(1 - \Phi_e(x)\big).$$

The test passes exactly when $\Lambda_i(x)$ is even. Choose $\ell = n + 1$, so that a count of at most 2^n is determined by its residue modulo 2^ℓ. The complemented Beigel–Tarui polynomial p_ℓ (Lemma 4.1) is 1 modulo 2^ℓ on even integers and 0 on odd integers. For example, violating two clauses in S_i still passes the test: $p_\ell(2) \equiv 1 \pmod{2^\ell}$. Writing m_{test} for the number of tests, our counting expression becomes

$$\underbrace{\sum_x \prod_{e\in G} \Phi_e(x)}_{\text{one factor per clause}} \qquad \longrightarrow \qquad \underbrace{\sum_x \prod_{i=1}^{m_{\text{test}}} p_\ell(\Lambda_i(x))}_{\text{one factor per test}}.$$

The left side is $\#G$. The least nonnegative residue of the right side modulo 2^ℓ is the number of assignments passing every test. When the tests catch every nonsatisfying assignment, this is exactly

$\#G$. The same encoding works for any sparse polynomial equation $P_i(x) = 0$ over $\mathbb{F}_2$: take $L_i(x)$ to be the integer sum of its listed monomials. Again the equation holds exactly when this sum is even.

To reuse the Taylor step, we need a product of input degree $O(n)$. Multiplying the m_{test} test factors directly gives only the bound $O(km_{\text{test}}\ell)$, potentially quadratic in n. The key is to truncate the product *across all tests together*. Section 4.1 writes $p_\ell(y)$ as a sum of terms $e_j(y)h(y)^j$, where e_j has degree at most one and $h(y) = y(y-1)$ is even at every integer. A product term containing ℓ or more copies of h is therefore zero modulo 2^ℓ on Boolean assignments, even if those copies come from different tests. We keep just the terms with fewer than ℓ copies in total, tracking that number as we multiply. Each retained term has at most m_{test} linear factors and $\ell - 1$ quadratic factors in the L_i's. Since $\deg L_i \leqslant k$, its input degree is at most $k(m_{\text{test}} + 2(\ell-1)) = O(n)$.

For the replacement forms, what matters is which variables each factor reads. Where Section 6.3 used one form for each variable in a clause, we now use one for each variable in a test, reusing it throughout that test. Each sparse equation reads only polylogarithmically many variables. Branching on the values of the $O(n/\sqrt{\log n})$ most frequent variables leaves each remaining variable in only polylogarithmically many tests, as required by Section 8. On each branch, fixing the shared measurements again lets us sum one group at a time. Because the truncated product has input degree $O(n)$, the degree-R replacements give $\deg_T H = O(nR)$: the same $O(1 + nR/Q)$ Taylor coefficients per point recover the contributions at $T = 0$ with $0, 1, \ldots, \ell - 1$ copies of h. Their sum modulo 2^ℓ, taken as the least nonnegative residue, is the exact branch count. Adding the branch counts gives the number of common zeros.

Theorem 6.2 (Counting common zeros of sparse polynomials)**.** *Fix integers $k \geqslant 2$ and $c \geqslant 1$. Let $P_1, \ldots, P_{m_{\text{test}}}$ be polynomials over $\mathbb{F}_2$ in $n \geqslant 20$ Boolean variables, with $m_{\text{test}} \leqslant cn$. Suppose each P_i is given as a list of at most $(\log n)^c$ monomials of degree at most k. The exact number of common zeros in $\{0,1\}^n$ can be computed deterministically in $2^{O_{k,c}(n(\log\log n)^2/\sqrt{\log n})}$ bit operations and $n^{O_{k,c}(1)}$ space.*

Section 9 proves the theorem.

For the final system constructed for G, the number of common zeros is $\#G$. Section 5 uses only $2^{(\log n)^{O_k(1)}}$ calls to the counter per leaf, which preserves the running-time exponent in Theorem 6.2. Finally, choosing σ of order $(\log\log n)^2/\sqrt{\log n}$ in Section 10.2 keeps B polylogarithmic and the number of leaves within $2^{O_k(n(\log\log n)^2/\sqrt{\log n})}$. Their satisfying assignments are disjoint, so adding their counts gives the bound in Proposition 10.1.

7. A fast counting algorithm via shared linear summaries

Sections 7–9 prove Theorem 6.2. We begin with the sum from Section 6.2: for each Boolean assignment, multiply the factors, then add the products over all assignments. Here the factors can be any rational polynomials. We show how to compute the sum when the replacement forms and measurements are given. Section 8 constructs them using only the variables read by each factor. Section 9 supplies one factor per sparse polynomial equation.

7.1. Polynomial factors and the value to recover.

Let $\mathcal{F}$ index the factors, and let $I_f \subseteq [n]$ be the variables read by Φ_f. With rational coefficients our target is

$$Z = \sum_{x \in \{0,1\}^n} \prod_{f \in \mathcal{F}} \Phi_f((x_v)_{v \in I_f}). \tag{7.1}$$

For clause satisfaction indicators, Z is the number of satisfying assignments. The proof also works with coefficients in any commutative $\mathbb{Q}$-algebra $\mathcal{E}$: sums and products then take place in $\mathcal{E}$, and interpolation uses only rational scalars. Readers may take $\mathcal{E} = \mathbb{Q}$ throughout this section.

Different factors may use different replacements for the same variable. Give each incidence (f, v) its own indeterminate $Y_{f,v}$ and suppose

$$\deg_{\{Y_{f,v}\}}\left(\prod_{f\in\mathcal{F}}\Phi_f((Y_{f,v})_{v\in I_f})\right)\leqslant d. \tag{7.2}$$

This is the degree of the whole product computed over $\mathcal{E}$.

7.2. **The required rows, partitions, and measurements.** We use the same partitions and measurements as in Section 6.4, with a group assignment for each factor in place of each clause. The final condition below records the coefficient bound needed for exact arithmetic. Here $\widetilde{\xi}$ denotes extension by zero from a group to all n coordinates.

Definition 7.1. For the factor scopes $(I_f)_{f\in\mathcal{F}}$, let $r\geqslant 2$, $D\geqslant 1$, and $1\leqslant R<Q$ be integers, and let $0<\tau\leqslant 1/2$. We require the following data and properties.

(1) For each $t\in[Q]$, a partition $[n]=S_{t,1}\sqcup\cdots\sqcup S_{t,r}$, allowing empty parts, with $|S_{t,g}|\leqslant 2n/r$.
(2) For every factor $f\in\mathcal{F}$, a sequence $\sigma_f\in[r]^Q$. The value $\sigma_f(t)$ assigns factor f to a group at point t.
(3) For every $t\in[Q]$ and $g\in[r]$, an integer matrix $C_{t,g}\in\mathbb{Z}^{[c_{t,g}]\times S_{t,g}}$, where $c_{t,g}$ is a nonnegative integer. Each row has sum of absolute values at most D, and
$$\sum_{g=1}^{r}c_{t,g}\leqslant 2\tau n.$$
For an assignment x, the entries of $C_{t,g}(x|_{S_{t,g}})$ are the measurements on group g.
(4) For every $f\in\mathcal{F}$ and $v\in I_f$, a row $\vec{P}_{f,v}(T)\in\mathbb{Q}[T]^{1\times n}$ with $\vec{P}_{f,v}(0)=\delta_v$ and $\deg\vec{P}_{f,v}\leqslant R$. For every $t\in[Q]$, $g\in[r]\setminus\{\sigma_f(t)\}$, and $\xi\in\mathbb{Q}^{S_{t,g}}$, require
$$C_{t,g}\xi=\vec{0}\quad\Longrightarrow\quad\langle\vec{P}_{f,v}(t),\widetilde{\xi}\rangle=0,$$
where $\widetilde{\xi}\in\mathbb{Q}^n$ is the extension of ξ by zero.
(5) With $d_{\mathrm{den}}=(R-1)!Q!$, every coefficient of $d_{\mathrm{den}}\vec{P}_{f,v}$ is an integer with $O(Q\log Q+\log D)$ bits, with an absolute implied constant.

To interpret the kernel condition, fix t, f, and $v\in I_f$, and put $g=\sigma_f(t)$. Write $x_h=x|_{S_{t,h}}$. If $x,x'\in\mathbb{Q}^n$ agree outside $S_{t,h}$ for some $h\neq g$ and satisfy $C_{t,h}x_h=C_{t,h}x'_h$, then $x'_h-x_h\in\ker C_{t,h}$. The condition gives

$$\langle\vec{P}_{f,v}(t),x'\rangle=\langle\vec{P}_{f,v}(t),x\rangle.$$

Applying this one group at a time shows that the tuple $(\langle\vec{P}_{f,v}(t),x\rangle)_{v\in I_f}$ is determined by x_g and $(C_{t,h}x_h)_{h\neq g}$. Fixing all the measurement vectors therefore makes the entire factor a function of x_g. This applies to every polynomial factor Φ_f, including factors with coefficients in $\mathcal{E}$.

Each measurement of a Boolean vector lies in $[-D,D]\cap\mathbb{Z}$. We bound the sum of absolute values of the matrix entries because the construction can place the same Boolean coordinate in several auxiliary positions. After those positions are identified, entries of $C_{t,g}$ need not belong to $\{0,1,-1\}$. A rank bound alone would not control the number of possible measurements: the single row $(1,2,4,\dots,2^{n-1})$ distinguishes all 2^n Boolean vectors.

Proposition 7.2 (Common counting routine)**.** *Let $\mathcal{E}$ be a commutative $\mathbb{Q}$-algebra. Suppose the product of the factors in (7.1), computed over $\mathcal{E}$, satisfies the degree bound (7.2), and that the replacement rows, partitions, and measurements satisfy Definition 7.1. Put*

$$q=\left\lfloor\frac{dR}{Q}\right\rfloor,\qquad J=q+1. \tag{7.3}$$

Then Z can be computed by examining at most

$$2Qr\left(\sum_{j=0}^{\min(q,n)} 2^j \binom{n}{j}\right)(2D+1)^{2\tau n}2^{2n/r}$$

Boolean vectors. For each vector, the algorithm evaluates at most $|\mathcal{F}|$ factors in $\mathcal{E}[z]/(z^J)$. The additional operations per vector and the final interpolation take polynomially many operations in $\mathcal{E}$ and over $\mathbb{Q}$, in the parameters $n, |\mathcal{F}|, D, Q, R, J, r$ and the total scope length. Apart from the factor evaluators' own storage, the routine retains only polynomially many algebra elements, rational numbers and index bits.

The proof has three parts. Lemma 7.3 evaluates a sum after fixing the measurements. Lemma 7.4 expresses each Taylor coefficient as restricted sums of that form. Hermite interpolation over $\mathbb{Q}$ then recovers the value at zero. Sections 7.3–7.5 give these steps and their cost.

For the proof, use the supplied replacement rows, their degree bound R, and the equalities $\vec{P}_{f,v}(0) = \delta_v$ to form

$$H(T) = \sum_{x\in\{0,1\}^n} \prod_{f\in\mathcal{F}} \Phi_f\big((\langle \vec{P}_{f,v}(T), x\rangle)_{v\in I_f}\big) \in \mathcal{E}[T]. \tag{7.4}$$

The replacements restore the inputs at zero and multiply the degree by at most R. Thus $H(0) = Z$ and $\deg_T H \leqslant d_H := dR$. Our application will have $d = O(n)$, as in the bounded-occurrence example. This uses the degree of the product over $\mathcal{E}$ in (7.2). Substitution is an $\mathcal{E}$-algebra homomorphism: expand that product in the independent incidence inputs, then replace each input by its degree-at-most-R form. Each surviving degree-j monomial acquires T-degree at most jR. In particular, coefficients already zero in a truncated coefficient algebra remain zero after substitution.

7.3. **Separating the sum by fixing measurements.** Fixing the measurements makes every factor depend on only its assigned group. The following lemma states the resulting calculation over a commutative ring, so it applies equally to rational-valued factors and to factors in $\mathcal{E}$. The restrictions $x_i = 1$ will be needed for Taylor coefficients and are local to individual groups.

Lemma 7.3. *Let $[n] = S_1 \sqcup \cdots \sqcup S_r$ be a partition, allowing empty parts. For each $g \in [r]$, let C_g be an integer matrix with c_g rows and columns indexed by S_g. Suppose each row has sum of absolute values at most an integer $D \geqslant 0$, and write $c = \sum_g c_g$. For each $g \in [r]$, let $A_g : \mathbb{Q}^n \to \mathcal{E}$, where $\mathcal{E}$ is a commutative ring with identity. Suppose*

$$A_g(x+\xi) = A_g(x) \qquad (x \in \mathbb{Q}^n,\ \xi \in \ker C_h,\ h \neq g),$$

where ξ is extended by zero outside S_h. Then, for any $E \subseteq [n]$, the sum

$$Z_E = \sum_{\substack{x\in\{0,1\}^n \\ x_i=1\ (i\in E)}} \prod_{g=1}^{r} A_g(x)$$

can be evaluated by examining at most

$$2(2D+1)^c \sum_{g=1}^{r} 2^{|S_g|}$$

Boolean vectors, with at most one A_g evaluation, $\mathrm{poly}(n, r, c, \log(D+2))$ additional bit operations, and $O(1)$ ring operations per vector. Apart from the evaluators' own storage and $O(1)$ ring elements, the procedure uses $\mathrm{poly}(n, r, c, \log(D+2))$ bits of space.

Proof. For $y_g \in \mathbb{Z}^{c_g}$, define

$$\mathcal{X}_g(y_g) = \{u \in \{0,1\}^{S_g} : C_g u = y_g\}, \qquad \mathcal{Y} = \prod_{g=1}^{r} \{-D, \dots, D\}^{c_g}.$$

Every tuple of measurements of a Boolean assignment belongs to $\mathcal{Y}$, and $|\mathcal{Y}| = (2D+1)^c$. We first identify the contribution of a fixed measurement tuple $\vec{y} = (y_1, \dots, y_r)$. If every $\mathcal{X}_g(y_g)$ is nonempty, let $\bar{x}_g \in \mathcal{X}_g(y_g)$ be a representative for each g, and put

$$\mathcal{X}_{g,E}(y_g) = \{u \in \mathcal{X}_g(y_g) : u_i = 1 \text{ for all } i \in E \cap S_g\}.$$

If $x_h \in \mathcal{X}_h(y_h)$ for every h, then $x_h - \bar{x}_h \in \ker C_h$. Applying the invariance hypothesis successively for $h \neq g$ gives

$$A_g\Big(\sum_{h=1}^{r} x_h\Big) = A_g\Big(x_g + \sum_{h \neq g} \bar{x}_h\Big).$$

The assignments with measurement tuple $\vec{y}$ and fixed bits indexed by E form $\prod_g \mathcal{X}_{g,E}(y_g)$. Define

$$\Sigma_g(\vec{y}) = \sum_{x_g \in \mathcal{X}_{g,E}(y_g)} A_g\Big(x_g + \sum_{h \neq g} \bar{x}_h\Big). \tag{7.5}$$

Distributivity shows that these assignments contribute $\prod_g \Sigma_g(\vec{y})$ to Z_E. If $\mathcal{X}_{g,E}(y_g)$ is empty, then $\Sigma_g(\vec{y}) = 0$. The representatives $\bar{x}_g$ themselves need not satisfy the fixed-bit conditions indexed by E. Every Boolean assignment has a unique measurement tuple, so

$$Z_E = \sum_{\vec{y} \in \mathcal{Y}} \prod_{g=1}^{r} \Sigma_g(\vec{y}),$$

where the product is taken to be zero if any $\mathcal{X}_g(y_g)$ is empty.

The algorithm evaluates this identity one measurement tuple at a time, starting with $Z = 0_{\mathcal{E}}$. For each $\vec{y} \in \mathcal{Y}$, a first scan of each $\{0,1\}^{S_g}$ finds a representative $\bar{x}_g \in \mathcal{X}_g(y_g)$; if any such set is empty, the algorithm skips this tuple. Otherwise it initializes $p = 1_{\mathcal{E}}$. To compute $\Sigma_g(\vec{y})$, it initializes $s = 0_{\mathcal{E}}$ and scans $u \in \{0,1\}^{S_g}$ again. Whenever

$$C_g u = y_g, \qquad u_i = 1 \quad (i \in E \cap S_g),$$

the algorithm performs the update

$$s \leftarrow s + A_g\Big(u + \sum_{h \neq g} \bar{x}_h\Big).$$

At the end of this scan, $s = \Sigma_g(\vec{y})$, and the algorithm sets $p \leftarrow ps$. After all groups, it sets $Z \leftarrow Z + p$. After all tuples $\vec{y}$ have been processed, $Z = Z_E$.

For each $\vec{y}$, the scan finding the representatives and the scan computing the $\Sigma_g(\vec{y})$ each examine at most $\sum_g 2^{|S_g|}$ Boolean vectors. This gives the stated bound. Checking a vector's measurements and fixed bits uses $\mathrm{poly}(n, r, c, \log(D+2))$ bit operations: each matrix row has sum of absolute values at most D, so its partial sums have $O(\log(D+2))$ bits. There is at most one factor evaluation per vector. The retained state is $\vec{y}$, $(\bar{x}_1, \dots, \bar{x}_r)$, g, u, s, p, and Z. The tuple $\vec{y}$ contains c integers in $[-D, D]$, the representatives contain n bits in total, and $s, p, Z \in \mathcal{E}$. □

To evaluate (7.4) at a fixed t, group the transformed factors by $\sigma_f(t)$ and apply the lemma in $\mathcal{E}$ with $E = \varnothing$. Definition 7.1 gives its invariance hypothesis, so the sum is $H(t)$. As in the overview, the two exponential costs are the measurement choices and the Boolean assignments in one group.

7.4. **Taylor coefficients from fixed Boolean substitutions.** The Q values $H(1), \dots, H(Q)$ need not determine H. With q, J from (7.3), we have $q = \lfloor d_H/Q \rfloor$. Since $QJ > d_H$, it suffices to compute

$$\mathcal{H}_t(z) = \sum_{j=0}^{J-1} ([z^j]H(t+z))z^j \qquad (t \in [Q]). \tag{7.6}$$

Hermite interpolation recovers H, and hence $Z = H(0)$, from these QJ coefficients in $\mathcal{E}$. Indeed, the rational Hermite matrix for polynomials of degree less than QJ is invertible over $\mathbb{Q}$. Its inverse acts on elements of $\mathcal{E}$ by rational linear combinations.

Fix t. The replacement forms at $t + z$ need not satisfy the kernel condition at t. Write

$$\Delta\vec{P}_{f,v}(z) = \vec{P}_{f,v}(t+z) - \vec{P}_{f,v}(t).$$

Every entry is divisible by z. For fixed $x \in \mathbb{Q}^n$, introduce independent indeterminates $\vec{X}$ and replace each input by

$$\langle \vec{P}_{f,v}(t), x\rangle + \langle \Delta\vec{P}_{f,v}(z), \vec{X}\rangle. \tag{7.7}$$

Let

$$\mathcal{G}_x(\vec{X}, z) = \prod_{f\in\mathcal{F}} \Phi_f\Big((\langle \vec{P}_{f,v}(t), x\rangle + \langle \Delta\vec{P}_{f,v}(z), \vec{X}\rangle)_{v\in I_f}\Big). \tag{7.8}$$

Setting $\vec{X} = x$ gives $\sum_x \mathcal{G}_x(x, z) = H(t + z)$. Holding $\vec{X}$ fixed instead makes every correction independent of x, so the kernel condition at t still separates the factors. Moreover, each occurrence of $\vec{X}$ carries a factor of z, giving

$$\deg_{\vec{X}}([z^j]\mathcal{G}_x(\vec{X}, z)) \leqslant j. \tag{7.9}$$

The next identity recovers the substitution $\vec{X} = x$ from fixed Boolean substitutions and restrictions on the assignments being summed.

Lemma 7.4. *Let $q \geqslant 0$ be an integer and let $\mathcal{E}$ be a commutative ring with identity. For each $x \in \{0,1\}^n$, let $p_x(X_1, \dots, X_n)$ be a polynomial over $\mathcal{E}$ of degree at most q. Then*

$$\sum_{x\in\{0,1\}^n} p_x(x) = \sum_{\substack{E\subseteq[n]\\ |E|\leqslant q}} \sum_{U\subseteq E} (-1)^{|E|-|U|} \sum_{\substack{x\in\{0,1\}^n\\ x_i=1\ (i\in E)}} p_x(\mathbf{1}_U). \tag{7.10}$$

Here $\mathbf{1}_U \in \{0,1\}^n$ is the indicator vector of U.

Proof. For fixed x, we replace every positive power of X_i by X_i. This leaves Boolean evaluations unchanged and does not increase the degree. By Möbius inversion on subsets, the resulting multilinear polynomial has the expansion

$$\widetilde{p}_x(\vec{X}) = \sum_{E\subseteq[n]} c_{x,E} \prod_{i\in E} X_i, \qquad c_{x,E} = \sum_{U\subseteq E} (-1)^{|E|-|U|} p_x(\mathbf{1}_U).$$

The degree bound gives $c_{x,E} = 0$ whenever $|E| > q$. The inversion is valid over the stated ring because the coefficient of any original multilinear monomial indexed by A in the alternating sum for $c_{x,E}$ is zero unless $A \subseteq E$, and then is

$$\sum_{A\subseteq U\subseteq E} (-1)^{|E|-|U|} = \begin{cases} 1, & A = E,\\ 0, & A \neq E. \end{cases}$$

Only addition and multiplication by 1 and -1 are involved. Setting $\vec{X} = x$ and summing over x now gives the claimed identity: the monomial $\prod_{i\in E} x_i$ selects exactly the assignments with $x_i = 1$ for every $i \in E$. □

Apply Lemma 7.4 to $[z^j]\mathcal{G}_x$ over $\mathcal{E}$ for each $j < J$. All coefficients can use the same pairs $U \subseteq E$, $|E| \leqslant q$, so compute them together in

$$\mathcal{R} = \mathcal{E}[z]/(z^J).$$

For a fixed pair (E, U), let $\Phi_{f,U}(x; z)$ be factor f in (7.8) after setting $\vec{X} = \mathbf{1}_U$. Group these factors by their assignments:

$$A_{g,U}(x; z) = \prod_{f:\sigma_f(t)=g} \Phi_{f,U}(x; z). \tag{7.11}$$

Their product is $\mathcal{G}_x(\mathbf{1}_U, z)$. For $h \neq g$, changing x by a vector in $\ker C_{t,h}$ changes neither the original inputs of these factors nor their fixed corrections. Lemma 7.3 therefore computes the restricted sum with $x_i = 1$ for $i \in E$ by examining at most

$$2r(2D+1)^{2\tau n} 2^{2n/r} \tag{7.12}$$

Boolean vectors. Combining all the pairs gives

$$H(t+z) \equiv \sum_{\substack{E\subseteq[n]\\|E|\leqslant q}} \sum_{U\subseteq E} (-1)^{|E|-|U|} \sum_{\substack{x\in\{0,1\}^n\\ x_i=1\ (i\in E)}} \mathcal{G}_x(\mathbf{1}_U, z) \pmod{z^J}. \tag{7.13}$$

The group products and sums are formed in $\mathcal{R}$. The result is $\mathcal{H}_t(z)$; repeating at all t supplies the interpolation data. The algorithm never stores a polynomial in all n indeterminates: it sets $\vec{X} = \mathbf{1}_U$ first and stores only the J coefficients in $\mathcal{E}$.

7.5. **The three exponential costs.** For each $t \in [Q]$, the number of pairs $U \subseteq E \subseteq [n]$ with $|E| \leqslant q$ is

$$C_q = \sum_{s=0}^{\min(q,n)} 2^s \binom{n}{s}. \tag{7.14}$$

In particular, $C_0 = 1$ and $C_q = 3^n$ for $q \geqslant n$. For every $0 < \eta \leqslant 1$,

$$\eta^q C_q \leqslant \sum_{s=0}^{n} \binom{n}{s} (2\eta)^s = (1+2\eta)^n, \tag{7.15}$$

because $\eta^q \leqslant \eta^s$ for every retained index $s \leqslant q$.

Completion of the proof of Proposition 7.2. There are Q points, C_q pairs per point, and at most (7.12) Boolean vectors per pair. Forming each group product uses at most $|\mathcal{F}|$ factor evaluations. Addition and multiplication modulo z^J cost $O(J^2)$ operations on coefficients in $\mathcal{E}$. The preceding Taylor identity and interpolation prove correctness. Process the points, pairs, measurement tuples, and local assignments sequentially, retaining their current indices and accumulators. Together with the QJ interpolation coefficients, this gives the stated storage bound. Bit complexity additionally depends on the cost of evaluating the factors and on coefficient sizes; these are verified for the sparse polynomial application in Section 9.2. □

When $d = O(n)$, we have $d_H = O(nR)$ and $q = O(nR/Q)$. Taking $\eta = R/Q$ in (7.15) gives

$$\log C_q \leqslant q \log(Q/R) + n \log(1 + 2R/Q) = O\left(n \frac{R}{Q} \log \frac{2Q}{R}\right).$$

Consequently, if factor evaluation and coefficient arithmetic have polynomial bit cost, the running time is a polynomial factor times

$$2^{O\left(n\left[\frac{R}{Q}\log\frac{2Q}{R} + \tau\log(2D+1) + \frac{1}{r}\right]\right)}. \tag{7.16}$$

These are exactly the three costs from the overview. The parameter choice in Lemma 8.8 makes their sum $O((\log\log n)^2/\sqrt{\log n})$ when factor scopes and their maximum incidence are bounded by

a fixed power of $\log n$. The routine also applies when d is larger; then one keeps the actual value $q = \lfloor dR/Q \rfloor$ in the bound of Proposition 7.2.

8. Constructing low-degree replacement forms

The counting routine needs one replacement form for each pair (f, v) where factor f reads variable v. For the application in Section 9, f is a polynomial equation and its scope is the union of the variable sets of its monomials. This section uses only those scopes. We construct the rows $\vec{P}_{f,v}(T)$ and measurement matrices $C_{t,g}$ in Definition 7.1, together with the partitions $S_{t,g}$ and factor assignments σ_f. The main difficulty is to keep $\deg \vec{P}_{f,v} \leqslant R$ with R much smaller than Q. The degree bound for H grows linearly with R, so a small R/Q reduces the number of Taylor coefficients needed at each point. Choosing the rows independently at $T = 0, 1, \dots, Q$ and interpolating would generally give degree Q. We instead construct their values at all these points together.

The construction first gives polynomial rows on a complete array of auxiliary coordinates. We then place each factor–variable pair in a copy of that array. Several positions can contain the same Boolean variable x_v; their values are always equal, and they do not become independent variables in the counting sum.

8.1. **The auxiliary coordinates.** We first specify the larger coordinate set $\mathcal{U}$ on which the auxiliary rows $\vec{P}_u(T)$ will act, together with the incidence bounds needed to place the actual pairs (f, v) into copies of it. Let $(I_f)_{f \in \mathcal{F}}$ be the factor scopes, with $I_f \subseteq [n]$, and write

$$\begin{aligned} \mathcal{J} &= \{(f, v) : f \in \mathcal{F},\ v \in I_f\}, \qquad M = |\mathcal{J}|, \\ |I_f| &\leqslant w \quad (f \in \mathcal{F}), \qquad \#\{f : v \in I_f\} \leqslant b \quad (v \in [n]). \end{aligned} \tag{8.1}$$

Thus w bounds the number of variables in a factor's scope, and b bounds the number of factors containing one variable. The scope bound w need not bound the degree of the factor polynomial. A factor with empty scope is constant and needs no auxiliary position; assign such a factor to group one at every point. If necessary, adjoin a distinct constant-one factor with scope $\{v\}$ for every $v \in [n]$. This changes neither the product nor the count, and increases the maximum incidence by at most one. With b denoting the bound after this addition, we assume

$$n \geqslant 1, \qquad b, w \geqslant 1, \qquad n \leqslant M \leqslant bn.$$

Throughout this section, $r \geqslant 2$ and $1 \leqslant R < Q$ are integers and $0 < \tau \leqslant 1/2$.

The output needed by the counting routine is the following.

Lemma 8.1. *Let $n, b, w \geqslant 1$ be integers, and let $(I_f)_{f \in \mathcal{F}}$ be subsets of $[n]$ of size at most w, with each $v \in [n]$ belonging to at most b sets. Put $M = \sum_f |I_f|$ and assume $n \leqslant M \leqslant bn$. Let $r \geqslant 2$ and $1 \leqslant R < Q$ be integers, let $0 < \tau \leqslant 1/2$, and put $D = r^{2Q}$. Suppose*

$$2(b + w)D^2 \leqslant M, \qquad 2Qr^2 \leqslant n,$$

and an interval $\mathcal{I} = \{A_0, \dots, A_0 + R - 1\} \subseteq \{0, \dots, Q\}$ satisfies

$$\Pr[W \notin \mathcal{I}] \leqslant \frac{\tau}{rb}, \qquad W \sim \mathrm{Bin}\left(Q, \frac{r-1}{2r}\right).$$

One can deterministically construct all the data in Definition 7.1 in $\mathrm{poly}(n, M, |\mathcal{F}|, D, Q, r, b, w)$ bit operations.

The auxiliary matrix and rows are constructed in the next two subsections. Lemma 8.6 then balances the actual factor incidences, and Section 8.5 transfers the rows and measurements to the original variables. The coefficient estimate in Lemma 8.7 completes the proof.

At each point $t \in [Q]$, the variable x_v will belong to one group, while a factor containing v may be assigned to another. We will choose

$$a_v \in [r]^Q \quad (v \in [n]), \qquad d_f \in [r]^Q \quad (f \in \mathcal{F}),$$

where $a_v(t)$ specifies the group containing x_v and $d_f(t)$ specifies the group assigned to factor f. The position for (f, v) will have label (a_v, d_f). In particular, all positions containing x_v will have the same first sequence a_v.

Before choosing a_v and d_f, take the complete set of possible labels:

$$\mathcal{U} = ([r]^Q)^2, \qquad u = (a_u, d_u), \qquad D = |\mathcal{U}| = r^{2Q}. \tag{8.2}$$

An auxiliary vector $\zeta \in \mathbb{Q}^{\mathcal{U}}$ has one rational coordinate ζ_u for each pair u. The label u stays fixed as ζ varies. For $t \in [Q]$ and $g \in [r]$, define

$$S^{\mathcal{U}}_{t,g} = \{u \in \mathcal{U} : a_u(t) = g\}. \tag{8.3}$$

For fixed t, these sets partition $\mathcal{U}$ into r sets of size D/r.

For each $u \in \mathcal{U}$, we seek a polynomial row $\vec{P}_u(T) \in \mathbb{Q}[T]^{1\times\mathcal{U}}$ of degree at most $R-1$ with $\vec{P}_u(0) = \delta_u$. At $T = t$, the value $\langle \vec{P}_u(t), \zeta\rangle$ should be determined by

$$\zeta|_{S^{\mathcal{U}}_{t,d_u(t)}} \quad \text{and} \quad C^{\mathcal{U}}\zeta,$$

where $C^{\mathcal{U}}$ is one integer matrix, independent of u, t, with entries in $\{0, 1, -1\}$ and few rows. Later, using $u_{f,v} = (a_v, d_f)$ will make every input of factor f use the group $d_f(t)$. The choices of a_v, d_f and the placement into copies are deferred to Sections 8.4 and 8.5, respectively.

8.2. **The matrices M_0 and $C^{\mathcal{U}}$.** To construct the rows $\vec{P}_u(T)$, we seek a matrix $M_0 \in \mathbb{Q}^{\mathcal{U}\times\mathcal{U}}$ that fixes $\ker C^{\mathcal{U}}$ and satisfies

$$M_0[u, y] \neq 0 \quad \Longrightarrow \quad \big|\{s \in [Q] : a_y(s) \neq d_u(s)\}\big| \leqslant R - 1. \tag{8.4}$$

The identity

$$\zeta_u = ((I - M_0)\zeta)_u + \sum_{y\in\mathcal{U}} M_0[u, y]\zeta_y \tag{8.5}$$

explains both requirements. If M_0 fixes $\ker C^{\mathcal{U}}$, the first term is determined by $C^{\mathcal{U}}\zeta$; we will leave it independent of T. In the sum, multiply each nonzero $M_0[u, y]$ by a polynomial that is one at zero and vanishes at every t with $a_y(t) \neq d_u(t)$. The bound (8.4) permits degree at most $R - 1$. At $T = t$, the sum will then use only the coordinates $y \in S^{\mathcal{U}}_{t,d_u(t)}$.

For $s \in [Q]$, let $\vartheta_s : \mathcal{U} \to \mathcal{U}$ exchange the entries $a_u(s)$ and $d_u(s)$, leaving every other entry unchanged. Define its permutation matrix Π_s by

$$(\Pi_s\zeta)_u = \zeta_{\vartheta_s(u)}.$$

The exchanged pair $\vartheta_s(u)$ belongs to $\mathcal{U}$ because this set contains every pair of sequences in $[r]^Q$. The matrices Π_s commute and satisfy $\Pi_s^2 = I$. Their product

$$V_0 = \prod_{s=1}^{Q} \Pi_s$$

exchanges the complete sequences a_u and d_u. Thus $V_0[u, y] \neq 0$ implies $a_y(s) = d_u(s)$ for every s; it satisfies (8.4) with no mismatches. We will modify V_0 so that the resulting matrix also fixes the kernel of a short list of integer measurements.

Define

$$W_0 = \frac{1}{2}\sum_{s=1}^{Q}(I - \Pi_s). \tag{8.6}$$

On a common eigenvector ψ of the Π_s, let j be the number of indices s for which $\Pi_s\psi = -\psi$. Then

$$W_0\psi = j\psi, \qquad V_0\psi = (-1)^j\psi, \qquad V_0 p(W_0)\psi = (-1)^j p(j)\psi$$

for every polynomial p. Therefore $V_0 p(W_0)$ fixes ψ whenever $p(j) = (-1)^j$. We will construct an explicit common eigenbasis in which the proportion of vectors with a given j is $\Pr[W = j]$ for $W \sim \mathrm{Bin}(Q, (r-1)/(2r))$.

For an interval $\mathcal{I} = \{A_0, \dots, A_0 + R - 1\} \subseteq \{0, \dots, Q\}$, we can enforce the equalities $f_{\mathcal{I}}(j) = (-1)^j$ for $j \in \mathcal{I}$ with a polynomial of degree at most $R-1$. We use the explicit interpolation formula

$$f_{\mathcal{I}}(z) = (-1)^{A_0} \sum_{j=0}^{R-1} (-2)^j \binom{z - A_0}{j}, \qquad M_0 = V_0 f_{\mathcal{I}}(W_0), \tag{8.7}$$

where

$$\binom{z - A_0}{j} = \frac{1}{j!} \prod_{a=0}^{j-1} (z - A_0 - a)$$

and the empty product is one. For $0 \leqslant q < R$, the binomial theorem gives $f_{\mathcal{I}}(A_0 + q) = (-1)^{A_0}(1-2)^q = (-1)^{A_0+q}$.

Lemma 8.2. *For $\mathcal{U}$, $\mathcal{I}$ and M_0 as above, there is a matrix $C^{\mathcal{U}} \in \{0, 1, -1\}^{[c_{\mathcal{U}}] \times \mathcal{U}}$ such that*

$$c_{\mathcal{U}} \leqslant D \Pr[W \notin \mathcal{I}], \qquad W \sim \mathrm{Bin}\left(Q, \frac{r-1}{2r}\right),$$

and $M_0\xi = \xi$ for every $\xi \in \ker C^{\mathcal{U}}$.

Proof. For $(\alpha, \delta) \in [r]^2$, let $e_{\alpha\delta}$ be the indicator function of that ordered pair. Exchanging its two entries has the orthogonal eigenbasis

$$e_{\alpha\alpha} \quad (\alpha \in [r]), \qquad e_{\alpha\delta} + e_{\delta\alpha}, \quad e_{\alpha\delta} - e_{\delta\alpha} \quad (1 \leqslant \alpha < \delta \leqslant r).$$

The first two types have eigenvalue $+1$, and the third has eigenvalue -1. There are $r(r+1)/2$ vectors with eigenvalue $+1$ and $r(r-1)/2$ with eigenvalue -1.

For each $s \in [Q]$, choose one of these functions $\psi_s : [r]^2 \to \{0, 1, -1\}$, and define

$$\psi(u) = \prod_{s=1}^{Q} \psi_s(a_u(s), d_u(s)) \qquad (u \in \mathcal{U}).$$

The $D = r^{2Q}$ choices give nonzero orthogonal vectors: their inner products factor into the inner products of the functions ψ_s. They form a basis of $\mathbb{Q}^{\mathcal{U}}$ with entries in $\{0, 1, -1\}$. If exactly j of the chosen ψ_s have eigenvalue -1, then $W_0\psi = j\psi$ and $V_0\psi = (-1)^j\psi$. Hence M_0 fixes ψ whenever $j \in \mathcal{I}$.

To make $\ker C^{\mathcal{U}}$ a space fixed by M_0, we define the rows of $C^{\mathcal{U}}$ to be the transposes of the basis vectors with $j \notin \mathcal{I}$. A fraction $(r-1)/(2r)$ of the choices at each position have eigenvalue -1, independently across positions. The fraction of the D basis vectors retained as rows is therefore $\Pr[W \notin \mathcal{I}]$, proving the row bound. If $C^{\mathcal{U}}\xi = \vec{0}$, then ξ is orthogonal to every retained basis vector. It is a linear combination of the basis vectors with $j \in \mathcal{I}$, all of which M_0 fixes. □

Fix the matrix $C^{\mathcal{U}}$ constructed in the proof of Lemma 8.2. For a bound $\epsilon > 0$, the condition

$$\Pr\left[\mathrm{Bin}\left(Q, \frac{r-1}{2r}\right) \notin \mathcal{I}\right] \leqslant \epsilon \tag{8.8}$$

thus gives $c_{\mathcal{U}} \leqslant \epsilon D$. After placing the factor inputs, we will use $\epsilon = \tau/(rb)$. We next prove the other requirement on M_0.

Lemma 8.3. *For all $u, y \in \mathcal{U}$, if $M_0[u,y] \neq 0$, then*

$$\left|\{s \in [Q] : a_y(s) \neq d_u(s)\}\right| \leqslant R-1.$$

Proof. Expand $f_{\mathcal{I}}(W_0)$ in the commuting matrices Π_s. Since $\deg f_{\mathcal{I}} \leqslant R-1$ and $\Pi_s^2 = I$, each resulting monomial is a scalar multiple of $\prod_{s\in J}\Pi_s$ for some $J \subseteq [Q]$ with $|J| \leqslant R-1$. Multiplication by V_0 gives

$$V_0 \prod_{s\in J}\Pi_s = \prod_{s\notin J}\Pi_s.$$

A nonzero (u,y) entry of this permutation matrix requires $a_y(s) = d_u(s)$ for every $s \notin J$. There are at most $R-1$ other positions. If more than $R-1$ positions disagree, every summand has zero (u,y) entry, so $M_0[u,y] = 0$ as well. □

8.3. **The polynomial rows.** We now use M_0 to construct the auxiliary rows $\vec{P}_u(T)$ sought in Section 8.1. Lemma 8.2 makes $(I-M_0)\zeta$ a function of $C^{\mathcal{U}}\zeta$. Lemma 8.3 bounds the number of points at which a nonzero term $M_0[u,y]\zeta_y$ uses a coordinate outside $S^{\mathcal{U}}_{t,d_u(t)}$. We now multiply that term by a polynomial vanishing at those points and equal to one at zero.

For $M_0[u,y] \neq 0$, define

$$E_{u,y} = \{s \in [Q] : a_y(s) \neq d_u(s)\}, \qquad \varphi_{u,y}(T) = \prod_{s\in E_{u,y}} \left(1 - \frac{T}{s}\right). \tag{8.9}$$

The empty product is one. Then $\deg \varphi_{u,y} \leqslant R-1$, $\varphi_{u,y}(0) = 1$, and $\varphi_{u,y}(t) = 0$ whenever $a_y(t) \neq d_u(t)$. Define

$$\vec{P}_u(T) = \delta_u(I - M_0) + \sum_{\substack{y\in\mathcal{U}\\ M_0[u,y]\neq 0}} M_0[u,y]\varphi_{u,y}(T)\delta_y. \tag{8.10}$$

This row acts as a linear form in ζ, with coefficients that are polynomials in the single parameter T.

Lemma 8.4. *The rows $\vec{P}_u(T)$ of (8.10) satisfy $\vec{P}_u(0) = \delta_u$ and $\deg \vec{P}_u \leqslant R-1$ for every $u \in \mathcal{U}$. For every $t \in [Q]$, the linear form $\langle \vec{P}_u(t), \zeta\rangle$ on $\mathbb{Q}^{\mathcal{U}}$ is determined by $\zeta|_{S^{\mathcal{U}}_{t,d_u(t)}}$ and $C^{\mathcal{U}}\zeta$. Moreover, let $C^{\mathcal{U}}_{t,g}$ be the restriction of $C^{\mathcal{U}}$ to the columns $S^{\mathcal{U}}_{t,g}$. For $g \neq d_u(t)$ and $\xi \in \mathbb{Q}^{S^{\mathcal{U}}_{t,g}}$,*

$$C^{\mathcal{U}}_{t,g}\xi = \vec{0} \quad\Longrightarrow\quad \langle \vec{P}_u(t), \widetilde{\xi}\rangle = 0,$$

where $\widetilde{\xi}$ extends ξ by zero to $\mathcal{U}$.

Proof. The degree bound follows from that of $\varphi_{u,y}$. At zero,

$$\vec{P}_u(0) = \delta_u(I - M_0) + \delta_u M_0 = \delta_u.$$

Applying (8.10) to ζ gives

$$\langle \vec{P}_u(T), \zeta\rangle = ((I-M_0)\zeta)_u + \sum_{\substack{y\in\mathcal{U}\\ M_0[u,y]\neq 0}} M_0[u,y]\varphi_{u,y}(T)\zeta_y. \tag{8.11}$$

At $T = t$, all summands with $a_y(t) \neq d_u(t)$ vanish. If $\widetilde{\xi}$ is supported on $S^{\mathcal{U}}_{t,g}$ with $g \neq d_u(t)$, each remaining summand is zero on $\widetilde{\xi}$. The hypothesis $C^{\mathcal{U}}_{t,g}\xi = \vec{0}$ also gives $C^{\mathcal{U}}\widetilde{\xi} = \vec{0}$, so $(I-M_0)\widetilde{\xi} = \vec{0}$ by Lemma 8.2. This proves the final implication of the lemma.

For the final assertion, two vectors with the same measurements under $C^{\mathcal{U}}$ differ by an element of its kernel. Thus $(I-M_0)\zeta$ is determined by $C^{\mathcal{U}}\zeta$. The remaining terms in (8.11) at $T = t$ use only $\zeta|_{S^{\mathcal{U}}_{t,d_u(t)}}$. □

Fix t and write $\zeta_g = \zeta|_{S^{\mathcal{U}}_{t,g}}$ and $C^{\mathcal{U}}_{t,g} = C^{\mathcal{U}}|_{S^{\mathcal{U}}_{t,g}}$, the restriction to these columns. Then

$$C^{\mathcal{U}}\zeta = \sum_{g=1}^{r} C^{\mathcal{U}}_{t,g}\zeta_g. \tag{8.12}$$

Fixing each vector $C^{\mathcal{U}}_{t,g}\zeta_g$ therefore fixes $C^{\mathcal{U}}\zeta$ while imposing separate conditions on the r auxiliary coordinate groups. When all vectors $C^{\mathcal{U}}_{t,g}\zeta_g$ are fixed, $\langle \vec{P}_u(t), \zeta\rangle$ is determined by $\zeta|_{S^{\mathcal{U}}_{t,d_u(t)}}$. We will obtain the corresponding separation on the actual Boolean variables by placing (f, v) at a coordinate labeled (a_v, d_f).

Remark 8.5. We illustrate how (8.10) confines dependence outside the assigned group to one linear measurement. Take $Q = R = 2$, outside the regime $R < Q$ used for the running-time bound, and $r = 2$. Use $\{0, 1\}$ as the group indices and $\mathcal{I} = \{0, 1\}$, so $f_{\mathcal{I}}(z) = 1 - 2z$. Starting with the label $(00, 11)$, the two exchanges generate four coordinates:

$$\zeta_0 = \zeta_{(00,11)}, \quad \zeta_1 = \zeta_{(10,01)}, \quad \zeta_2 = \zeta_{(01,10)}, \quad \zeta_{12} = \zeta_{(11,00)}.$$

Both exchanges preserve this set of four coordinates, so we can restrict the calculation to it. On this restriction, $M_0 = \Pi_1 + \Pi_2 - \Pi_1\Pi_2$ fixes the kernel of

$$c(\zeta) = \zeta_0 - \zeta_1 - \zeta_2 + \zeta_{12}.$$

The row indexed by $u = (00, 11)$ gives

$$\langle \vec{P}_u(T), \zeta\rangle = \zeta_0 - \frac{T}{2}\zeta_1 - T\zeta_2.$$

It equals ζ_0 at zero, while

$$\langle \vec{P}_u(1), \zeta\rangle = c(\zeta) + \frac{1}{2}\zeta_1 - \zeta_{12}, \qquad \langle \vec{P}_u(2), \zeta\rangle = c(\zeta) - \zeta_2 - \zeta_{12}.$$

At $t = 1$, the labels of ζ_1, ζ_{12} satisfy $a_y(1) = 1 = d_u(1)$. At $t = 2$, the labels of ζ_2, ζ_{12} satisfy $a_y(2) = 1 = d_u(2)$. Fixing $c(\zeta)$ therefore leaves $\langle \vec{P}_u(t), \zeta\rangle$ dependent only on coordinates in its assigned group. Here $c(\zeta)$ plays the role of $C^{\mathcal{U}}\zeta$ on the four-coordinate restriction. The full construction must give this property for every $u \in \mathcal{U}$ with at most ϵD measurement rows.

8.4. **Choosing the variable and factor groups.** We now choose the families $(a_v)_{v\in[n]}$ and $(d_f)_{f\in\mathcal{F}}$ from Section 8.1, and set

$$u_{f,v} = (a_v, d_f) \qquad ((f, v) \in \mathcal{J}).$$

These choices have two requirements. First, if a label occurs c_u times, its incidences need c_u different positions in copies of the auxiliary array. Thus the number of copies needed is

$$c_u = \#\{(f, v) \in \mathcal{J} : u_{f,v} = u\}, \qquad h = \max_{u\in\mathcal{U}} c_u.$$

Second, the actual variable group $\{v : a_v(t) = g\}$ must be small. Bounding h alone would bound the number of incidences in a group, not give the required bound on its number of Boolean variables. The next lemma controls both quantities.

Lemma 8.6. *Let $(I_f)_{f\in\mathcal{F}}$ satisfy (8.1), with $n \leqslant M \leqslant bn$, and put $D = r^{2Q}$ for integers $r \geqslant 2$ and $Q \geqslant 1$. Suppose*

$$2(b + w)D^2 \leqslant M, \qquad 2Qr^2 \leqslant n. \tag{8.13}$$

One can deterministically choose $a_v, d_f \in [r]^Q$, with $d_f(t) = 1$ for factors of empty scope, so that

$$\max_{u\in\mathcal{U}} \#\{(f, v) \in \mathcal{J} : (a_v, d_f) = u\} \leqslant \frac{2M}{D},$$

$$\#\{v \in [n] : a_v(t) = g\} \leqslant \frac{2n}{r} \qquad (t \in [Q],\ g \in [r]).$$

The construction takes $\mathrm{poly}(n, M, |\mathcal{F}|, D, Q, r, b, w)$ *bit operations.*

Proof. We first estimate the effect of random labels; we will then fix their values deterministically. For this estimate, all a_v and all d_f with nonempty I_f are independent and uniform on $[r]^Q$. For empty I_f, we set d_f identically one. Each incidence label $u_{f,v} = (a_v, d_f)$ is uniform on $\mathcal{U}$. Put $n_{t,g} = \#\{v : a_v(t) = g\}$, and consider

$$\Phi = \sum_{u \in \mathcal{U}} (c_u - M/D)^2, \qquad \Xi = \sum_{t=1}^{Q} \sum_{g=1}^{r} (n_{t,g} - n/r)^2.$$

Their expansions as sums of indicators are

$$\Phi = \sum_{(f,v),(f',v') \in \mathcal{J}} \mathbb{1}_{\{(a_v, d_f) = (a_{v'}, d_{f'})\}} - \frac{M^2}{D},$$
$$\Xi = \sum_{t=1}^{Q} \sum_{v,v' \in [n]} \mathbb{1}_{\{a_v(t) = a_{v'}(t)\}} - \frac{Qn^2}{r}.$$

Two incidence labels are independent unless they share a factor or a variable. For each (f, v), there are at most $b + w$ incidences sharing f or v, including (f, v) itself. Independent pairs have equality probability $1/D$; the other pairs have probability at most one. Hence

$$\mathbb{E}\Phi \leqslant M(b + w).$$

For each t and $v \neq v'$, the values $a_v(t), a_{v'}(t)$ are independent and uniform on $[r]$. The second expansion therefore gives

$$\mathbb{E}\Xi = Qn(1 - 1/r) \leqslant Qn.$$

To obtain the labels deterministically, we fix the a_v and the d_f for nonempty I_f one at a time. At each step we choose a value that does not increase the conditional expectation of

$$\frac{\Phi}{M(b + w)} + \frac{\Xi}{Qn}.$$

Such a choice exists because the current expectation is the average over the possible values of the next a_v or d_f. The final value is at most two. Since both summands are nonnegative,

$$c_u \leqslant \frac{M}{D} + \sqrt{2M(b + w)}, \qquad n_{t,g} \leqslant \frac{n}{r} + \sqrt{2Qn}.$$

The two inequalities in (8.13) make the square-root terms at most M/D and n/r, respectively.

The following enumeration computes the conditional expectations in $\mathrm{poly}(n, M, |\mathcal{F}|, D, Q, r, b, w)$ bit operations. Put $K = r^Q$, so $D = K^2$. Each indicator in the expansion of Φ depends on at most the four independent choices $a_v, d_f, a_{v'}, d_{f'}$; repeated indices contribute just one choice. Each indicator in Ξ depends on at most $a_v, a_{v'}$. After any partial fixing, enumerate at most $K^4 = D^2$ assignments to the remaining choices in one indicator. Its conditional probability has denominator dividing K^4. Summing the $M^2 + Qn^2$ indicators computes both expectations. Including the normalizations $M(b + w)$ and Qn, all rational values in these calculations have numerators and denominators with

$$O(\log(2D) + \log(2M) + \log(2n) + \log(2Q) + \log(2 + b + w))$$

bits. There are at most $n + |\mathcal{F}|$ choices to fix, each with K candidate values. Since $K^4 = D^2$, the entire construction takes $\mathrm{poly}(n, M, |\mathcal{F}|, D, Q, r, b, w)$ bit operations. □

8.5. **Placing copies of the variables in the auxiliary arrays.** We now transfer the auxiliary rows $\vec{P}_u(T)$ and matrix $C^{\mathcal{U}}$ to the rows $\vec{P}_{f,v}(T)$ and matrices $C_{t,g}$ on the original variables required by Definition 7.1. Assume (8.13) and apply Lemma 8.6. With $h = \max_u c_u$, index h copies of the auxiliary coordinates by $[h] \times \mathcal{U}$. For each $(f, v) \in \mathcal{J}$, we assign $c(f, v) \in [h]$ so that all pairs $(c(f, v), u_{f,v})$ are distinct. This is possible because no label occurs more than h times, and

$$hD \leqslant 2M \leqslant 2bn.$$

Define the linear map $\iota : \mathbb{Q}^n \longrightarrow \mathbb{Q}^{[h]\times\mathcal{U}}$ by

$$(\iota x)_{c,u} = \begin{cases} x_v, & (c, u) = (c(f, v), u_{f,v}) \text{ for some } (f, v) \in \mathcal{J}, \\ 0, & \text{otherwise.} \end{cases}$$

The occupied positions are distinct, so this definition is unambiguous. Different positions can contain the same x_v, and unused positions are fixed at zero. For $c \in [h]$, define the projection

$$\pi_c : \mathbb{Q}^{[h]\times\mathcal{U}} \longrightarrow \mathbb{Q}^{\mathcal{U}}, \qquad (\pi_c\eta)_u = \eta_{c,u}.$$

Now put

$$\begin{aligned} \vec{P}_{f,v}(T) &= \vec{P}_{u_{f,v}}(T)\pi_{c(f,v)}\iota, \\ S_{t,g} = \{v \in [n] : a_v(t) = g\}, &\qquad \sigma_f(t) = d_f(t). \end{aligned} \tag{8.14}$$

The row $\vec{P}_{f,v}(T)$ acts on the actual variables $x \in \mathbb{Q}^n$. At $T = 0$, it extracts the value in position $(c(f, v), u_{f,v})$, which is x_v. Thus $\vec{P}_{f,v}(0) = \delta_v$, and its degree is at most $R - 1$. Lemma 8.6 gives $|S_{t,g}| \leqslant 2n/r$.

Define the measurement matrices by

$$\widetilde{C} = \operatorname{diag}(\underbrace{C^{\mathcal{U}}, \dots, C^{\mathcal{U}}}_{h \text{ copies}}), \qquad \widehat{C} = \widetilde{C}\iota, \qquad C_{t,g} = \widehat{C}|_{S_{t,g}}. \tag{8.15}$$

The restriction keeps the columns in $S_{t,g}$ and all rows, including zero rows. Thus $\widehat{C}x$ records the auxiliary measurements in each copy after its coordinates have been filled from x. The matrices $C_{t,g}$ are integral. Their entries need not belong to $\{0, 1, -1\}$, because coefficients at several positions containing the same x_v are added. Each row of $\widetilde{C}$ has at most D nonzero entries, all equal to 1 or -1. Multiplication by ι adds entries in a common column and discards unused positions, so it cannot increase the sum of their absolute values. Consequently every row of $C_{t,g}$ has absolute row sum at most D, and its value on a Boolean assignment lies in $[-D, D] \cap \mathbb{Z}$.

If (8.8) holds, each $C_{t,g}$ has $c_{t,g} = hc_{\mathcal{U}}$ rows. Hence

$$\sum_{g=1}^{r} c_{t,g} = rhc_{\mathcal{U}} \leqslant rh\epsilon D \leqslant 2r\epsilon M \leqslant 2r\epsilon bn. \tag{8.16}$$

Setting $\epsilon = \tau/(rb)$ makes this at most $2\tau n$.

To verify separation, fix $t \in [Q]$, $g \in [r]$ and $(f, v) \in \mathcal{J}$ with $g \neq \sigma_f(t)$. Suppose $\xi \in \mathbb{Q}^{S_{t,g}}$ satisfies $C_{t,g}\xi = \vec{0}$, and extend it by zero to $\widetilde{\xi} \in \mathbb{Q}^n$. Equation (8.15) gives $\widehat{C}\iota\widetilde{\xi} = \vec{0}$. Thus, for every $c \in [h]$,

$$C^{\mathcal{U}}\pi_c\iota\widetilde{\xi} = \vec{0}.$$

Every nonzero coordinate of $\pi_c\iota\widetilde{\xi}$ contains a variable v' with $a_{v'}(t) = g$. Its auxiliary label has first sequence $a_{v'}$, so this vector is supported on $S^{\mathcal{U}}_{t,g}$. Lemma 8.4, applied in copy $c(f, v)$ with $d_{u_{f,v}}(t) = d_f(t) = \sigma_f(t)$, gives

$$\langle \vec{P}_{f,v}(t), \widetilde{\xi} \rangle = \langle \vec{P}_{u_{f,v}}(t), \pi_{c(f,v)}\iota\widetilde{\xi} \rangle = 0.$$

This is the kernel condition in Definition 7.1. All inputs of factor f use the same assigned group $d_f(t)$, and the counting sum still ranges over exactly the n Boolean bits.

8.6. **Coefficient bounds and the completed construction.** The substitution rows $\vec{P}_{f,v}(T) \in \mathbb{Q}[T]^{1\times n}$ in (8.14) have polynomial entries with rational coefficients. We now bound the bits needed to store those coefficients; these bounds will control exact arithmetic in Section 9.2. We first give a common denominator and coefficient bounds for the auxiliary rows $\vec{P}_u(T)$ defined in (8.10). Passing from $\vec{P}_u(T)$ to $\vec{P}_{f,v}(T)$ by (8.14) only adds coefficients in common columns or discards unused columns, so the same denominator and bound on the sum of absolute values apply.

Lemma 8.7. *For the auxiliary rows in (8.10) and their transferred rows in (8.14), put* $d_{\mathrm{den}} = (R-1)!Q!$. *Then* $d_{\mathrm{den}}\vec{P}_u(T) \in \mathbb{Z}[T]^{1\times D}$ *for every* $u \in \mathcal{U}$. *The sum of the absolute values of all coefficients in each row* $d_{\mathrm{den}}\vec{P}_u(T)$ *is at most* $2^{O(Q\log Q)}$. *The same bound holds for* $d_{\mathrm{den}}\vec{P}_{f,v}(T)$. *The matrices* $M_0, C^{\mathcal{U}}$ *and the rows* $\vec{P}_u(T)$ *can be constructed in* $\mathrm{poly}(D,Q,r)$ *bit operations.*

Proof. For $0 \leqslant j < R$, the corresponding summand of $f_{\mathcal{I}}(W_0)$ equals

$$(-1)^{A_0}(-2)^j\binom{W_0 - A_0 I}{j} = \frac{(-1)^{A_0+j}}{j!}\prod_{a=0}^{j-1}\bigl(2W_0 - 2(A_0+a)I\bigr).$$

Each matrix factor on the right is integral because $2W_0 = QI - \sum_s \Pi_s$. Also $j!$ divides $(R-1)!$. Multiplication by the permutation matrix V_0 therefore shows that $(R-1)!M_0$ is integral. The coefficients of $\varphi_{u,y}$ have denominators dividing $\prod_{s\in E_{u,y}} s$, which divides $Q!$. Equation (8.10) proves the asserted common denominator.

For the coefficient estimates, we use the matrix norm $\|A\|_\infty = \max_i \sum_j |A[i,j]|$ and the polynomial norm $\|\sum_j a_j T^j\|_1 = \sum_j |a_j|$. Since $\|W_0\|_\infty \leqslant Q$ and $0 \leqslant A_0 + a \leqslant Q$ for all factors above, each $2W_0 - 2(A_0+a)I$ has norm at most $4Q$. It follows that

$$\|M_0\|_\infty \leqslant R(4Q)^{R-1}, \qquad \|\varphi_{u,y}\|_1 \leqslant \prod_{s\in E_{u,y}} (1+1/s) \leqslant 2^Q.$$

The sum of the absolute values of all coefficients in the row $d_{\mathrm{den}}\vec{P}_u$ is therefore at most

$$d_{\mathrm{den}}\bigl(1 + (1+2^Q)R(4Q)^{R-1}\bigr).$$

Its logarithm is $O(Q\log Q)$ because $R < Q$. For $u = u_{f,v}$, multiplication of $d_{\mathrm{den}}\vec{P}_u(T)$ by $\pi_{c(f,v)}\iota$ adds coefficients in common columns and discards unused columns. By (8.14), the resulting row is $d_{\mathrm{den}}\vec{P}_{f,v}(T)$, so it satisfies the same bound.

Enumerating the eigenbasis in Lemma 8.2, evaluating $f_{\mathcal{I}}(W_0)$, and forming the polynomials $\varphi_{u,y}$ require $\mathrm{poly}(D,Q,r)$ arithmetic operations. Using the common denominators above, the coefficients stored during these matrix and polynomial calculations have numerators and denominators with $O(Q\log Q)$ bits. Thus the construction takes $\mathrm{poly}(D,Q,r)$ bit operations. □

Proof of Lemma 8.1. Lemma 8.2 constructs $C^{\mathcal{U}}$ with $c_{\mathcal{U}} \leqslant \tau D/(rb)$. Lemmas 8.3 and 8.4 give the auxiliary polynomial rows. Lemma 8.6 chooses the variable and factor groups with $hD \leqslant 2M$ and $|S_{t,g}| \leqslant 2n/r$. The maps in (8.14) and (8.15) then give rows with $\vec{P}_{f,v}(0) = \delta_v$, degree at most $R-1 \leqslant R$, and the required kernel implication. Every row of $C_{t,g}$ has absolute row sum at most D, and (8.16) gives $\sum_{g=1}^{r} c_{t,g} \leqslant 2\tau n$ for each $t \in [Q]$. Lemma 8.7 supplies the common denominator and coefficient bounds. Together, the constructions take $\mathrm{poly}(n, M, |\mathcal{F}|, D, Q, r, b, w)$ bit operations, and the resulting arrays occupy $\mathrm{poly}(n, M, |\mathcal{F}|, D, Q, r, b, w)$ bits. □

8.7. **Choosing the parameters.** In the application, both the largest scope w and the maximum number b of factors containing a variable will be bounded by a fixed power of $\log n$. We now choose parameters satisfying Lemma 8.1 in that regime. The exponent of the counting cost depends on the three quantities

$$\frac{R}{Q}\log\frac{2Q}{R}, \qquad \tau\log(2D+1), \qquad \frac{1}{r}.$$

They account respectively for recovering Taylor coefficients, enumerating measurement values, and enumerating assignments in one group. The following choices bound all three at once.

Lemma 8.8. *Fix an integer $\beta \geqslant 1$. Let $n \geqslant 20$, put $\lambda = \lceil \log_2 n \rceil$ and $\nu = \lceil \log_2 \lambda \rceil$, and suppose b, w are positive integers with $b, w \leqslant \lambda^\beta$ and $n \leqslant M \leqslant bn$. For all sufficiently large n, the parameters*

$$r = \left\lfloor \frac{\sqrt{\lambda}}{\nu^2} \right\rfloor, \qquad Q = \left\lfloor \frac{\lambda}{128\nu} \right\rfloor, \qquad R = (\beta+3)\lceil \sqrt{\lambda} \rceil, \tag{8.17}$$

$$\tau = \frac{1}{r\lambda}, \qquad D = r^{2Q},$$

together with

$$\mu = \frac{Q(r-1)}{2r}, \qquad A_0 = \left\lfloor \mu - \frac{R-1}{2} \right\rfloor, \qquad \mathcal{I} = \{A_0, \dots, A_0 + R - 1\},$$

satisfy all numerical hypotheses of Lemma 8.1. They also satisfy

$$R \geqslant 32, \qquad 6R \leqslant Q, \qquad D \leqslant 2^{\lambda/128},$$

and

$$\frac{R}{Q} \log \frac{2Q}{R} + \tau \log(2D+1) + \frac{1}{r} = O_\beta\left(\frac{\nu^2}{\sqrt{\lambda}} \right). \tag{8.18}$$

Given the numerical values n, M, b, w, the parameters and numerical hypotheses can be computed and checked exactly in $\mathrm{poly}_\beta(\log n)$ bit operations. For fixed β, only finitely many n fail these checks.

Proof. For sufficiently large n, the parameters have orders

$$r = \Theta(\sqrt{\lambda}/\nu^2), \qquad Q = \Theta(\lambda/\nu), \qquad R = \Theta_\beta(\sqrt{\lambda}).$$

In particular, eventually $r \geqslant 2$, $R \geqslant 32$, and $6R \leqslant Q$. Since $\log_2 r \leqslant \nu/2$,

$$\log_2 D = 2Q \log_2 r \leqslant 2 \frac{\lambda}{128\nu} \frac{\nu}{2} = \frac{\lambda}{128}.$$

The two balance inequalities follow for sufficiently large n:

$$2(b+w)D^2 \leqslant 4\lambda^\beta 2^{\lambda/64} \leqslant n \leqslant M, \qquad 2Qr^2 \leqslant n.$$

For the second, Q, r are bounded by fixed powers of λ, whereas $n > 2^{\lambda-1}$. The first follows from the same comparison.

For the specified interval, $Q/4 \leqslant \mu \leqslant Q/2$. Since $R \leqslant Q/6$, the interval lies in $\{0, \dots, Q\}$. Its center differs from μ by less than one, and $R \geqslant 32$ ensures that it contains every integer within $R/4$ of μ. For $W \sim \mathrm{Bin}(Q, (r-1)/(2r))$, Hoeffding's inequality gives

$$\Pr[W \notin \mathcal{I}] \leqslant 2e^{-R^2/(8Q)} \leqslant 2e^{-16(\beta+3)^2\nu} \leqslant 2^{-(\beta+2)\nu} \leqslant \lambda^{-(\beta+2)} \leqslant \frac{1}{br^2\lambda}.$$

The last inequality uses $b \leqslant \lambda^\beta$ and $r^2 \leqslant \lambda$. This proves (8.8) with $\epsilon = \tau/(rb) = 1/(br^2\lambda)$. Also $0 < \tau \leqslant 1/2$ once $r \geqslant 2$.

Finally,

$$\frac{R}{Q} = O_\beta(\nu/\sqrt{\lambda}), \qquad \log(2Q/R) = O(\nu), \qquad \tau \log(2D+1) = O(1/r).$$

Together with $1/r = O(\nu^2/\sqrt{\lambda})$, these prove (8.18).

We finish by showing that the parameters in (8.17) can be constructed exactly within the stated bit bound. The algorithm finds $\lambda = \lceil \log_2 n \rceil$ as the least integer with $n \leqslant 2^\lambda$, and then finds $\nu = \lceil \log_2 \lambda \rceil$ in the same way. It obtains r as the largest integer satisfying $r^2\nu^4 \leqslant \lambda$, computes Q by integer division, and finds $\lceil \sqrt{\lambda} \rceil$ by an integer square-root search to obtain R. If $r < 2$ or $Q \leqslant R$, the algorithm reports failure at this stage. Otherwise it constructs τ, ϵ, D and the endpoints of $\mathcal{I}$

using integer or rational arithmetic. It then checks $\mathcal{I} \subseteq \{0, \dots, Q\}$, the two balance inequalities, $R \geqslant 32$, and $6R \leqslant Q$. The tail probability in (8.8) has denominator $(2r)^Q$, so its required bound can also be checked by integer arithmetic. These computations take $\mathrm{poly}_\beta(\log n)$ bit operations. The preceding estimates show that all checks pass for sufficiently large n. □

9. Counting common zeros by polynomial interpolation over $\mathbb{Q}$

We now prove Theorem 6.2. The input is a system $\mathcal{P}$ of $m_{\mathrm{test}} \leqslant cn$ polynomial equations $P_i(x) = 0$ over $\mathbb{F}_2$ in n Boolean variables. Each P_i is a list of at most $(\log n)^c$ monomials of degree at most k, for fixed k, c. Our target is $N(\mathcal{P})$ from (2.1). Use the integer lifts L_i, scopes I_i, linear coefficient polynomials e_j, and coefficient algebra $\mathcal{E}_\ell$ from Sections 4.1–4.2, with $\ell = n + 1$. Lemma 4.2 bounds the product degree by $k(m_{\mathrm{test}} + 2n) = O_{k,c}(n)$. On each branch fixing the few variables of high incidence, Lemmas 8.8 and 8.1 supply the data used by Proposition 9.1. The full deduction, including the cost of these branches, is in Section 9.3.

9.1. Counting with supplied replacement forms.

For equation i supply Section 7 with the factor

$$\mathcal{B}_i((X_v)_{v\in I_i}, V) = \sum_{j=0}^{\ell-1} e_j(L_i(\vec{X}))h(L_i(\vec{X}))^j V^j \quad \text{in } \mathcal{E}_\ell[(X_v)_{v\in I_i}]. \tag{9.1}$$

Within a factor, every occurrence of X_v uses the same replacement form; different factors may use different forms for X_v. Even if each factor receives its own independent inputs, L_i has degree at most k in those inputs. Lemma 4.2 bounds the input degree of the product *in* $\mathcal{E}_\ell$ by $d = k(m_{\mathrm{test}} + 2(\ell - 1))$. Thus the product satisfies precisely the degree hypothesis (7.2) of the common routine. Adding factors equal to one with scope $\{v\}$ ensures that all declared variables occur in the construction.

Let $H(T) \in \mathcal{E}_\ell[T]$ be (7.4) for these factors. Its V-coefficients at $T = 0$ are integers because all replacements restore their Boolean inputs at zero. By the same lemma, summing their values gives

$$\deg_T H \leqslant d_H := kR(m_{\mathrm{test}} + 2(\ell - 1)), \qquad \mathcal{S}_\ell(H(0)) \equiv N(\mathcal{P}) \pmod{2^\ell}. \tag{9.2}$$

Consequently the least nonnegative residue of $\mathcal{S}_\ell(H(0))$ is the exact count. Interpolation takes place over the rationals; the congruence is needed only on Boolean inputs at $T = 0$.

9.2. Cost of evaluating the factors.

Proposition 9.1. *Let $\mathcal{P}$ be a list of m_{test} polynomial equations over $\mathbb{F}_2$ on n declared Boolean variables, each given by a monomial list of degree at most k. Suppose the factors (9.1), including the neutral factors, are supplied with the data of Definition 7.1. Let $S \geqslant 1$ bound the total length of the monomial lists and factor scopes, and let ℓ be any positive integer with $2^\ell > 2^n$. With d_H from (9.2) and q, J from (7.3), $N(\mathcal{P})$ can be computed deterministically in*

$$\mathrm{poly}(n, m_{\mathrm{test}}, \ell, D, Q, R, J, r, k, S) \left(\sum_{s=0}^{\min(q,n)} 2^s \binom{n}{s} \right) (2D+1)^{2\tau n} 2^{2n/r}$$

bit operations and $\mathrm{poly}(n, m_{\mathrm{test}}, \ell, D, Q, R, J, r, k, S)$ bits of working space.

Proof. Apply Proposition 7.2 with coefficients in $\mathcal{E}_\ell$. At an evaluation point t, the routine uses $\mathcal{E}_\ell[z]/(z^J)$, so a stored element has just $J\ell$ rational coefficients. To evaluate factor i at a fixed Boolean substitution $\vec{X} = \mathbf{1}_U$ in (7.7), first evaluate each listed monomial by multiplying its at most k substituted inputs, then add the results to get L_i. Use this value in (9.1), successively multiplying by $h(L_i)$ to obtain its powers. This takes polynomially many rational operations in the displayed parameters and never enumerates assignments to the entire scope I_i. The polynomials e_j are computed once by Lemma 4.1.

For bit complexity, the rows have common denominator $d_{\mathrm{den}} = (R-1)!Q!$. Each substituted monomial has denominator dividing $\Delta = d_{\mathrm{den}}^{k}$, every lifted factor has denominator dividing $\Gamma = \Delta^{2\ell-1}$, and their product has denominator dividing $\Omega = \Gamma^{m_{\mathrm{test}}}$. Thus exponentially many summands can be accumulated over one denominator. Appendix B bounds the bit lengths in these sums by

$$B_{\mathrm{sum}} = \mathrm{poly}(n, m_{\mathrm{test}}, \ell, k, Q, \log(D+2), \log(S+2)). \tag{9.3}$$

For each $0 \leqslant s < \ell$ we interpolate the rational polynomial $[V^s]H(T)$ from its QJ Taylor coefficients. The appendix verifies polynomial bit cost for this interpolation and the final integer remainder in (9.2). The $O(QJ\ell)$ stored coefficients, the supplied rows, partitions, assignments, and measurement matrices, and the sequential enumerations in Proposition 7.2 give the stated space bound. □

9.3. **Constructing the forms and completing the proof.** Each sparse polynomial reads few variables, but one variable might occur in many equations. We first fix the values of the few variables with the largest numbers of occurrences. On each resulting branch we can construct the rows, partitions, assignments, and measurement matrices required by Proposition 9.1.

Proof of Theorem 6.2. Let $\mathcal{P}$ be the supplied polynomial system. An empty list of equations gives 2^n common zeros; a constant equation $1 = 0$ gives none. Discard equations $0 = 0$. Include a neutral factor equal to one with scope $\{v\}$ for each declared variable v, and write

$$M = \sum_{f \in \mathcal{F}} |I_f|$$

for the total number of factor–variable incidences. The input bounds imply $M = n(\log n)^{O_{k,c}(1)}$, and every scope has $(\log n)^{O_{k,c}(1)}$ variables. A particular variable, however, may occur in many of these scopes.

Put $\lambda = \lceil \log_2 n \rceil$ and

$$b_0 = \left\lceil \frac{M}{n} \right\rceil \lceil \sqrt{\lambda} \rceil.$$

Let $S \subseteq [n]$ be the variables belonging to more than b_0 scopes. For each $b \in \{0,1\}^S$, we fix $x|_S = b$ and count the completions on $[n] \setminus S$; we call this restricted problem a branch. Since $|S| \leqslant M/b_0 \leqslant n/\sqrt{\lambda}$, there are at most $2^{n/\sqrt{\lambda}}$ branches. Each branch has the same number n' of free variables, with

$$n' \geqslant n - \frac{n}{\sqrt{\lambda}} \geqslant \frac{n}{2} \qquad (n \geqslant 20).$$

Substitute the fixed bits in every P_i over $\mathbb{F}_2$ and combine duplicate monomials modulo two before forming its integer lift. Neither degree nor the number of monomials increases. A constant equation can again be handled immediately. The branches partition the Boolean cube, so their common-zero counts add to $N(\mathcal{P})$.

To apply Section 8, remove the fixed variables from the scopes and relabel the free variables by $[n']$. Write $I'_f \subseteq [n']$ for the restricted scopes and $M' = \sum_f |I'_f|$ for their total incidence count. Let b be the maximum factor incidence of a free variable and let $w = \max_f |I'_f|$. Then $b \leqslant b_0$, and both b and w are bounded by a fixed power of $\log n'$. The remaining neutral factors ensure $M' \geqslant n'$. Set

$$\lambda' = \lceil \log_2 n' \rceil, \qquad \nu' = \lceil \log_2 \lambda' \rceil,$$

and choose the smallest integer $\beta \geqslant 1$ with $b, w \leqslant (\lambda')^\beta$. For fixed k, c, the possible values of β are bounded by a constant for all sufficiently large n. We apply Lemma 8.8 to the restricted scopes (I'_f) on $[n']$, with incidence count M', using

$$r = \left\lfloor \frac{\sqrt{\lambda'}}{(\nu')^2} \right\rfloor, \qquad Q = \left\lfloor \frac{\lambda'}{128\nu'} \right\rfloor, \qquad R = (\beta+3)\lceil \sqrt{\lambda'} \rceil.$$

If $r < 2$ or $Q \leqslant R$, use the exhaustive-search fallback below before forming τ, ϵ, or the parity interpolation interval $\mathcal{I}$. Otherwise set

$$\tau = \frac{1}{r\lambda'}, \qquad \epsilon = \frac{1}{br^2\lambda'}, \qquad D = r^{2Q},$$

and carry out the remaining feasibility checks from that lemma. For all sufficiently large n, Lemma 8.1 supplies the required rows and measurement matrices, with

$$r = \Theta\left(\frac{\sqrt{\log n'}}{(\log\log n')^2}\right), \qquad Q = \Theta\left(\frac{\log n'}{\log\log n'}\right), \qquad R = O_{k,c}(\sqrt{\log n'}), \qquad D \leqslant (2n')^{1/128}.$$

On every branch we may use the original precision $\ell = n + 1$, since even the unrestricted count is smaller than 2^ℓ. As $n' \geqslant n/2$, both ℓ and m_{test} are $O_c(n')$. Thus $d_H = O_{k,c}(n'R)$, $J = O_{k,c}(n'R/Q) + 1$, and (7.16), in dimension n', has exponent

$$O_{k,c}\left(\frac{n'(\log\log n')^2}{\sqrt{\log n'}}\right) = O_{k,c}\left(\frac{n(\log\log n)^2}{\sqrt{\log n}}\right).$$

All substitution data, coefficient arrays, and interpolation systems have total representation length $n^{O_{k,c}(1)}$ bits. Multiplying the per-branch running time by at most $2^{n/\sqrt{\lambda}}$ leaves the displayed exponential bound unchanged, so summing the branch answers gives the required running time.

To implement the construction for every input size, the algorithm checks its numerical hypotheses using integer arithmetic. In addition to the preliminary guards above, it checks $2(b + w)D^2 \leqslant M'$ and $2Qr^2 \leqslant n'$ from Lemma 8.6, the endpoints of $\mathcal{I}$, $R \geqslant 32$, $6R \leqslant Q$, and the binomial tail bound in (8.8), which can be tested exactly. If any check fails, the algorithm abandons the branched computation and checks the original equations directly on all 2^n assignments. For fixed k, c, the bounds just proved and Lemma 8.8 show that a failure can occur only for finitely many n. This fallback is therefore absorbed in the theorem's constants.

To combine the branch counts within the stated space bound, we process the branches one at a time. We initialize an integer accumulator $Z = 0$ and add each branch's answer to it. The construction and Proposition 9.1 use $n^{O_{k,c}(1)}$ bits of space on each branch. Disjointness gives $0 \leqslant Z \leqslant 2^n$ throughout, and the final value is $N(\mathcal{P})$, as claimed. □

10. Counting arbitrary formulas by interpolation

The rational counting routine gives the following bound on arbitrary formulas. We first count one sparse formula and then sum over the leaves of Lemma 2.3.

Proposition 10.1. *For each fixed $k \geqslant 2$, the interpolation algorithm counts the satisfying assignments of a k-CNF on $n \geqslant 20$ declared variables with m clauses in $(n + m)^{O_k(1)}2^{O_k(\rho(n))}$ bit operations and $(n + m)^{O_k(1)}$ bits of space.*

10.1. **Counting one sparse formula.** Suppose $n \geqslant 20$ and $1 \leqslant m \leqslant Bn$, where B is bounded by a fixed power of $\log n$ depending only on k. Construct the initial matrix by Lemma 2.4. This uses $\exp((\log n)^{O_k(1)})$ bit operations and polynomial space. Both P and D_0 are $(\log n)^{O_k(1)}$, so every candidate in Lemma 2.5 is an instance of Theorem 6.2 with a single fixed $c = c(k)$. This applies to discarded candidates as well as the final system: the counter requires only the number, degree and sparsity of the equations. Expanding each row and combining monomials costs polynomial time and space for fixed k.

At most $3n(2m)^P + 1 = \exp((\log n)^{O_k(1)})$ calls suffice, including one final count. Each call takes $2^{O_k(n(\log\log n)^2/\sqrt{\log n})}$ bit operations. Since any fixed power of $\log n$ is $o(n(\log\log n)^2/\sqrt{\log n})$, the total time, including construction and candidate comparisons, is

$$(n + m)^{O_k(1)}2^{O_k(n(\log\log n)^2/\sqrt{\log n})}. \tag{10.1}$$

The final count is $\#F$ by the greedy lemma. A formula with no clauses or containing an empty clause is handled directly. Candidates are examined sequentially; only the current rows, a candidate, the best row and its count need be kept in addition to the counter's polynomial working space.

10.2. **Returning to a general formula.** We use disjoint sparsification to partition the satisfying assignments among sparse formulas on the same n declared variables. The preceding algorithm counts each of these formulas. The parameter σ controls a tradeoff: making it smaller reduces the number of formulas but increases their occurrence bound B. Our solver permits B to grow as a fixed power of $\log n$, so we can let σ tend to zero while keeping both the number of formulas and the cost per formula within the desired time bound.

Lemma 2.3, proved in Appendix A, supplies the disjoint leaves and their occurrence bound.

Proof of Proposition 10.1. We first normalize the input in $(n+m)^{O_k(1)}$ time by deleting repeated literals, tautological clauses, duplicate clauses, and clauses containing another clause. We retain all n declared variables. An empty clause gives count zero, and a formula with no clauses gives count 2^n. We handle $n < 20$ by exhaustive search. For $n \geqslant 20$, we next select the sparsification parameter. Define

$$\lambda_n = \lceil \log_2 n \rceil, \qquad \nu_n = \lceil \log_2 \lambda_n \rceil, \qquad s = \left\lfloor \frac{\sqrt{\lambda_n}}{\nu_n^2} \right\rfloor.$$

We compute s by integer comparisons, taking the largest nonnegative integer satisfying $s^2 \nu_n^4 \leqslant \lambda_n$. If $s < 2$, we use exhaustive search; $\sqrt{\lambda_n}/\nu_n^2$ tends to infinity, so there are only finitely many such sizes. Otherwise we apply Lemma 2.3 with $\sigma = 1/s$. The tree has at most $2^{n/s}\,\mathrm{poly}_k(n,s)$ nodes, and its leaves have occurrence bound

$$B = O_k\big((s \log s)^{k-1}\big) = (\log n)^{O_k(1)}.$$

Each leaf uses the original n variables and has at most Bn nonempty clauses. We count each leaf by the sparse-formula algorithm of Section 10.1, whose running time is given by (10.1). Leaves with no clauses or containing an empty clause are handled directly. The clause count in that application is the number of clauses in the leaf; m in the final bound remains the input clause count.

For $s \geqslant 2$, the floor in its definition gives

$$\frac{n}{s} = O\left(\frac{n\nu_n^2}{\sqrt{\lambda_n}}\right) = O(\rho(n)).$$

Thus both the number of leaves and the time to count each leaf have exponent $O_k(\rho(n))$. Their polynomial factors, together with normalization and the work at the tree nodes, are bounded by $(n+m)^{O_k(1)}$. The total number of bit operations is therefore

$$(n+m)^{O_k(1)} 2^{O_k(\rho(n))}.$$

The finite exhaustive-search cases are absorbed in the constants. All parameters, initial checks, and additional rows are computed by the deterministic procedures above; every counting query uses the common-zero algorithm of Theorem 6.2.

To bound the space used by the full algorithm, we traverse the sparsification tree depth first and count one leaf at a time. The current path has $\mathrm{poly}_k(n,s)$ nodes, and each normalized formula has a $\mathrm{poly}_k(n)$-bit encoding. Since $s \leqslant \sqrt{\lceil \log_2 n \rceil}$, storing this path and counting one leaf use $\mathrm{poly}_k(n,m)$ bits. Disjointness keeps the sum of the processed leaf counts between zero and 2^n, so its accumulator uses at most $n+1$ bits. Thus the total space is $\mathrm{poly}_k(n,m)$ bits. □

Appendix A. Partitioning the satisfying assignments

Fix $k \geqslant 2$ and a rational $0 < \sigma \leqslant 1$. We prove Lemma 2.3 by repeatedly splitting a formula into two cases. Both children use the same n variables, and every satisfying assignment of the parent satisfies exactly one child. We follow the rule for choosing a shared set of literals and the insertion

accounting of [CIP06, Section 6]. The refinement that makes the two cases disjoint is standard in exact counting [DHM+14, Appendix A].

Throughout this appendix, a clause is a set of literals and denotes their disjunction. Its *width* is the number of literals it contains. We delete tautological clauses, repeated clauses, and clauses containing another clause. Thus the remaining clauses form an antichain under inclusion. We keep all n declared variables and do not propagate unit clauses. An empty clause gives count zero, and the empty formula gives count 2^n.

A.1. **The branching rule.** Fix an integer $M \geqslant 2$. We will obtain an occurrence bound $B = O_k(M^{k-1})$ at every leaf, then choose M so that the tree has at most $\mathrm{poly}_k(n, \sigma^{-1})2^{\sigma n}$ nodes. Put

$$\theta_j = M^j \qquad (0 \leqslant j < k).$$

Writing F for the current family of clauses, define

$$\deg_c(H) = \#\{C \in F : |C| = c,\ H \subseteq C\}$$

for each literal set H. We branch whenever a nonempty proper subset H of a width-c clause satisfies

$$\deg_c(H) \geqslant \theta_{c-|H|}. \tag{A.1}$$

The algorithm chooses the smallest possible c, and then the largest possible $|H|$; break any remaining ties deterministically. Thus a nonempty shared literal set H leaving $j = c - |H|$ other literals must occur in at least M^j clauses before we branch on it.

Let $H \cup P_1, \ldots, H \cup P_z$ be all the width-c clauses containing the chosen H, and let G consist of the other clauses. Put $j = c - |H|$. Here P_a is the set of literals left after removing H from the ath selected clause, so each P_a has width j and $z \geqslant \theta_j$. The two children are

$$F_A = G \wedge H, \qquad F_B = G \wedge \bigwedge_{a=1}^{z} P_a \wedge \bigwedge_{\ell \in H} \{\neg \ell\}. \tag{A.2}$$

We restore the antichain condition in each child by removing clauses made redundant by its new clauses. The sets P_a may overlap. The satisfying assignments of the children are disjoint: F_A requires some literal of H to be true, whereas F_B requires every literal of H to be false.

Lemma A.1. *For a branch chosen by (A.1) with the stated smallest-width and largest-$|H|$ priorities, the normalized children in (A.2) partition the satisfying assignments of their parent. Every new clause has width less than c. The A-child inserts one new clause, and the B-child inserts at least θ_j new clauses of width j. All newly inserted clauses survive the immediate normalization. Every old clause removed in either child contains a newly inserted strict subset.*

Proof. Every assignment satisfies exactly one of the conditions *some literal of H is true* and *every literal of H is false*. In the first case, all selected clauses are satisfied, leaving $G \wedge H$. In the second, each selected clause reduces to P_a, and the unit clauses $\{\neg \ell\}$ enforce this case. Thus the two children partition the parent's satisfying assignments.

The old antichain contains no clause included in H or in any P_a: such a clause would be a strict subset of $H \cup P_a$. Thus H and the distinct equal-width clauses P_a are new. No P_a contains a variable of H, since its parent is nontautological. Consequently, none of the unit clauses $\{\neg \ell\}$, $\ell \in H$, is a subset of any P_a. Any unit clause $\{\neg \ell\}$ already present is retained and is not counted as a new insertion. The remaining new unit clauses and the clauses P_a all survive. Each selected old clause contains the newly inserted clause H for child A, or its newly inserted subset P_a for child B. Every other removed clause contains a newly inserted strict subset by the normalization rule. □

A.2. **Counting insertions.** To bound the tree, we count every clause introduced along one path, including clauses later deleted. Clauses present in the input are excluded from this count. By Lemma A.1, each step inserts a clause, so bounding the total number of insertions also bounds the path length.

For this accounting, a clause is marked when a branch introduces it. First we bound how many marked clauses can still be present and contain a given literal set. The smallest-width priority controls those present before a branch; the largest-$|H|$ priority controls those introduced by that branch.

Lemma A.2. *At every node of this branching tree, for $1 \leqslant i < j < k$ and every literal set S of size i, fewer than $2\theta_{j-i}$ marked clauses of width j that are still present contain S.*

Proof. Initially there are no marked clauses. It suffices to consider a branch that inserts clauses of width j. Its selected width c is larger than j. Before insertion, fewer than θ_{j-i} current width-j clauses contain S: otherwise the smallest-width priority would select a width at most j.

Choosing child A inserts only one clause. Suppose child B inserts clauses P_a of width j. If none of the P_a contains S, the number of new width-j clauses containing S is zero. Otherwise S lies in some P_a and is therefore disjoint from H. In this case P_a contains S exactly when the selected clause $H \cup P_a$ contains $H \cup S$. Since $0 < i < j$, $H \cup S$ is larger than H and is a proper subset of a selected width-c clause. The priority for the largest shared set therefore gives

$$\#\{a : S \subseteq P_a\} = \deg_c(H \cup S) < \theta_{c-|H\cup S|} = \theta_{j-i}.$$

The other new clauses $\{\neg\ell\}$ have width one. Thus the old and new clauses of width j together satisfy the asserted bound. Deletion cannot increase it. □

Lemma A.3. *Along any path or path prefix of this tree, let T_j count the width-j clauses inserted by branches, including those later deleted and excluding the input clauses. Then*

$$T_j \leqslant 4(3M)^{j-1}n \qquad (1 \leqslant j < k).$$

Proof. The case $n = 0$ is immediate, so assume $n \geqslant 1$. For the current path prefix, let L_j be the number of marked width-j clauses still present, and let D_j be the number of width-j insertions whose clauses have since been deleted. Then $T_j = L_j + D_j$. Units are never deleted, so $T_1 \leqslant 2n \leqslant 4n$. For $j \geqslant 2$, sum Lemma A.2 over the $2n$ single literals. This gives $L_j \leqslant 4n\theta_{j-1}$.

Whenever a marked clause of width j is deleted, assign that deletion to one newly introduced strict subset contained in it; Lemma A.1 guarantees such a subset. If the new clause has width $i < j$, Lemma A.2 shows that it was contained in fewer than $2\theta_{j-i}$ marked clauses of width j immediately before the branch. Thus one insertion of a width-i clause receives at most that many deletions. Summing over the widths i of the new clauses gives $D_j \leqslant 2\sum_{i=1}^{j-1} \theta_{j-i}T_i$. Therefore

$$T_j = L_j + D_j \leqslant 4n\theta_{j-1} + 2\sum_{i=1}^{j-1} \theta_{j-i}T_i. \tag{A.3}$$

The first term counts clauses still present; the sum counts those that were deleted. In particular, the extra unit clauses making the B-child disjoint may delete marked wider clauses; those deletions are charged to the $i = 1$ term, and the units themselves are included in $T_1 \leqslant 2n$. Dividing by nM^{j-1} and applying induction gives

$$\frac{T_j}{nM^{j-1}} \leqslant 4 + 2\sum_{i=1}^{j-1} 4 \cdot 3^{i-1} = 4 \cdot 3^{j-1},$$

as required. □

A.3. **The size of the tree.** We now convert Lemma A.3 into a bound on the number of tree nodes. It controls both the length of a path and the number of times that path chooses the B-child in (A.2). Write these bounds as Cn and $\Gamma n/M$, respectively, where

$$C = 4\sum_{j=0}^{k-2}(3M)^j, \qquad \Gamma = 4\sum_{j=0}^{k-2}3^j = 2(3^{k-1}-1).$$

Lemma A.3 gives depth at most Cn. A choice of child B with $|P_a| = j$ inserts at least M^j clauses of width j, so a path contains at most $4 \cdot 3^{j-1}n/M$ such choices. In total, a path chooses B at most $\Gamma n/M$ times.

A path with few choices of B has a short description. Because the branching rule is deterministic, a node is determined by its depth and the positions at which its path chooses B; every other position chooses A. For $M \geqslant \Gamma$, this bounds the number of nodes by

$$(Cn+1)\sum_{a\leqslant\lfloor\Gamma n/M\rfloor}\binom{Cn}{a} \leqslant (Cn+1)^2 2^{(\Gamma n/M)\log_2(4CM)}.$$

Here $\Gamma n/M \leqslant n \leqslant Cn/4$. For a positive integer a, use $\binom{Cn}{a} \leqslant (4Cn/a)^a$; this upper bound increases for $a \leqslant Cn/4$. There are at most $Cn+1$ summands. The case $\Gamma n/M < 1$ is immediate.

We choose M to make the exponent in this bound at most σn. Test powers of two $M \geqslant \Gamma$ in increasing order. For each choice, compute C and $h = \lceil\log_2(4CM)\rceil$, and stop when

$$\Gamma h \leqslant \sigma M.$$

All comparisons are exact. Since $C = O_k(M^{k-2})$ and Γ depends only on k, the first successful choice satisfies

$$M = O_k\big(\sigma^{-1}\log(2/\sigma)\big).$$

For an explicit comparison, put $t = \log_2(2/\sigma) \geqslant 1$ and take a power of two between $K\sigma^{-1}t$ and $2K\sigma^{-1}t$. Since $\log_2 t \leqslant t$ and $\log_2 C = O_k(1+\log_2 M)$, this choice has $h = O_k(\log_2 K + t)$. A sufficiently large constant $K = K(k)$ therefore gives $\Gamma h \leqslant Kt \leqslant \sigma M$ and $M \geqslant \Gamma$. The first successful power of two is no larger. The tree therefore has at most $(Cn+1)^2\,2^{\sigma n}$ nodes.

At a leaf, (A.1) fails for every nonempty proper subset of a clause. A fixed literal belongs to fewer than θ_{c-1} width-c clauses when $c \geqslant 2$, and to at most one unit clause. Hence each variable occurs at most

$$B = 2\sum_{j=0}^{k-1}M^j \leqslant 4M^{k-1} = O_k\Big(\big[\sigma^{-1}\log(2/\sigma)\big]^{k-1}\Big).$$

Thus every leaf has the required bound on occurrences per variable. The repeated two-case partition also shows that each original satisfying assignment occurs at exactly one leaf, so summing the leaf counts gives the original count.

To implement the branching rule at a node, we compute $\deg_c(H)$ for its current formula. For each width-c clause, we enumerate its nonempty proper subsets H and increment the counter indexed by (c, H). There are $O_k(n^k)$ possible nonempty normalized clauses, each with at most 2^k subsets. The clause counters use $O_k(\log(n+2))$ bits, and the thresholds use $O_k(\log M)$ bits. Computing the counts, choosing a branch, and normalizing its children therefore use $\mathrm{poly}_k(n, \log M)$ bit operations and bits of working space per node. The bound on M gives $\log M = O_k(\log(2/\sigma))$.

With L_σ as in Lemma 2.3, the parameter search examines $O_k(L_\sigma)$ powers of two. Its integer operands have $O_k(L_\sigma)$ bits, so constructing the numerical parameters uses $\mathrm{poly}_k(L_\sigma)$ bit operations and bits of working space.

Appendix B. Exact arithmetic and space

Section B.1 bounds the integer coefficients of the linear coefficient polynomials of Lemma 4.1. It uses only Section 4.1 and the coefficient norm defined below, and it is the only part of this appendix used in the proof of Theorem 1.1. The rest of the appendix concerns the interpolation algorithm. Section 8 bounds the rational coefficients of the substitution rows $\vec{P}_{f,v}(T)$. We complete the bit-cost and space estimates in Proposition 9.1 by bounding the lifted factors, the sums used to compute the Taylor polynomials of H, and the final interpolation. The essential point is that each sum uses a fixed denominator. Its numerator grows with the number of summands, so its bit length grows only with the logarithm of that number. For any polynomial $G = \sum_\alpha c_\alpha \vec{X}^\alpha$ in finitely many indeterminates, write $\|G\|_1 = \sum_\alpha |c_\alpha|$ for its coefficient norm.

B.1. **Integer coefficients of the lifted indicator.** We first bound the coefficients of the linear coefficient polynomials supplied by Lemma 4.1. They occur in the test tensors of Section 4.3 and in every equation factor $\mathcal{B}_i$ in (9.1), so their sizes enter the bit-cost bounds for evaluating those tensors and factors. For an integer $\ell \geqslant 1$, recall $h(Y) = Y(Y-1)$ and the polynomials from Lemma 4.1:

$$p_\ell(Y) = (1-Y)^\ell \sum_{j=0}^{\ell-1} \binom{\ell+j-1}{j} Y^j = \sum_{j=0}^{\ell-1} e_j(Y) h(Y)^j, \qquad \deg e_j \leqslant 1.$$

That lemma proves the parity congruence and constructs the e_j by successive monic division by h. Here we bound the integers occurring in this construction.

Lemma B.1. *Write $e_j(Y) = a_j + b_j Y$ for $0 \leqslant j < \ell$. Then $a_j, b_j \in \mathbb{Z}$ and*

$$|a_j|, |b_j| \leqslant 2^{C\ell^2} \qquad (0 \leqslant j < \ell)$$

for an absolute constant C. Successive division of p_ℓ by h constructs these polynomials using $\mathrm{poly}(\ell)$ *bit operations and bits of working space.*

Proof. The coefficient-norm inequality $\|fg\|_1 \leqslant \|f\|_1 \|g\|_1$, together with the binomial identity

$$\sum_{j=0}^{\ell-1} \binom{\ell+j-1}{j} = \binom{2\ell-1}{\ell-1}$$

gives

$$\|p_\ell\|_1 \leqslant 2^\ell \binom{2\ell-1}{\ell-1} \leqslant 2^{3\ell}.$$

Monic division by $h = Y^2 - Y$ preserves integer coefficients. Each leading-term elimination increases the sum of the coefficient norms of the current quotient and remainder by at most a factor of three. There are $O(\ell^2)$ eliminations in all the successive divisions. Thus every intermediate coefficient has absolute value at most $2^{O(\ell^2)}$, which proves the stated bound for a_j, b_j. Its signed binary encoding consequently uses $O(\ell^2)$ bits. Each coefficient vector has $O(\ell)$ entries. The successive divisions use $O(\ell^2)$ integer arithmetic operations on integers of $O(\ell^2)$ bits, giving the claimed $\mathrm{poly}(\ell)$ bit-operation and space bounds. □

B.2. **Bit complexity of the sums over Boolean assignments.** We now bound every intermediate value in the computation of $\mathcal{H}_t(z) = H(t+z) \bmod z^J$ from (7.13). Starting with the substitution rows $\vec{P}_{f,v}(T)$, we bound the substituted factors, their sums over each group of Boolean variables, and the accumulator T_t for the Taylor data in Proposition 9.1. Fix a polynomial system $\mathcal{P}$ in n Boolean variables, with m_{test} equations of degree at most k, precision ℓ such that $2^\ell > 2^n$, and replacement rows, partitions, factor assignments, and measurement matrices for its equation factors. Suppose

the rows $\vec{P}_{f,v}$, partitions $S_{t,g}$, and matrices $C_{t,g}$ satisfy Definition 7.1, with parameters Q, R, r, D, τ. Recall the degree bound for H and the number of Taylor coefficients retained at each node:

$$d_H = kR\big(m_{\text{test}} + 2(\ell - 1)\big), \qquad J = \left\lfloor \frac{d_H}{Q} \right\rfloor + 1.$$

Let $S \geqslant 1$ bound the total length of the monomial lists and factor scopes, as in Proposition 9.1. All polynomials in the Taylor variable z and the shared truncation marker V are stored in their unique representative modulo (z^J, V^ℓ). Removing terms outside these degree ranges cannot increase the coefficient norm.

The substitution rows have common denominator

$$d_{\text{den}} = (R-1)!Q!.$$

We use an integer $B_P \geqslant 1$ such that every integer coefficient of every row $d_{\text{den}}\vec{P}_{f,v}(T)$ has absolute value at most 2^{B_P}. Definition 7.1 gives $B_P = O(Q \log Q + \log D)$. At a fixed node $t \in [Q]$ and for a fixed correction set $U \subseteq [n]$, the input to factor f at variable v is

$$\theta_{f,v}(z; x) = \langle \vec{P}_{f,v}(t), x \rangle + \big\langle \vec{P}_{f,v}(t+z) - \vec{P}_{f,v}(t), \mathbf{1}_U \big\rangle, \qquad x \in \{0,1\}^n.$$

Let $L_i(z; x)$ be the sum of the listed monomials of equation i after these substitutions. Each $d_{\text{den}}\theta_{f,v}(z; x)$ is an integer polynomial of coefficient norm at most

$$B_0 = d_{\text{den}} + 3n(R+1)2^{B_P}(Q+1)^R.$$

Indeed, each of the three row evaluations in $\theta_{f,v}$ contributes at most $n(R+1)2^{B_P}(Q+1)^R$ after denominators are cleared. The bound holds also for the representatives in the group sums.

Put $\Delta = d_{\text{den}}^k$. For a monomial of degree $u \leqslant k$, multiplying its substituted value by Δ gives an integer polynomial of norm at most $d_{\text{den}}^{k-u} B_0^u \leqslant B_0^k$, including the constant monomial. As S bounds the total list length,

$$B = \max\{\Delta, SB_0^k\} \quad \text{satisfies} \quad \|\Delta L_i(z; x)\|_1 \leqslant B.$$

Consequently $\log d_{\text{den}} = O(Q \log(Q+2))$, $\log B_0 = O(Q \log(Q+2) + \log(nD+2))$, and $\log B = O(k \log B_0 + \log(S+2))$.

The substituted factor of equation i is

$$\mathcal{B}_i(z, V; x) = \sum_{j=0}^{\ell-1} e_j\big(L_i(z; x)\big) h\big(L_i(z; x)\big)^j V^j \pmod{(z^J, V^\ell)}.$$

Since e_j is integral and has degree at most one, its substitution has denominator dividing Δ; the denominator of $h(L_i)^j$ divides Δ^{2j}. Thus, with

$$\Gamma = \Delta^{2\ell-1},$$

every coefficient of $\Gamma\mathcal{B}_i(z, V; x)$ is an integer. To bound the resulting integer numerators, put

$$E_\ell = \max\{1, \|e_0\|_1, \ldots, \|e_{\ell-1}\|_1\}, \qquad B_1 = \ell E_\ell (B + \Delta + 2)^{2\ell-1}.$$

For $A_i = \Delta L_i$, the two substituted factors have integer numerators $\Delta e_j(L_i)$ and $A_i(A_i - \Delta)$, with norms at most $E_\ell(B+\Delta)$ and $B(B+\Delta)$, respectively. Padding each term to denominator Γ therefore gives

$$\|\Gamma\mathcal{B}_i\|_1 \leqslant B_1, \qquad B_1 \geqslant \Gamma, \qquad \log B_1 = O\big(\ell^2 + \ell \log(B + \Delta + 2) + \log(\ell + 1)\big).$$

We next apply these estimates to the actual group sums. Write $S_g = S_{t,g}$ and $C_g = C_{t,g}$, and let c_g be the number of rows of C_g. Thus $c = \sum_g c_g \leqslant 2\tau n$, and every coordinate of a Boolean measurement $C_g x_g$ lies in $[-D, D] \cap \mathbb{Z}$. Let $m_{\text{test},g}$ be the number of equation factors assigned to group g at t, so $\sum_g m_{\text{test},g} = m_{\text{test}}$. Their product $A_{g,U}(x; z)$ from (7.11) is represented in $\mathbb{Q}[z, V]/(z^J, V^\ell)$. It has

denominator dividing $\Gamma^{m_{\mathrm{test},g}}$ and numerator norm, over that denominator, at most $B_1^{m_{\mathrm{test},g}}$. The neutral constant-one factors do not affect this bound.

For a pair $U \subseteq E \subseteq [n]$ with $|E| < J$ and an attainable measurement tuple $\vec{y} = (y_1, \dots, y_r)$, choose Boolean representatives $\bar{x}_h$ with $C_h \bar{x}_h = y_h$. The group sum in (7.5) is

$$\Sigma_g(\vec{y}) = \sum_{\substack{x_g \in \{0,1\}^{S_g} \\ C_g x_g = y_g \\ (x_g)_v = 1 \ (v \in E \cap S_g)}} A_{g,U}\Big(x_g + \sum_{h \neq g} \bar{x}_h; z\Big).$$

Here vectors on individual groups are extended by zero. The sum has at most $2^{|S_g|}$ terms, all over the same denominator, so

$$\|\Gamma^{m_{\mathrm{test},g}} \Sigma_g(\vec{y})\|_1 \leqslant 2^{|S_g|} B_1^{m_{\mathrm{test},g}}.$$

For $1 \leqslant g \leqslant r$, define the partial product $\Pi_g(\vec{y}) = \prod_{h=1}^{g} \Sigma_h(\vec{y})$, with truncation after each multiplication. Its denominator divides

$$\Gamma^{\sum_{h \leqslant g} m_{\mathrm{test},h}}.$$

Consequently every full contribution $\Pi_r(\vec{y})$ can be placed over the fixed denominator

$$\Omega = \Gamma^{m_{\mathrm{test}}},$$

and it satisfies

$$\|\Omega \Pi_r(\vec{y})\|_1 \leqslant 2^n B_1^{m_{\mathrm{test}}}.$$

Let $T_t(z, V)$ be the accumulator that adds these full contributions with sign $(-1)^{|E|-|U|}$ over all pairs and measurement tuples. By (7.13), its final value is the $\mathcal{E}_\ell$-valued Taylor polynomial $\mathcal{H}_t(z)$. The number M_* of signed updates at one node is bounded by

$$M_* \leqslant \left(\sum_{s=0}^{\min(J-1,n)} 2^s \binom{n}{s} \right) (2D+1)^c \leqslant 3^n (2D+1)^c.$$

Every intermediate value of T_t, not just its final value, therefore satisfies

$$\|\Omega T_t\|_1 \leqslant M_* \, 2^n B_1^{m_{\mathrm{test}}}.$$

Each coefficient of the integer polynomial ΩT_t has

$$O(\log(M_* + 1) + n + m_{\mathrm{test}} \log B_1)$$

bits. This estimate also bounds the integer numerators of all group sums and partial products with the fixed denominators given above. The denominator satisfies

$$\log \Omega = k m_{\mathrm{test}} (2\ell - 1) \log d_{\mathrm{den}} = O(k(m_{\mathrm{test}} \ell + 1) Q \log(Q+2)).$$

Each contribution multiplies at most m_{test} lifted equation factors. Processing further pairs or measurement tuples adds new contributions; it does not multiply them into the previous ones. This is why exponentially many iterations contribute only $\log(M_* + 1)$ to the bit estimate. Each stored polynomial has at most $J\ell \leqslant (d_H + 1)\ell$ coefficients. The enumeration retains its current indices and Boolean representatives, one group sum, one partial product, and the accumulator T_t, without storing a table of all assignments or all measurement tuples. These estimates give (9.3) and polynomial bit cost per rational operation. The $QJ\ell$ Taylor coefficients and the working state fit in $\mathrm{poly}(n, m_{\mathrm{test}}, \ell, D, Q, R, J, r, k, S)$ bits.

B.3. **Interpolation and the integer answer.** For $0 \leqslant s < \ell$, let $H_s(T) = [V^s]H(T) \in \mathbb{Q}[T]$. Section B.2 bounds the computed data $[z^j V^s]\mathcal{H}_t(z) = [z^j]H_s(t+z)$ for $t \in [Q]$, $0 \leqslant j < J$. Fix s. We interpolate this coordinate, using the same rational Hermite matrix for all ℓ coordinates. Put $M_H = QJ > d_H$. In the monomial basis $1, T, \ldots, T^{M_H-1}$, the Hermite interpolation matrix is the integer matrix

$$\mathsf{V}_{(t,j),u} = \begin{cases} \binom{u}{j} t^{u-j}, & u \geqslant j, \\ 0, & u < j, \end{cases} \qquad t \in [Q], \quad 0 \leqslant j < J, \quad 0 \leqslant u < M_H.$$

It is nonsingular by the Hermite interpolation statement in Section 7. Its entries have $O(M_H \log(Q+2))$ bits. Let $\mathbf{y}$ be the vector of computed coefficients $[z^j V^s]\mathcal{H}_t(z)$, ordered by (t,j), and let $\mathbf{h}$ be the monomial coefficient vector of H_s, padded with zeros to length M_H. Thus $\mathsf{V}\mathbf{h} = \mathbf{y}$. The vector $\mathbf{b} = \Omega\mathbf{y}$ is integral; write B_{data} for a bound on its entries' bit lengths. The bounds in Section B.2 give

$$B_{\mathrm{data}} \leqslant \mathrm{poly}(n, m_{\mathrm{test}}, \ell, k, Q, \log(D+2), \log(S+2)).$$

Only the constant coefficient $h_0 = H_s(0)$ is needed. Let $\mathsf{V}^{(0)}$ be obtained from V by replacing its column indexed by $u = 0$ with $\mathbf{b}$, and put

$$N_H = \det \mathsf{V}^{(0)}, \qquad D_H = \det \mathsf{V} \neq 0.$$

Cramer's rule for $\mathsf{V}(\Omega\mathbf{h}) = \mathbf{b}$ gives

$$H_s(0) = h_0 = \frac{N_H}{\Omega D_H}. \tag{B.1}$$

We compute these two determinants by fraction-free Gaussian elimination [Bar68, Sections I–II and IV–V]. In its one-step form, if a_{ij} are the current entries, a_{kk} is a nonzero pivot, and d is the preceding pivot (initially $d = 1$), the update on the remaining submatrix is

$$a'_{ij} = \frac{a_{kk}a_{ij} - a_{ik}a_{kj}}{d} \qquad (i, j > k).$$

Sylvester's determinant identity makes each division exact and identifies the new entries with bordered minors. Row and column interchanges bring any nonzero remaining entry to the pivot position, with their signs recorded. If a nonzero pivot cannot be found, the determinant is zero; otherwise the last pivot is the determinant up to the recorded sign. This is also the pivoting algorithm in [MJK21, Algorithm 2 and proof of Theorem 6]. For an $M_H \times M_H$ integer matrix, this algorithm uses $O(M_H^3)$ integer arithmetic operations; pivoting allows it to handle singular matrices as well. The retained elimination entries are signed minors of the input matrix. Each update forms products and a difference of such entries, followed by an exact integer division. Consequently, bounds on the minors of V and $\mathsf{V}^{(0)}$ also bound the intermediate integers used to compute D_H and N_H. Every minor of $(\mathsf{V} \mid \mathbf{b})$ has absolute value at most

$$2^{O(M_H(B_{\mathrm{data}} + M_H \log(Q+2) + \log(M_H+1)))}.$$

Indeed, its determinant is a sum of at most $M_H!$ products of at most M_H entries. The intermediate integers in fraction-free elimination therefore have

$$O\big(M_H\big(B_{\mathrm{data}} + M_H \log(Q+2) + \log(M_H+1)\big)\big)$$

bits in signed binary notation. Computing N_H, D_H thus uses $\mathrm{poly}(M_H, B_{\mathrm{data}})$ bit operations and bits of working space.

At $T = 0$ every $H_s(0)$ is an integer by construction, so the division in (B.1) is exact for each s. After computing these ℓ integers, put $u = \sum_{s=0}^{\ell-1} H_s(0)$. By (9.2), the least nonnegative residue

$$u - 2^\ell \left\lfloor \frac{u}{2^\ell} \right\rfloor = N(\mathcal{P})$$

is the exact count. Reusing the same workspace for successive coordinates, all ℓ interpolations, the sum and the remainder use $\mathrm{poly}(M_H, B_{\mathrm{data}}, \log(\Omega+1), \ell)$ bit operations and polynomially many bits of working space.

B.4. **Arithmetic for the technique overview.** We also justify the bit-complexity and space bounds for the bounded-occurrence algorithm in Section 6. Lemma 8.7 bounds the bit lengths of the replacement coefficients by a polynomial in n. Solving the polynomial-size rational linear systems in Section 6.4 gives measurement coefficients $\vec{\lambda}_{e,v,h}$ of polynomial bit length as well. Let δ be the product of the denominators of all these coefficients and all replacement coefficients, over all evaluation points. There are only polynomially many, so δ also has polynomial bit length. Every clause product uses at most $M \leqslant 3n$ linear inputs; consequently, all coefficients in the restricted sums can be accumulated with common denominator δ^{3n}. Products of polynomially many input coefficients of polynomial bit length, and sums of exponentially many such terms over this common denominator, still have polynomial bit length. The exact Hermite interpolation above therefore has polynomial bit cost for these Taylor data as well. Finally, the enumerations can be performed sequentially, retaining only the construction, current indices, coefficient arrays, running sums, and the polynomially many Taylor coefficients needed for interpolation. This gives polynomial space and only polynomial bit-operation overhead.

Appendix C. Additional Consequences of Refuting ETH

We first prove Corollaries 1.2 and 1.3. Theorem 1.1 also gives a circuit lower bound and subexponential-time algorithms for inverting Boolean maps with constant output locality. The fixed-density case gives the quantitative circuit lower bound in Corollary 1.4. Both circuit arguments use only the decision version of our 3-SAT algorithm. Exact preimage counting uses its full counting guarantee.

C.1. **Approximating Max-k-SAT and counting in graphs.**

Proof of Corollary 1.2. We may assume $m > 0$. Let $f(x)$ be the fraction of clauses satisfied by x, and let $\widehat{f}(x)$ be that fraction in a sample of $N = K(n+1)$ clauses chosen independently and uniformly with replacement, where $K = \lceil 8/\varepsilon^2 \rceil$ is a fixed integer. Hoeffding's inequality [AW15, Lemma 3.3], applied to the satisfaction bits and their complements, bounds the failure probability for one assignment by $2e^{-N\varepsilon^2/2}$. A union bound over all 2^n assignments gives, with probability $1 - 2^{-\Omega(n)}$,

$$|\widehat{f}(x) - f(x)| \leqslant \varepsilon/2 \qquad \text{for every } x \in \{0,1\}^n.$$

Thus, if $\widehat{x}$ maximizes $\widehat{f}$, then $f(\widehat{x}) \geqslant \max_x f(x) - \varepsilon$. Given an integer threshold b, a Boolean circuit with at most two inputs per gate and size $O_{k,\varepsilon}(n)$ tests whether an assignment satisfies at least b sampled clauses. Compute the N clause bits and add them in a balanced binary tree. Pad the list of clause bits with zeros to the next power of two. At level j there are $O(N/2^j)$ additions of $O(j)$-bit integers, each using $O(j)$ gates; their total size is $O(N \sum_{j \geqslant 1} j/2^j) = O(N)$. Comparing the sum with b uses $O(\log(N+1))$ more gates. The standard circuit-to-3-CNF reduction [Bar23, Lemmas 15.9–15.10] produces a formula with $O_{k,\varepsilon}(n)$ variables and clauses that is satisfiable exactly when this threshold can be attained. For a gate $z = g(u,v)$, a direct version uses, for each $(a,c) \in \{0,1\}^2$, the clause

$$(u \neq a) \ \vee \ (v \neq c) \ \vee \ (z = g(a,c)). \tag{C.1}$$

Each parenthesized expression is one literal or a constant. The four clauses force exactly the gate's truth table. Introducing one variable per gate and requiring output one gives a unique extension of each accepting input, by induction through the gates. Theorem 1.1, binary search on the threshold, and self-reduction then find an exact sample optimizer within the stated bounds. Only the fixed-density bound of Theorem 1.1 is used, for formulas with $O_{k,\varepsilon}(n)$ variables and clauses, so the

exponent obtained is $O_{k,\varepsilon}(n/\sqrt{\log n})$, which is $O(\rho(n))$; the interpolation algorithm of Sections 6–10 is not needed. The N clause indices are drawn uniformly from $[m]$ by binary rejection sampling with fair coins. Truncating it after $O(n+\log(n/\varepsilon^2))$ trials per index gives worst-case $\mathrm{poly}(n, m, \varepsilon^{-1})$ sampling time and adds at most $2^{-\Omega(n)}$ to the failure probability. □

Proof of Corollary 1.3. For $G = (V, E)$, introduce a Boolean variable x_u for each $u \in V$. Satisfying assignments of

$$\bigwedge_{\{u,w\}\in E} (\neg x_u \vee \neg x_w)$$

correspond bijectively to the independent sets $\{u \in V : x_u = 1\}$. For proper q-colorings, use one Boolean variable per vertex and color, require exactly one color at each vertex, and forbid equal colors at adjacent vertices. This gives a counting-preserving q-CNF encoding with qv variables and $O_q(v+e)$ clauses.

For perfect matchings, use one Boolean variable per edge and a linear-size circuit checking that exactly one selected edge meets each vertex. For incident bits $z_1, \dots, z_d$, maintain bits s_j and t_j for at least one and at least two selected edges among the first j bits:

$$s_0 = t_0 = 0, \qquad s_j = s_{j-1} \vee z_j, \qquad t_j = t_{j-1} \vee (s_{j-1} \wedge z_j).$$

The vertex test is $s_d \wedge \neg t_d$, including value zero when $d = 0$. Summing the degrees gives $2e$, so combining the vertex tests uses $O(v+e)$ gates. The circuit-to-3-CNF encoding [Bar23, Lemmas 15.9–15.10], in the form (C.1), gives $O(v+e)$ variables and clauses, with unique gate values for each accepted edge set. It therefore preserves the number of perfect matchings. Theorem 1.1 gives the stated time and space bounds. □

C.2. **Circuit lower bounds for E^{NP}.** For the qualitative bound below, write $\mathrm{SIZE}(O(n))$ for the languages having nonuniform circuits with $O(n)$ bounded-fan-in AND, OR, and NOT gates.

Corollary C.1. *We have*

$$\mathrm{E}^{\mathrm{NP}} \not\subseteq \mathrm{SIZE}(O(n)).$$

Proof. Williams proved that if ETH is false, then E^{NP} does not have linear-size circuits [Wil13, Theorem 6.1]. Apply this implication to the deterministic subexponential-time 3-SAT algorithm supplied by Theorem 1.1. □

The connection is an instance of deriving circuit lower bounds from faster satisfiability algorithms. To encode a Boolean circuit with s gates and n inputs, introduce a variable for each gate, enforce its value from its inputs, and add a unit clause requiring the output bit to equal 1. The resulting 3-CNF has $O(n+s)$ variables and clauses and is satisfiable exactly when the circuit is. Our theorem therefore decides satisfiability of linear-size circuits in $2^{o(n)}$ time. Williams's argument combines such a speedup with hypothetical small circuits for witnesses to contradict a nondeterministic time hierarchy. The corollary gives a language in E^{NP} without a linear-size circuit family; it does not assert a superpolynomial circuit lower bound for NP.

Proof of Corollary 1.4. We use Williams's witness-circuit argument [Wil13, Theorem 6.1] with the local reduction of Jahanjou, Miles, and Viola [JMV13, Theorem 1 and Section 6].

By Žák's nondeterministic time hierarchy [Žák83], in the form stated in [Wil13, Theorem 2.1 in the full version], fix a unary language $H \in \mathrm{NTIME}(2^n) \setminus \mathrm{NTIME}(2^{n/2})$ on multitape Turing machines. Its time-constructibility hypothesis holds for these exponential bounds, and $2^{(n+1)/2} = o(2^n)$ is the separation required there. View the nondeterministic choices of its fixed machine as a certificate y, padded to the time allowance, so the verifier has the input form (x, y) required by the local reduction. We use the local reduction in the following form [JMV13, Theorem 1]. For a machine running in time $T = T(n) \geqslant n$ on inputs (x, y) with $|x| = n$, given $x \in \{0,1\}^n$ one can output in $\mathrm{poly}(n, \log T)$ time a decoder $D : \{0,1\}^q \to \{0,1\}^{3v+3}$ that maps each index to a clause of a 3-CNF in v-bit

variables, where $v = \Theta(q)$; this 3-CNF is satisfiable exactly when the machine accepts (x, y) for some $y \in \{0,1\}^T$. Moreover, for every $r \leqslant n$ one can have $q = \max(\log_2 T, n/r) + O(\log n) + O(\log\log T)$, with every output bit of D a decision tree of depth $O(\log r)$. For all sufficiently large n apply this to the fixed machine on 1^n with locality parameter $r = 2$. A time allowance $T = 2^n n^{O(1)}$ includes the polylogarithmic overhead of simulating the machine in the reduction and gives $q = n + O(\log n)$ clause-index bits, and construction takes polynomial time in n. The decoder gives three variable labels and their signs on each index. Let φ_n consist of the clauses decoded on all 2^q indices. It is satisfiable exactly when $1^n \in H$.

Each of the decoder's $3v + 3 = O(n)$ output bits is a decision tree of depth $O(\log 2) = O(1)$, so it depends on at most a fixed number w of index bits: a depth-h binary tree queries at most $2^h - 1$ distinct coordinates. For an output bit f, declare coordinate j relevant exactly when some pair $z, z \oplus e_j$, both of Hamming weight at most w, has different f-values; here e_j has a single one in position j. Every such pair witnesses genuine dependence. Conversely, if f depends on j, take a pair $z, z \oplus e_j$ with different f-values, and set to zero every bit outside the set of at most w coordinates on which f depends; this set contains j. The f-values are unchanged and both weights are at most w, so the enumeration finds j. Thus $\sum_{h=0}^{w} \binom{q}{h} = q^{O(w)}$ decoder evaluations recover all supports. Evaluating assignments supported on each recovered set gives its truth table. Each is a Boolean function of at most w bits, implementable by $O_w(1)$ fan-in-two gates, giving $O(n)$ gates for the entire decoder in polynomial construction time.

Pad each variable label to dn bits for a fixed positive integer d and declare all 2^{dn} variables. If $|i| = dn$ for such an n and φ_n is satisfiable, let $W(i)$ be bit i of its lexicographically first satisfying assignment, ordering variables by their labels. Set $W(i) = 0$ in all other cases. The length of i determines n. To decide W, write out the 2^q clauses, ask an NP oracle whether the formula is satisfiable, and, if so, recover the first assignment by asking whether its current prefix followed by zero has a satisfying extension. At most 2^{dn} prefix queries of length $2^{O(n)}$ suffice, so $W \in \mathrm{E}^{\mathrm{NP}}$.

Put $a(t) = \sqrt{\log_2 t}$. Testing whether the count in the constant-density part of Theorem 1.1 is positive decides 3-SAT with V variables and at most $5V$ clauses in time $V^b 2^{AV/a(V)}$ for fixed $A, b > 0$. We absorb the change of logarithm base in A. The solver uses polynomial space, so a multitape simulation can scan its workspace for each bit access at a cost $V^{O(1)}$, absorbed in b.

Fix a positive rational $\eta < 1/(12A)$ and put $s_n = \lfloor \eta n \lfloor a(n) \rfloor \rfloor$ for $n \geqslant 2$. The threshold is computable using the binary length of n and the integer square root of $\lfloor \log_2 n \rfloor$. Suppose that $C_W(dn) \leqslant s_n$ for all sufficiently large n. To decide H, reject nonunary inputs. On 1^n, guess a circuit on dn index bits with at most s_n gates. Feed the decoder's three variable labels to three copies of this circuit and test whether the indexed clause is violated. Accept if the test circuit is unsatisfiable. When $1^n \in H$, a circuit computing W at length dn gives an accepting guess. When $1^n \notin H$, every guess violates a clause.

The test circuit has q inputs and at most $3s_n + O(n)$ gates. The encoding above uses one variable per input and gate, at most four clauses of width at most three per gate to enforce its truth table, and one unit clause to pin the output. Padding with unused variables gives $U_n = 3s_n + Bn$ variables and at most $5U_n$ clauses for a fixed integer B. Since $U_n = (3\eta + o(1))na(n)$ and $a(U_n) \sim a(n)$, the test takes $2^{(3A\eta + o(1))n}$ time. Guessing and building the circuits take polynomial time, so H is decidable nondeterministically in time $2^{n/2}$ for all sufficiently large n. The finitely many exceptional lengths can be hardwired, contradicting the choice of H. Thus $C_W(dn) > s_n$ for infinitely many n. Since $s_n \sim \eta n \sqrt{\log_2 n}$ and d is fixed, taking $L = W$ and any $0 < c < \eta/(d\sqrt{\log 2})$ proves (1.2). □

C.3. **Inverting Boolean maps with constant output locality.** Fix a constant d. Consider a Boolean map $G : \{0,1\}^n \to \{0,1\}^m$ whose ith output bit has the form

$$G(\vec{x})_i = g_i(\vec{x}|_{S_i}), \qquad S_i \subseteq [n], \quad |S_i| \leqslant d.$$

The input description of G lists the sets S_i and the truth tables of the Boolean functions g_i. Thus each output inspects only constantly many input bits, although the sets S_i may overlap arbitrarily. Given a target $\vec{b} \in \{0,1\}^m$, inversion asks for any $\vec{x}$ with $G(\vec{x}) = \vec{b}$.

Corollary C.2. *For every fixed d and $n \geqslant 20$, given G by the sets S_i and the truth tables of g_i as above, and given $\vec{b} \in \{0,1\}^m$, the number $|G^{-1}(\vec{b})|$ can be computed exactly, and a preimage can be found whenever one exists, in deterministic time*

$$(n+m)^{O_d(1)} 2^{O_d(\rho(n))}$$

and space $(n+m)^{O_d(1)}$.

Proof. For each i and each local assignment $\vec{\alpha} \in \{0,1\}^{S_i}$ with $g_i(\vec{\alpha}) \neq b_i$, include the clause

$$\bigvee_{j \in S_i} \ell_{j,\alpha_j}, \qquad \ell_{j,0} = X_j, \quad \ell_{j,1} = \neg X_j.$$

This clause forbids exactly the local assignment $\vec{\alpha}$. If S_i is empty, its constraint is constant and can be checked directly. The conjunction of the remaining clauses is a CNF on the original n declared variables, with width at most d and at most $2^d m$ clauses. Its satisfying assignments are exactly $G^{-1}(\vec{b})$; no auxiliary variables or multiplicity corrections are needed. Apply Theorem 1.1 with $k = \max\{2, d\}$; its arbitrary-density bound gives exponent $O_d(n \log\log n / \sqrt{\log n})$, which is $O(\rho(n))$, so the interpolation algorithm of Sections 6–10 is not needed. The usual variable-by-variable self-reduction finds a preimage using at most n further counting queries, which only changes the polynomial factor. Each query may be discarded before the next, preserving polynomial space. □

Conditional counts also allow an exactly uniform preimage to be sampled, when the preimage set is nonempty, with the same bound in expected time up to polynomial factors [JVV86]. In particular, the corollary applies to constant-locality versions of Goldreich's candidate one-way functions, which evaluate local predicates on overlapping subsets of input bits [Gol00]. For polynomially many output bits, it gives a $2^{o(n)}$-time inversion algorithm for every output in the image, ruling out $2^{\Omega(n)}$ inversion hardness for these families. This does not rule out one-wayness against polynomial-time algorithms.

References

[ABHS19] Amir Abboud, Karl Bringmann, Danny Hermelin, and Dvir Shabtay. *SETH-Based Lower Bounds for Subset Sum and Bicriteria Path*. 30th ACM–SIAM Symposium on Discrete Algorithms, 2019, 41–57. 6

[AW14] Amir Abboud and Virginia Vassilevska Williams. *Popular Conjectures Imply Strong Lower Bounds for Dynamic Problems*. 55th IEEE Symposium on Foundations of Computer Science, 2014, 434–443. 7

[AGP23] Josh Alman, Yunfeng Guan, and Ashwin Padaki. *Smaller Low-Depth Circuits for Kronecker Powers*. 34th ACM–SIAM Symposium on Discrete Algorithms, 2023, 4159–4187. 8

[AW15] Josh Alman and Ryan Williams. *Probabilistic Polynomials and Hamming Nearest Neighbors*. 56th IEEE Symposium on Foundations of Computer Science, 2015, 136–150. 8, 28, 66

[AW17] Josh Alman and Ryan Williams. *Probabilistic Rank and Matrix Rigidity*. 49th ACM Symposium on Theory of Computing, 2017, 641–652. 8

[AR94] Noga Alon and Yuval Roichman. *Random Cayley Graphs and Expanders*. Random Structures & Algorithms **5**(2), 1994, 271–284. 9

[BI16] Arturs Backurs and Piotr Indyk. *Which Regular Expression Patterns Are Hard to Match?* 57th IEEE Symposium on Foundations of Computer Science, 2016, 457–466. 7

[BI18] Arturs Backurs and Piotr Indyk. *Edit Distance Cannot Be Computed in Strongly Subquadratic Time (unless SETH is false)*. SIAM Journal on Computing **47**(3), 2018, 1087–1097. Conference version in STOC 2015. 7

[Bar23] Boaz Barak. *Introduction to Theoretical Computer Science*. Online textbook, 2023. Chapter 15, Lemmas 15.9–15.10. Archived 2023 edition (v0.95). 66, 67

[Bar68] Erwin H. Bareiss. *Sylvester's Identity and Multistep Integer-Preserving Gaussian Elimination*. Mathematics of Computation **22**(103), 1968, 565–578. 65

[BI13] Chris Beck and Russell Impagliazzo. *Strong ETH Holds for Regular Resolution*. 45th ACM Symposium on Theory of Computing, 2013, 487–494. 6

[BZ10] Richard P. Brent and Paul Zimmermann. *Modern Computer Arithmetic*. Cambridge University Press, 2010. Author version 0.5.9, October 7, 2010. 14

[BT94] Richard Beigel and Jun Tarui. *On ACC*. Computational Complexity **4**(4), 1994, 350–366. 3, 23

[BGKM23] Vishwas Bhargava, Sumanta Ghosh, Mrinal Kumar, and Chandra Kanta Mohapatra. *Fast, Algebraic Multivariate Multipoint Evaluation in Small Characteristic and Applications*. Journal of the ACM **70**(6), 2023, 42:1–42:46. Conference version in STOC 2022. 8

[Bjö16] Andreas Björklund. *Below All Subsets for Some Permutational Counting Problems*. 15th Scandinavian Symposium and Workshops on Algorithm Theory, LIPIcs **53**, 2016, 17:1–17:11. 8

[BKW19] Andreas Björklund, Petteri Kaski, and Ryan Williams. *Generalized Kakeya Sets for Polynomial Evaluation and Faster Computation of Fermionants*. Algorithmica **81**, 2019, 4010–4028. Conference version in IPEC 2017. 8

[Bri14] Karl Bringmann. *Why Walking the Dog Takes Time: Fréchet Distance Has No Strongly Subquadratic Algorithms Unless SETH Fails*. 55th IEEE Symposium on Foundations of Computer Science, 2014, 661–670. 7

[BK15] Karl Bringmann and Marvin Künnemann. *Quadratic Conditional Lower Bounds for String Problems and Dynamic Time Warping*. 56th IEEE Symposium on Foundations of Computer Science, 2015, 79–97. 7

[Cal09] Chris Calabro. *The Exponential Complexity of Satisfiability Problems*. Ph.D. dissertation, University of California, San Diego, 2009. Chapter 5. 9, 38

[CIP06] Chris Calabro, Russell Impagliazzo, and Ramamohan Paturi. *A Duality between Clause Width and Clause Density for SAT*. 21st IEEE Conference on Computational Complexity, 2006, 252–260. 8, 38, 59

[CGI+16] Marco L. Carmosino, Jiawei Gao, Russell Impagliazzo, Ivan Mihajlin, Ramamohan Paturi, and Stefan Schneider. *Nondeterministic Extensions of the Strong Exponential Time Hypothesis and Consequences for Non-reducibility*. 7th Innovations in Theoretical Computer Science Conference, 2016, 261–270. 6

[CCK+20] Parinya Chalermsook, Marek Cygan, Guy Kortsarz, Bundit Laekhanukit, Pasin Manurangsi, Danupon Nanongkai, and Luca Trevisan. *From Gap-Exponential Time Hypothesis to Fixed Parameter Tractable Inapproximability: Clique, Dominating Set, and More*. SIAM Journal on Computing **49**(4), 2020, 772–810. Conference version in FOCS 2017. 6

[CW21] Timothy M. Chan and R. Ryan Williams. *Deterministic APSP, Orthogonal Vectors, and More: Quickly Derandomizing Razborov–Smolensky*. ACM Transactions on Algorithms **17**(1), Article 2, 2021, 1–14. Authors' manuscript, Theorems 1.6 and 2.5. 6, 23

[CHKX06] Jianer Chen, Xiuzhen Huang, Iyad A. Kanj, and Ge Xia. *Strong Computational Lower Bounds via Parameterized Complexity*. Journal of Computer and System Sciences **72**(8), 2006, 1346–1367. 6

[CLWYY25] Zongchen Chen, Aditya Lonkar, Chunyang Wang, Kuan Yang, and Yitong Yin. *Counting Random k-SAT near the Satisfiability Threshold*. 57th ACM Symposium on Theory of Computing, 2025, 867–878. Theorem 1.2 is numbered as in the authors' full version. 5

[Coo71] Stephen A. Cook. *The Complexity of Theorem-Proving Procedures*. 3rd ACM Symposium on Theory of Computing, 1971, 151–158. 1

[Cur18] Radu Curticapean. *Block Interpolation: A Framework for Tight Exponential-Time Counting Complexity*. Information and Computation **261**, Part 2, 2018, 265–280. Conference version in ICALP 2015. 6

[CDL+16] Marek Cygan, Holger Dell, Daniel Lokshtanov, Dániel Marx, Jesper Nederlof, Yoshio Okamoto, Ramamohan Paturi, Saket Saurabh, and Magnus Wahlström. *On Problems as Hard as CNF-SAT*. ACM Transactions on Algorithms **12**(3), Article 41, 2016, 1–24. 6, 7

[DHM+14] Holger Dell, Thore Husfeldt, Dániel Marx, Nina Taslaman, and Martin Wahlén. *Exponential Time Complexity of the Permanent and the Tutte Polynomial*. ACM Transactions on Algorithms **10**(4), Article 21, 2014, 1–32. 3, 6, 8, 38, 59

[Din16] Irit Dinur. *Mildly Exponential Reduction from Gap 3SAT to Polynomial-Gap Label-Cover*. Electronic Colloquium on Computational Complexity, Report TR16-128, 2016. 6

[DL20] Zeev Dvir and Allen Liu. *Fourier and Circulant Matrices are Not Rigid*. Theory of Computing **16**(20), 2020, 1–48. 8

[vzGG13] Joachim von zur Gathen and Jürgen Gerhard. *Modern Computer Algebra*. 3rd edition, Cambridge University Press, 2013. 14, 18

[Gol00] Oded Goldreich. *Candidate One-Way Functions Based on Expander Graphs*. Electronic Colloquium on Computational Complexity, Report TR00-090, 2000. 69

[Gol08] Oded Goldreich. *P, NP, and NP-Completeness: The Basics of Computational Complexity*. Author's draft, June 14, 2008, Proposition 4.7. The Cambridge University Press edition appeared in 2010. 38

[HKZZ19] Thomas Dueholm Hansen, Haim Kaplan, Or Zamir, and Uri Zwick. *Faster k-SAT Algorithms Using Biased-PPSZ*. 51st ACM Symposium on Theory of Computing, 2019, 578–589. 5

[Her14] Timon Hertli. *3-SAT Faster and Simpler—Unique-SAT Bounds for PPSZ Hold in General*. SIAM Journal on Computing **43**(2), 2014, 718–729. Conference version in FOCS 2011. 5

[Hus11] Thore Husfeldt. *Invitation to Algorithmic Uses of Inclusion–Exclusion*. 38th International Colloquium on Automata, Languages, and Programming, Part II, Lecture Notes in Computer Science **6756**, 2011, 42–59. 7

[IMP12] Russell Impagliazzo, William Matthews, and Ramamohan Paturi. *A Satisfiability Algorithm for AC^0*. 23rd ACM–SIAM Symposium on Discrete Algorithms, 2012, 961–972. Publisher's version. 6

[IP01] Russell Impagliazzo and Ramamohan Paturi. *On the Complexity of k-SAT*. Journal of Computer and System Sciences **62**(2), 2001, 367–375. 1, 5, 6

[IPZ01] Russell Impagliazzo, Ramamohan Paturi, and Francis Zane. *Which Problems Have Strongly Exponential Complexity?* Journal of Computer and System Sciences **63**(4), 2001, 512–530. 1, 6, 8, 38

[JMV13] Hamid Jahanjou, Eric Miles, and Emanuele Viola. *Local Reductions*. Electronic Colloquium on Computational Complexity, Report TR13-099, revision 2, 2013. Conference version in ICALP 2015, 749–760. 67

[JVV86] Mark R. Jerrum, Leslie G. Valiant, and Vijay V. Vazirani. *Random Generation of Combinatorial Structures from a Uniform Distribution*. Theoretical Computer Science **43**(2–3), 1986, 169–188. 4, 69

[JC26] Tao Jiang and Shaowei Cai. *A Better Analysis For PPSZ For 3-SAT*. arXiv:2607.10697, 2026. 5

[Koi06] Mikko Koivisto. *Optimal 2-Constraint Satisfaction via Sum–Product Algorithms*. Information Processing Letters **98**(1), 2006, 24–28. 7

[KSY14] Swastik Kopparty, Shubhangi Saraf, and Sergey Yekhanin. *High-Rate Codes with Sublinear-Time Decoding*. Journal of the ACM **61**(5), 2014, 28:1–28:20. 8

[KW16] Ioannis Koutis and Ryan Williams. *Algebraic Fingerprints for Faster Algorithms*. Communications of the ACM **59**(1), 2016, 98–105. 7

[Kut26] Konstantin Kutzkov. *Complementary Branching for #SAT: Bounded-Degree Bounds and Faster #3-SAT*. 35th International Joint Conference on Artificial Intelligence, 2026, 2282–2290. 6

[Li21] Ray Li. *Settling SETH vs. Approximate Sparse Directed Unweighted Diameter (up to (NU)NSETH)*. 53rd ACM Symposium on Theory of Computing, 2021, 1684–1696. 6

[Liu18] Sixue Liu. *Chain, Generalization of Covering Code, and Deterministic Algorithm for k-SAT*. 45th International Colloquium on Automata, Languages, and Programming, LIPIcs **107**, 2018, 88:1–88:13. 5

[LQ15] Henri Lombardi and Claude Quitté. *Commutative Algebra: Constructive Methods. Finite Projective Modules*. Algebra and Applications 20, Springer, 2015. English translation by Tania K. Roblot; corrected author version, arXiv:1605.04832v4, September 18, 2024. 14

[LMS18] Daniel Lokshtanov, Dániel Marx, and Saket Saurabh. *Known Algorithms on Graphs of Bounded Treewidth Are Probably Optimal*. ACM Transactions on Algorithms **14**(2), Article 13, 2018, 1–30. Conference version in SODA 2011. 6, 7

[MR17] Pasin Manurangsi and Prasad Raghavendra. *A Birthday Repetition Theorem and Complexity of Approximating Dense CSPs*. 44th International Colloquium on Automata, Languages, and Programming, LIPIcs **80**, 2017, 78:1–78:15. 6

[MS08] Igor L. Markov and Yaoyun Shi. *Simulating Quantum Computation by Contracting Tensor Networks*. SIAM Journal on Computing **38**(3), 2008, 963–981. The cited numbering is that of arXiv:quant-ph/0511069v7, 2009. 7

[Mar10] Dániel Marx. *Can You Beat Treewidth?* Theory of Computing **6**, 2010, 85–112. 6

[MJK21] Johannes Middeke, David J. Jeffrey, and Christoph Koutschan. *Common Factors in Fraction-Free Matrix Decompositions*. Mathematics in Computer Science **15**, 2021, 589–608. The cited numbering is that of the author preprint, arXiv:2005.12380, May 25, 2020. 65

[NN93] Joseph Naor and Moni Naor. *Small-Bias Probability Spaces: Efficient Constructions and Applications*. SIAM Journal on Computing **22**(4), 1993, 838–856. 9, 28

[Ope26] OpenAI, *The Unique Games Theorem*, OpenAI Math Release preprint OAI:The-Unique-Games-Theorem-September-23-2026, 2026. 4

[PPSZ05] Ramamohan Paturi, Pavel Pudlák, Michael E. Saks, and Francis Zane. *An Improved Exponential-Time Algorithm for k-SAT*. Journal of the ACM **52**(3), 2005, 337–364. Conference version in FOCS 1998. 5

[Rag88] Prabhakar Raghavan. *Probabilistic Construction of Deterministic Algorithms: Approximating Packing Integer Programs*. Journal of Computer and System Sciences **37**(2), 1988, 130–143. 9, 16, 30

[Rag08] Prasad Raghavendra. *Optimal Algorithms and Inapproximability Results for Every CSP?* 40th ACM Symposium on Theory of Computing, 2008, 245–254. 4

[RV13] Liam Roditty and Virginia Vassilevska Williams. *Fast Approximation Algorithms for the Diameter and Radius of Sparse Graphs*. 45th ACM Symposium on Theory of Computing, 2013, 515–524. 7

[Sch24] Dominik Scheder. *PPSZ is better than you think*. TheoretiCS **3**, Article 5, 2024, 1–37. 5

[Sch99] Uwe Schöning. *A Probabilistic Algorithm for k-SAT and Constraint Satisfaction Problems*. 40th IEEE Symposium on Foundations of Computer Science, 1999, 410–414. 5

[Sho09] Victor Shoup. *A Computational Introduction to Number Theory and Algebra*. Second edition, Cambridge University Press, 2009. Author version 2, June 16, 2008. 14

[SS09] Alexander D. Scott and Gregory B. Sorkin. *Polynomial Constraint Satisfaction Problems, Graph Bisection, and the Ising Partition Function*. ACM Transactions on Algorithms **5**(4), 2009, 45:1–45:27. 8

[SKC94] Bart Selman, Henry A. Kautz, and Bram Cohen. *Noise Strategies for Improving Local Search*. 12th National Conference on Artificial Intelligence, 1994, 337–343. 5

[SS96] Michael Sipser and Daniel A. Spielman. *Expander Codes*. IEEE Transactions on Information Theory **42**(6), 1996, 1710–1722. 3, 9, 27

[Tan81] R. Michael Tanner. *A Recursive Approach to Low Complexity Codes*. IEEE Transactions on Information Theory **27**(5), 1981, 533–547. 9, 27

[Tov84] Craig A. Tovey. *A Simplified NP-Complete Satisfiability Problem*. Discrete Applied Mathematics **8**(1), 1984, 85–89. 38

[Val77] Leslie G. Valiant. *Graph-Theoretic Arguments in Low-Level Complexity*. Mathematical Foundations of Computer Science 1977, Lecture Notes in Computer Science **53**, 1977, 162–176. 8

[Val79] Leslie G. Valiant. *The Complexity of Enumeration and Reliability Problems*. SIAM Journal on Computing **8**(3), 1979, 410–421. 6

[VW19] Virginia Vassilevska Williams. *On Some Fine-Grained Questions in Algorithms and Complexity*. Proceedings of the International Congress of Mathematicians (ICM 2018), World Scientific, 2019, 3447–3487. 6, 7

[VW21] Nikhil Vyas and Ryan Williams. *On Super Strong ETH*. Journal of Artificial Intelligence Research **70**, 2021, 473–495. 5

[Wah08] Magnus Wahlström. *A Tighter Bound for Counting Max-Weight Solutions to 2SAT Instances*. 3rd International Workshop on Parameterized and Exact Computation, Lecture Notes in Computer Science **5018**, 2008, 202–213. 6

[Wil05] Ryan Williams. *A New Algorithm for Optimal 2-Constraint Satisfaction and Its Implications*. Theoretical Computer Science **348**(2–3), 2005, 357–365. 7

[Wil13] Ryan Williams. *Improving Exhaustive Search Implies Superpolynomial Lower Bounds*. SIAM Journal on Computing **42**(3), 2013, 1218–1244. Conference version in STOC 2010. The numbering of Lemma 3.2 and Theorem 6.1 follows the author's full version. 67

[Wil14] Richard Ryan Williams. *The Polynomial Method in Circuit Complexity Applied to Algorithm Design (Invited Talk)*. 34th International Conference on Foundation of Software Technology and Theoretical Computer Science, LIPIcs **29**, 2014, 47–60. 7

[Wil16] Richard Ryan Williams. *Strong ETH Breaks With Merlin and Arthur: Short Non-Interactive Proofs of Batch Evaluation*. 31st Conference on Computational Complexity, LIPIcs **50**, 2016, 2:1–2:17. 6

[WY07] David Woodruff and Sergey Yekhanin. *A Geometric Approach to Information-Theoretic Private Information Retrieval*. SIAM Journal on Computing **37**(4), 2007, 1046–1056. 8

[Žák83] Stanislav Žák. *A Turing Machine Time Hierarchy*. Theoretical Computer Science **26**(3), 1983, 327–333. 67